GUJARAT TECHNOLOGICAL UNIVERSITY

Program Name: Engineering

Level: Diploma

Branch: Information & Communication Technology

Course / Subject Code : DI03032021

Course / Subject Name : Java Programming

w. e. f. Academic Year:	2024-25
Semester:	3rd
Category of the Course:	PCC

Prerequisite:	Basic computer programming concepts: Students should have a foundational understanding of programming principles, including variables, control structures, data types, and basic algorithms. Familiarity with any programming language, such as C or Python, is recommended.
Rationale:	This course is designed to teach object-oriented programming concepts, techniques, and applications using the Java programming language. Java is a simple, portable, distributive, robust, secure, dynamic, architecture-neutral, object-oriented programming language that enables development of applications for diverse computing platforms. The aim of this course is for students to master platform-independent object-oriented programming with Java as the foundation for advanced technologies like web applications, cloud computing, and enterprise systems. Students will learn to design, implement, test, and debug Java applications using object-oriented principles. The course emphasizes hands-on programming experience through practical exercises and projects that reinforce theoretical concepts and develop problem-solving abilities.

Course Outcome:

After Completion of the Course, Student will able to:

No	Course Outcomes	RBT Level
01	Write simple Java programs using basic syntax, data types, and control structures.	R,U,A
02	Apply object-oriented programming concepts to solve real-world problems.	R,U,A
03	Develop Java applications using inheritance, interfaces, and packages.	R,U,A
04	Implement exception handling and multithreading in Java programs.	R,U,A
05	Develop Java applications using file handling and collections framework.	R,U,A

**Revised Bloom's Taxonomy (RBT)*

GUJARAT TECHNOLOGICAL UNIVERSITY

Program Name: Engineering

Level: Diploma

Branch: Information & Communication Technology

Course / Subject Code : DI03032021

Course / Subject Name : Java Programming

Teaching and Examination Scheme:

Teaching Scheme (in Hours)			Total Credits L+T+ (PR/2)	Assessment Pattern and Marks				Total Marks
				Theory		Tutorial / Practical		
L	T	PR	C	ESE (E)	PA(M)	PA(I)	ESE (V)	
3	0	2	4	70	30	20	30	150

Course Content:

Unit No.	Content	No. of Hours	% of Weightage
1.	**1. Introduction to Java Programming** 1.1 Explain Java features and platform independence. 1.1.1 Introduction to Java and history 1.1.2 Features of Java and JVM architecture 1.1.3 Java applications and platform independence 1.2 Install and configure Java development environment. 1.2.1 JDK, JRE, and JVM components 1.2.2 Java environment setup and tools 1.3 Write, compile, and execute simple Java programs. 1.3.1 Structure of Java program 1.3.2 Compilation and execution process 1.3.3 Comments and documentation 1.4 Apply data types, variables, and operators in Java programs. 1.4.1 Primitive data types and variables 1.4.2 Type conversion and casting 1.4.3 Operators (arithmetic, relational, logical, bitwise) 1.4.4 Operator precedence and associativity	09	20
2.	**2. Object-Oriented Programming Concepts** 2.1. Explain object-oriented programming principles. 2.1.1. Object-oriented programming fundamentals 2.1.2. Class, object, encapsulation, inheritance, polymorphism 2.1.3. Procedural vs. Object-oriented programming 2.2. Create classes and objects in Java. 2.2.1. Class definition and object creation 2.2.2. Memory allocation for objects 2.2.3. Object references and garbage collection 2.3. Apply encapsulation through access modifiers. 2.3.1. Access modifiers (public, private, protected, default)	09	20

	2.3.2. Data hiding and encapsulation principles 2.3.3. Getter and setter methods 2.4. Implement constructors and methods in Java programs. 2.4.1. Methods and parameter passing 2.4.2. 'this' keyword and its usage 2.4.3. Constructor types and chaining 2.5. Use method overloading and static components. 2.5.1. Method overloading principles 2.5.2. Constructor overloading 2.5.3. Static variables, methods, and blocks 2.5.4. Final keyword with variables, methods, and classes		
3.	**3. Inheritance, Interfaces, and Packages** 3.1. Implement inheritance in Java programs. 3.1.1. Inheritance concepts and types 3.1.2. Extending classes with 'extends' keyword 3.1.3. Super keyword and constructor chaining 3.1.4. Types of inheritance (single, multilevel, hierarchical) 3.2. Apply method overriding for polymorphism. 3.2.1. Method overriding principles 3.2.2. Dynamic method dispatch 3.2.3. Object class and its methods 3.2.4. toString(), equals(), hashCode() methods 3.3. Create and implement interfaces. 3.3.1. Interface definition and implementation 3.3.2. Multiple inheritance through interfaces 3.3.3. Default and static methods in interfaces 3.4. Develop abstract classes for partial implementation. 3.4.1. Abstract classes and methods 3.4.2. Concrete methods in abstract classes 3.4.3. Abstract class vs. Interface 3.5. Organize code using packages. 3.5.1. Package creation and naming conventions 3.5.2. Import statement and package access 3.5.3. Built-in packages in Java	09	20
4.	**4. Exception Handling and Multithreading** 4.1. Implement exception handling in Java applications. 4.1.1. Exception handling fundamentals 4.1.2. Exception hierarchy and types 4.1.3. Try, catch, finally blocks 4.1.4. Multiple catch blocks and exception propagation 4.1.5. Throw and throws keywords 4.2. Create custom exceptions for specific requirements	09	20

	4.2.1. Custom exception creation 4.2.2. Exception handling best practices 4.3. Explain multithreading concepts and benefits. 4.3.1. Multithreading concepts and benefits 4.3.2. Process vs. Thread 4.3.3. Thread lifecycle and states 4.4. Create and manage threads in Java programs 4.4.1. Thread creation approaches 4.4.2. Extending Thread class 4.4.3. Implementing Runnable interface		
5.	**5. Collections & File Handling** 5.1. Implement string manipulation in Java programs. 5.1.1. String class and its methods 5.1.2. String immutability concept 5.1.3. StringBuilder and StringBuffer 5.1.4. String operations and regular expressions 5.2. Apply Java Collections Framework for data management. 5.2.1. Collections Framework overview 5.2.2. List interface and implementations (ArrayList, LinkedList) 5.2.3. Set interface and implementations (HashSet, TreeSet) 5.2.4. Map interface and implementations (HashMap, TreeMap) 5.2.5. Iterator and enhanced for-loop 5.3. Perform file input/output operations. 5.3.1. File handling fundamentals 5.3.2. File class and operations 5.3.3. Byte streams vs. Character streams 5.3.4. Reading and writing text files 5.4. Implement serialization for object persistence. 5.4.1. Object serialization concepts 5.4.2. Serializable interface 5.4.3. ObjectInputStream and ObjectOutputStream	09	20
	Total	**45**	**100**

Suggested Specification Table with Marks (Theory):

Distribution of Theory Marks (in %)					
R Level	**U Level**	**A Level**	**N Level**	**E Level**	**C Level**
20	30	50	--	--	--

Where R: Remember; U: Understanding; A: Application, N: Analyze and E: Evaluate C: Create (as per Revised Bloom's Taxonomy)

References/Suggested Learning Resources:

(a) Books:

1. Java: The Complete Reference by Herbert Schildt, McGraw Hill Education, 12th Edition, 2021, ISBN: 9781260440232
2. Programming with Java by E Balagurusamy, McGraw Hill Education, 6th Edition, 2019, ISBN: 9789353161491
3. Core Java: Volume I - Fundamentals by Cay S. Horstmann, Prentice Hall, 11th Edition, 2018, ISBN: 9780135166307
4. Head First Java by Kathy Sierra and Bert Bates, O'Reilly Media, 3rd Edition, 2022, ISBN: 9781491910771
5. Thinking in Java by Bruce Eckel, Prentice Hall, 4th Edition, 2006, ISBN: 9780131872486

(b) Open source software and website:

- Softwares
  - Java Development Kit – JDK 23
  - IntelliJ IDEA Community Edition
  - Visual Studio Code
  - Eclipse IDE
- Official Documentation
  - Official Java Documentation
- Online Tutorials
  - W3Schools Java Tutorial
  - JavaTpoint - Java Programming Tutorials
  - GeeksforGeeks - Java Programming Language
  - Learn Java Online
  - TutorialsPoint - Java Tutorials and Examples
- Video Courses
  - Introduction to Java Programming (Udacity YouTube Playlist)
  - Programming in Java by Debasis Samanta, IIT Kharagpur (NPTEL YouTube Playlist)
  - Java Tutorials For Beginners In Hindi (CodeWithHarry YouTube Playlist)
- Comprehensive Courses
  - Java Programming Nanodegree by Udacity
  - Core Java Specialization by LearnQuest on Coursera
  - Java Masterclass 2025: 130+ Hours of Expert Lessons by Tim Bulchka on Udemy
  - Introduction to Object-Oriented Programming with Java by Georgia Tech on edX

Suggested Course Practical List:

Sr. No.	Practical Outcomes (PrOs)	Unit No.	Approx. Hrs. required
1	Install JDK, write a simple "Hello World" program, compile, and execute using Java compiler and interpreter.	I	1
2	Write a Java program to implement basic arithmetic operations and demonstrate type casting.	I	1
3	Create a Java program that demonstrates the use of decision-making and loop statements.	I	1
4	Develop a program to perform operations on one-dimensional and two-dimensional arrays.	I	1
5	Write a program that demonstrates the use of enhanced for-loop with arrays and simple lambda expressions.	I	1
6	Create a class to represent a Student with appropriate attributes and methods, then instantiate and manipulate Student objects.	II	1
7	Implement a class with proper encapsulation using private attributes and public getter/setter methods.	II	1
8	Create a program that demonstrates the use of constructors and 'this' keyword.	II	1
9	Develop a program to demonstrate method overloading for different operations.	II	1
10	Implement a class with static methods, variables, and blocks, then demonstrate their behaviour.	II	1
11	Create a program to demonstrate single, multilevel, and hierarchical inheritance.	III	1
12	Implement method overriding and demonstrate dynamic method dispatch.	III	1
13	Develop a program with an abstract class containing both abstract and concrete methods.	III	1
14	Create and implement interfaces for multiple inheritance and define default methods.	III	1
15	Develop a program that organizes classes in packages and demonstrates import statements.	III	1
16	Implement exception handling using try, catch, and finally blocks for different scenarios.	IV	1
17	Create custom exceptions and demonstrate their usage in handling specific error conditions.	IV	1
18	Develop a multithreaded application by extending Thread class to perform concurrent tasks.	IV	1
19	Implement a multithreaded program using Runnable interface and demonstrate thread states.	IV	1

20	Create a program that demonstrates thread synchronization and inter-thread communication.	IV	1
21	Develop a program to demonstrate String class methods and string manipulation operations.	V	1
22	Implement a program using ArrayList and LinkedList to demonstrate List interface capabilities.	V	1
23	Create an application that utilizes HashSet and HashMap for efficient data management.	V	1
24	Develop a program to read, write, and manipulate text files using file streams.	V	1
25	Implement object serialization to save and retrieve object states from files.	V	2
26	Develop a program that demonstrates serialization of collection objects.	V	2
27	Create a simple CRUD application combining collections, file handling, and exception handling.	Multiple	2
	Total		**30**

List of Laboratory/Learning Resources Required:

Sr. No.	Equipment Name with Broad Specifications	PrO. No.
1	Computer with latest configuration with Windows/Linux/Unix Operating System.	All
2	JDK (Java Development Kit) Version 11 or above	All
3	Any IDE - Eclipse, IntelliJ IDEA, NetBeans, Visual Studio Code	All

Suggested Project List:

A suggestive list of micro-projects is given here. Similar micro-projects could be added by the concerned course teacher:

1. **Library Management System**: Implement a system to manage books, members, and borrowing with proper data validation, exception handling, and file-based persistence.
2. **Student Information System**: Create an application that maintains student records, course enrolments, and grade calculations using inheritance for different student types.
3. **E-commerce Cart Management**: Develop a console-based shopping cart system with product categories (using inheritance), order management, and data persistence.
4. **Task Management Application**: Build a task tracker with task categories, priorities, deadlines, and status updates using collections and file handling.
5. **Quiz Management System**: Create an interactive quiz application with question types (using inheritance), scoring, timer functionality (using threads), and result storage.
6. **Personal Finance Manager**: Develop an application to track income, expenses, and budgets, with data visualization, categorization, and persistence.

7. **Address Book Application**: Implement a contact management system with search, sort, and filter capabilities using collections and file handling.

Suggested Activities for Students:

1. **Code Review Sessions**: Organize peer code review sessions where students can present their code and receive constructive feedback from their classmates. This helps in improving coding practices and learning from each other.
2. **Hackathons**: Conduct hackathons where students can work on real-world projects in teams. This encourages collaboration, problem-solving, and application of Java concepts in practical scenarios.
3. **Guest Lectures**: Invite industry experts to give guest lectures on advanced Java topics, current trends, and best practices. This provides students with insights into the professional world and emerging technologies.
4. **Coding Challenges**: Introduce weekly coding challenges that focus on specific Java concepts. These challenges can be solved individually or in groups, promoting continuous learning and practice.
5. **Project Showcases**: Organize project showcase events where students can present their completed projects to the class. This fosters a sense of accomplishment and allows students to learn from each other's work.
6. **Online Forums**: Create online forums or discussion boards where students can ask questions, share resources, and discuss Java-related topics. This provides a platform for continuous engagement and support.

* * * * * * *

History of Java

- JAVA શરૂઆતમાં 1991 માં develop કરવામાં આવી ત્યારે તેને "OAK" નામ આપવામાં આવ્યું હતું, પરંતુ તેનું નામ બદલીને "JAVA" 1995 માં કરવામાં આવ્યું હતું.
- Java ને નાના embedded systems device માટે બનાવવામાં આવી હતી જેવા કે સેટ-ટોપ બોક્સ.
- Platform-independent language બનાવી એ primary motivation હતું. જેનો use કરી ને કોઈપણ consumer electronic devices માં સોફ્ટવેર એમ્બડે થાય છે.
- JAVA પ્રોગ્રામીંગ ભાષા મૂળ રીતે સન માઈક્રોસિસ્ટમ્સ દ્વારા વિકસાવવામાં આવી હતી જે જેમ્સ ગોસ્લિંગ દ્વારા શરૂ કરવામાં આવી હતી અને ૧૯૯૫ માં સન માઈક્રોસિસ્ટમ્સના કોર component તરીકે રજૂ કરવામાં આવી હતી. (Java 1.0 [J2SE]).
- Write Once, Run Anywhere (WORA), popular platforms પર no-cost run-times આપે છે.
- Java 2, નવા વર્ઝન વિવિધ પ્રકારનાં પ્લેટફોર્મ્સ માટે બનેલા છે. J2EE એ એન્ટરપ્રાઈઝ એપ્લિકેશન માટે technologies અને API નો સમાવેશ કરે છે. સામાન્ય રીતે server એન્વાયરમેન્ટ્સમાં ચાલે છે, જ્યારે J2ME API મોબાઈલ એપ્લિકેશન્સ માટે ઑપ્ટિમાઈઝ કરે છે.
- ડેસ્કટૉપ version ને J2SE નું નામ આપવામાં આવ્યું હતું. 2006 માં, માર્કેટિંગ હેતુ માટે Sun Java EE, Java ME અને Java SE અનુક્રમે નવા J2 version નામ આપવામાં આવ્યું.
- 13 નવેમ્બર, 2006 ના રોજ, Sun એ General Public License (GPL) ની હેઠળ, મોટાભાગના JAVA ને ફ્રી અને open-source software (FOSS) તરીકે જાહેર કર્યું.
- 8 મે, 2007 ના રોજ, Sun એ Java ના બધા કોડને free અને open-source બનાવ્યાં જાવા ના કોડનો એક નાનો ભાગમાં પણ Sun કૉપિરાઈટ ધરાવતો નથી.

Write and explain Features of JAVA. OR Explain advantages of JAVA.

- JAVA “Write Once, Run Anywhere” આપે છે, તે પ્લેટફોર્મ પર no-cost run-time પૂરું પાડે છે. ખૂબ secure અને configurable security આપે છે, તે નેટવર્ક અને ફાઈલ ઍક્સેસ ને allow કરે છે.

1) Simple (સિમ્પલ)

✓ તે સરળ છે કારણ કે તેમાં C અને C++ જેવી અન્ય langauge ના features હોય છે અને JAVA એ complexity ને દૂર કરે છે, કારણ કે તે પોઈન્ટર, સ્ટોરેજ Class અને goto statement નો ઉપયોગ કરતું નથી અને તે multiple Inheritance સપોર્ટ કરતું નથી.

2) Secure (સિક્યૂઅર)

✓ જ્યારે code ને એક મશીનમાંથી બીજી મશીનમાં transfer કરીએ છીએ, ત્યારે તે પ્રથમ code ચેક કરે છે વાયરસ છે કે નહીં, તે કોડની safety ચેક કરે છે, જો તેમાં વાયરસ હોય તો તે કોડને ક્યારેય ચલાવશે નહીં.

3) Object-Oriented (ઑબ્જેક્ટ - ઑરિએન્ટેડ)

✓ આપણે જાણીએ છીએ કે pure object oriented language માં તમામ કોડ classes અને objects ના રૂપમાં છે.
✓ Java નો આ feature વધારે important છે.તે કોડ reusability અને maintainability ને સપોર્ટ કરે છે

4) Robust (રોબસ્ટ)

✓ પ્રોગ્રામ failures માટે બે મુખ્ય કારણો છે:
  1. **Memory management mistake**
  2. **Mishandled exception or Run time errors**
✓ JAVA ડાયરેક્ટ પોઈન્ટર મેનીપ્યુલેશનને સપોર્ટ કરતું નથી. આ JAVA પ્રોગ્રામને memory overwrite થવા દેતું નથી.
✓ જાવા મેમરી allocation અને de-allocation પોતે manage કરે છે. de-allocation સંપૂર્ણપણે automatic છે, કારણ કે જાવા unused object માટે garbage collection આપે છે.

- ✓ JAVA ઓબ્જેક્ટ-ઑરિએન્ટેડ exception handling આપે છે. સારી રીતે લખેલા JAVA પ્રોગ્રામમાં, તમામ run-time errors પ્રોગ્રામ દ્વારા manage કરી શકાય છે.

5) Multithreaded (મલ્ટિથ્રેડેડ)

- ✓ Thread એ એક સાથે અલગ અલગ પ્રોગ્રામ execute કરે છે.
- ✓ Multiple threads નો use કરી ને આપણે એવો JAVA પ્રોગ્રામ્સ લખી શકીએ છીએ જે ઘણીબધી tasks ને execute કરે છે.
- ✓ Multithreadingનો મુખ્ય ફાયદો એ છે કે તે same memory shares કરે છે.
- ✓ મલ્ટિમીડિયા અને વેબ એપ્લિકેશન માટે Threads મહત્વપૂર્ણ છે

6) Distributed (ડિસ્ટ્રિબ્યૂટેડ)

- ✓ JAVAને ઈન્ટરનેટના distributed environment માટે ડિઝાઈન કરવામાં આવ્યું છે, કારણ કે તે TCP / IP protocols ને handle કરે છે
- ✓ JAVAમાં HTTP અને FTP જેવા વધારે ઉપયોગમાં લેવાતા protocol આવેલા છે.
- ✓ ઈન્ટરનેટ પ્રોગ્રામર્સ આ protocols use કરી ને કૉલ કરી શકે છે અને ઈન્ટરનેટ પરની કોઈ પણ રીમોટ મશીનમાંથી ફાઈલોને access કરે છે.તેના માટે લોકલ સિસ્ટમ પર કોડ લખવાની જરૂર નથી.

7) Architecture-Neutral (આર્કિટેક્ચર- ન્યૂટ્રલ)

- ✓ તેનો અર્થ એ કે એક પ્લેટફોર્મમાં લખાયેલા પ્રોગ્રામ્સ ફરીથી લખી અથવા recompile કર્યા વગર કોઈપણ અન્ય પ્લેટફોર્મ પર ચાલી શકે છે. બીજા શબ્દોમાં કહીએ તો (write once, run anywhere, anytime, forever)
- ✓ JAVA પ્રોગ્રામ bytecode ફોર્મેટમાં compile થાય છે જે કોઈપણ મશીન depended નથી. પરંતુ તે મશીન માટે JVM દ્વારા machine code માં સરળતાથી translate કરી શકાય છે.
- ✓ જ્યારે applets અથવા applications developed કરવામાં આવે ત્યારે આ ખૂબ મદદરૂપ થશે જે કોઈપણ મશીન દ્વારા ડાઉનલોડ કરવામાં આવે છે અને કોઈપણ સિસ્ટમમાં ચાલે છે.

8) Platform Independent (પ્લેટફોર્મ ઈન્ડિપેન્ડન્ટ)

- ✓ તેનો મતલબ એ છે કે જ્યારે આપણે જાવામાં પ્રોગ્રામ compile કરીએ છીએ, ત્યારે તે પ્રોગ્રામનો byte code બનાવશે અને જ્યારે પ્રોગ્રામ રન કરીશું ત્યારે byte code execute થાશે.
- ✓ JAVAમાં તે ફરજિયાત નથી, કે જે ઓપરેટિંગ સિસ્ટમમાં JAVAમાં પ્રોગ્રામ બનાવે છે, તે જ ઑપરેટિંગ સિસ્ટમમાં આપણે પ્રોગ્રામને એક્ઝેક્યુટ કરવું પડશે.

9) Interpreted (ઈન્ટરપ્રિટેડ)

- ✓ મોટાભાગની programming languages ઓ compiled અથવા interpreted હોય છે, JAVA બંને compiled અને interpreted છે.
- ✓ Java compiler જાવા source ફાઈલને byte code માં translate કરે છે અને Java interpreter translate કરેલા byte code ને સીસ્ટમ પર સીધા જ execute કરે છે જે JVM દ્વારા કરવામાં આવે છે.

10) High Performance (હાય પેર્ફોર્મન્સ)

- ✓ JAVA પ્રોગ્રામ્સને machine code ના બદલે byte code તરીકે ઓળખાતી intermediate representation માં convert કરવામાં આવે છે અને JVM કોઈપણ મશીન પર JAVA ચલાવે છે કે જેના પર JVM ઈન્સ્ટોલ કરેલું છે.
- ✓ JAVA bytecode ને ફક્ત Just-In-Time compile નો ઉપયોગ કરીને સારા performance માટે machine codeમાં translate કરવામાં આવે છે.
- ✓ તેથી, JAVA પ્રોગ્રામ્સ પ્રોગ્રામ અથવા સ્ક્રીપ્ટ્સ કરતાં વધુ ઝડપી છે જે સંપૂર્ણ રીતે interpreted languages માં લખાય છે પરંતુ C અને C++ પ્રોગ્રામ્સ કરતાં slow છે જે machine language માં compile થાય છે.

11) Dynamic (ડાઈનેમિક)

- ✓ Run time એ remote server પર રહેલા classes નો use કરી ને java એન્વાયર્નમેન્ટ extend થઈ શકે છે.
- ✓ Classes સાથે લિંક કરતી વખતે run time એ java interpreter name resolution કરે છે.
- ✓ Java interpreter મેમરીમાં ઑબ્જેક્ટની પ્લેસમેન્ટ નક્કી કરવા માટે જવાબદાર છે.

JDK (Java Development Kit)

- JAVA ડેવલપર કિટ પ્રોગ્રામ્સ develop કરવા માટે ટૂલ્સ અને JRE આપે છે.JRE નો use કરી ને programs run થાય છે.
- JDK માં compiler (javac.exe), Java application launcher (java.exe) અને Appletviewer વગેરે જેવા tools આવેલા છે ... Compiler જાવા કોડને byte code માં translate કરે છે.
- Java application launcher JRE ને ઓપન કરે છે અને ક્લાસ લોડ કરી ને તેની main method ને call કરે છે.
- JAVA programs ચલાવવા માટે JRE ની જ જરૂર પડે છે. JRE java ફાઈલોને એક્ઝેક્યુશન કરવા માટે use થાય છે
- i.e. **JRE = JVM + Java Packages Classes(like util, math, lang, awt,swing etc)+runtime libraries**.

JRE (Java Runtime Environment)

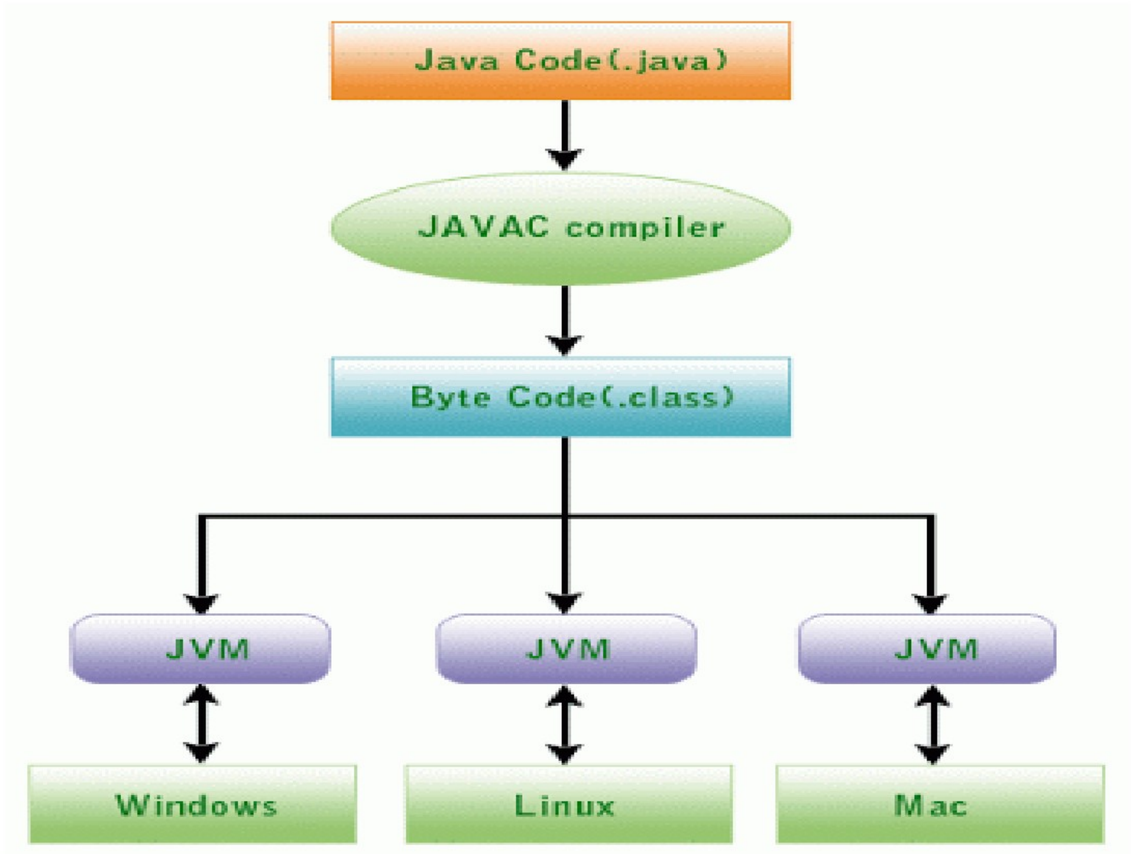
- Java Runtime Environment માં JVM, class libraries અને other supporting files હોય છે.
- તેમાં compiler, debugger વગેરે જેવા કોઈપણ development tools હોતા નથી.
- JVM પ્રોગ્રામ run કરે છે અને તે class libraries અને other supporting files નો ઉપયોગ કરે છે.

JVM (Java Virtual Machine)

- JVM ને Virtual કહેવામાં આવે છે કારણ કે તે મશીન ઈન્ટરફેસ પૂરું પાડે છે જે ઓપરેટિંગ સિસ્ટમ અને મશીન હાર્ડવેર આર્કિટેક્ચર પર આધારિત નથી.
- હાર્ડવેર અને ઑપરેટિંગ સિસ્ટમથી independent કરવાનો એક પાયો છે (WORA)
- જ્યારે આપણે જાવા ફાઈલ compile કરીએ છીએ ત્યાર આઉટપુટ '.exe' નથી પરંતુ તે '.class' ફાઈલ છે.
- '.Class' ફાઈલમાં byte codes છે જે JVM દ્વારા સમજી શકાય છે.
- JAVA Virtual મશીન bytecodeને મશીન કોડમાં ઓપરેટિંગ સિસ્ટમ અને હાર્ડવેર ના આધારે interprets કરે છે.
- Garbage collection અને array bounds checking વગેરે JVM દ્વારા થાય છે.
- JVM પોતે પ્લેટફોર્મ આધારિત છે.
- 2014 પછીના મોટાભાગના JVMs interpreter ને બદલે JIT (જસ્ટ ઈન ટાઈમ) compiler નો use કરે છે.

BYTE CODE

- Bytecode એ JAVA source code નું intermediate representation છે. જે source code ને compiler કરીને java compiler દ્વારા બનાવવામાં આવે છે.
- આ byte code એ મશીન independent code છે. તેને intermediate code પણ કહેવાય છે જે પછી JVM દ્વારા interpreted અને execute કરવામાં આવે છે.
- Bytecode JVM માટે એક મશીન કોડ છે. પરંતુ machine code એ platform specific છે જ્યારે byte code એ platform independent છે. જે તે વચ્ચેનો મુખ્ય તફાવત છે.
- તે .class ફાઈલમાં store છે જે source codeનું compiling કર્યા પછી બનાવવામાં આવે છે.



JAVA Environment Setup

- જ્યારે વિન્ડોઝ માટે path સેટ કરવો હોય ત્યારે તમે c: \ Program Files \ java \ jdk ડાયરેક્ટરીમાં જાવા ઈન્સ્ટોલ કર્યું છે.
- MY Computer પર રાઈટ-ક્લિક કરો અને 'propetry' પસંદ કરો.
- 'Advanced' ટેબ માં 'Environment variables' બટન પર ક્લિક કરો.
- ત્યારબાદ, System variables' માં 'Path' વેરિયેબલને બદલવું જેથી તેમાં Java executableનો પાથ હોય છે.
- ઉદાહરણ,જો અત્યારે path 'C:\WINDOWS\SYSTEM32' હોયતો, તેને change કરી ને 'C:\WINDOWS\SYSTEM32;c:\Program Files\java\jdk\bin' કરવો.

Describe the sample structure of JAVA program

- JAVA પ્રોગ્રામમાં ઘણા classes હોય છે કે જેમાં ફક્ત એક જ Class ને main method હોય છે
- Classesમાં data members અને methods હોય છે જે data members પર કાર્ય કરે છે.
- Methods માં data type declarations અને executable statements હોય છે.
- Java Programમાં નીચે બતાવ્યા પ્રમાણે એક અથવા વધુ sections આવેલ હોય છે.

Documentation Section	Suggested
Package Statement	Optional
Import Statements	Optional
Interface Statement	Optional
Class Definitions	Optional

Main Method class
{
Main Method Definition
}
	Essential

Documentation Section

- Documentation વિભાગમાં પ્રોગ્રામનું નામ, author અને અન્ય વિગતો comment linesમાં આપી શકાય છે, જે પ્રોગ્રામર પછી reference તરીકે use કરશે.
- Java પણ commentનો ઉપયોગ કરે છે જેમ કે / * .... * / ને documentation comment તરીકે ઓળખાય છે.
- આ ફોર્મનો documentation માટે ઉપયોગ થાય છે.

Package Statement

- Java ફાઈલમાં પ્રથમ statement એ package સ્ટેટમેન્ટ છે.
- આ statement પેકેજનું નામ declare કરે છે અને compilerને જણાવે છે કે અહીં defined કરેલા classes આ પેકેજના છે.

  Example: Package Student;

Import Statements

- Package statement પછી એક કરતા વધારે import statements હોય શકે છે. આ c માં #include statement સમાન છે.

  Example: Import student.test;

Interface Statement

- Interface એ class જેવું છે પરંતુ interface માં only method declarations હોય છે.
- Interface statement option છે અને તેનો ઉપયોગ જ્યારે આપણે પ્રોગ્રામમાં multiple inheritance implement કરવાનું હોય ત્યારે થાય છે.

Class Definition

- Java પ્રોગ્રામમાં multiple class definitions હોઈ શકે છે.
- JAVA પ્રોગ્રામમાં classes વધારે મહત્વ ધરાવે છે.
- આ classesનો ઉપયોગ real worldના ઑબ્જેક્ટ્સને map કરવા માટે થાય છે.

Main Method Class

- દરેક Java પ્રોગ્રામ માં main method હોવી જરૂરી છે. જે programના execution નો starting point છે.
- Simple java program માં ફક્ત આ એક part પણ હોય શકે છે.
- Main method વિવિધ classesના objects બનાવે છે અને તેમની વચ્ચે communication કરે છે.

Procedure-Oriented vs. Object-Oriented Programming

Procedure Oriented Programming (POP)	Object Oriented Programming (OOP)
• Procedures અને functions ને મહત્વ આપવામાં આવે છે.	• Procedures અને functions ને બદલે Dataને મહત્વ આપવામાં આવે છે.
• પ્રોગ્રામ ની ડિઝાઈન ટોપ ડાઉન હોય છે.	• પ્રોગ્રામ ની ડિઝાઈન બોટમ અપ હોય છે.
• મોટા programsને નાના programsમાં divide કરવામાં આવે છે જે ને functions કહે છે.	• મોટા programsને classes અને objectsમાં divide કરવામાં આવે છે.
• POPમાં કોઈ access specifier નથી.	• OOP માં Public, Private, Protected વગેરે નામના access specifier હોય છે.

• મોટા ભાગના function શેરિંગ માટે Global ડેટાનો ઉપયોગ કરે છે જે system માં function થી function access થાય છે.	• ડેટા function થી function access થતો નથી, તે public અથવા private રાખવામાં આવે છે જેથી ડેટાના ઍક્સેસને કન્ટ્રોલ કરી શકાય.
• ડેટા અને function ને add કરવા મુશ્કેલ છે.	• ડેટા અને function ને add કરવા સરળ છે.
• Inheritance, polymorphism, data encapsulation, abstraction, access specifier જેવા Concepts નથી.	• Inheritance, polymorphism, data encapsulation, abstraction, access specifier જેવા Concepts છે.
• **Examples**: C, Fortran, Pascal, etc…	• **Examples**: C++, Java, C#, etc…

Basic Concepts of OOP

- OOP ને વધુ powerful, secure, reliable અને સરળ બનાવવા માટે વિવિધ concepts આપે છે.

Object

- Object એ classનો instance છે.
- Real world માંથી object તરીકે person, computer વગેરે હોઈ શકે છે.
- દરેક objectમાં ઓછામાં ઓછી એક unique ઓળખ હોય છે.
- Object એ પ્રોગ્રામનો component છે જે program ના જુદા જુદા ભાગ સાથે interact કરે છે.
- object એ class type નો variable છે.

Class

- Class એ એક template છે જે objectsના attributes અને behaviorને દર્શાવે છે.
- Class એ blueprint અથવા prototype છે જેમાંથી objectsને બનાવવામાં આવે છે.
- Class એ objectsનો સમૂહ છે.
- Class એ abstract data type (ADT) નું implementation છે. એ attributes અને methods ને define કરે છે જે ADTના data structure અને operationsને implement કરે છે.

Data Abstraction

- background details નહિ ફક્ત essential features ને જ રજૂ કરે છે
- ડેટા ને secure કરવા માટે classમાં Implement કરવામાં આવે છે.
- Abstraction કરવા માટે abstract class અને interface ઉપયોગ કરીએ છીએ.

Encapsulation

- એક જ યુનિટમાં ડેટા અને functionsને Wrap up (બાઈન્ડીંગ) ને ઈનકેપ્સ્યુલેશન તરીકે ઓળખવામાં આવે છે.
- Data બહારના world માટે ઍક્સેસિબલ નથી, માત્ર તે જ functions ઍક્સેસ કરી શકે છે જે class ની અંદર declare કરેલા હોય છે.

Inheritance

- Inheritance એ પ્રક્રિયા છે, જેના દ્વારા class અન્ય classની properties અને methods ને મેળવી શકે છે.
- જુના class માંથી નવા class ને બનાવવાની process ને inheritance તરીકે ઓળખવામાં આવે છે.
- નવા classને derived class કહેવાય છે અને જુના classને base class કહેવાય છે.
- Derived class માં base classના બધા features હોઈ છે અને પ્રોગ્રામર derived classમાં નવા features ઉમેરી શકે છે.

Polymorphism

- Polymorphism એટલે ability to take more than one form (એક કરતાં વધુ ફોર્મ લેવાની ક્ષમતા).
- તે એક કરતાં વધુ related purpose માટે એક નામનો ઉપયોગ કરી શકે છે.
- તેનો અર્થ એ થાય કે operator અને function એ different situations અલગ રીતે કાર્ય કરે છે.
- પોલીમોર્ફિઝમ કરવા માટે method overloading અને method overriding નો ઉપયોગ કરીએ છીએ.

Message Passing

- પ્રોગ્રામમાં ઑબ્જેક્ટનો સેટ હોય છે જે એકબીજા સાથે કોમ્યુનિકેશન કરે છે.
- **કોમ્યુનિકેશન કરવા માટેના basic steps**
  1. Objects અને તેમના behavior defines કરવા માટે classes બનાવવા.
  2. class ની definition માંથી objects બનાવવા
  3. objects વચ્ચે communication કરવું

Dynamic and Static Binding

Static Binding	Dynamic Binding
• objectનો પ્રકાર compile સમય (કમ્પાઈલર દ્વારા) પર નક્કી કરવામાં આવે છે, તે static binding તરીકે ઓળખાય છે.	• objectનો પ્રકાર run-time પર નક્કી થાય છે, તે dynamic binding તરીકે ઓળખાય છે.
• Static binding એ binding માટે Type માહિતીનો ઉપયોગ કરે છે.	• Dynamic binding એ binding માટે Objectનો ઉપયોગ કરે છે.
• static, private, final methods અને variables ને static binding ઉપયોગ કરીને resolve કરી શકાય છે.	• રનટાઈમ દરમિયાન રનટાઈમ ઑબ્જેક્ટ નો use કરી ને Virtual methods ને resolve કરવામાં આવે છે.
• Overloaded methods ને static bindingનો ઉપયોગ કરીને resolve કરી શકાય છે.	• Overridden methods ને રનટાઈમ પર dynamic bindingનો ઉપયોગ કરીને resolve કરી શકાય છે.

Write a Program to print “My First Program In Java” in java.

```
/* Write a Program to print “My First Program In Java” in java.*/
import  java.util.*;
class MyFirstProgram
{
        public static void main(String[] args)
        {
                // This statement will print the message.
                System.out.println("My First Program In Java");
        }
}
```

Primitive Data Types

- Primitive data Type ને ચાર ભાગમાં વહેચી શકાય છે:

1) Integers :

- આ groupમાં byte, short, int અને long હોય છે.
- આ બધા માં signed, positive અને negative values હોય છે.

DataType	Size	Example
byte	8-bit	byte b, c;
short	16-bit	short b,c;
int	32-bit	int b,c;
long	64-bit	long b,c;

2) Floating-point :

- આ groupમાં float અને double હોય છે, જે નંબરોને fractional precision (ફ્રૈક્શનલ પ્રીસિશ્ન) સાથે store કરે છે.

DataType	Size	Example
float	32 bits	float a,b;
double	64 bits	double pi;

3) Characters :

- આ groupમાં char નો સમાવેશ થાય છે, જે character set ના સિમ્બોલ્સ letters અને numbers ને દર્શાવે છે.
- java માં char 16-બીટ ની જગ્યા રોકે છે. char ની range 0 થી 65,536 છે.તેમાં કોઈ negative અક્ષરો નથી.ઉદાહરણ તરીકે char name = 'x';

4) Boolean :

- આ groupમાં boolean data type હોય છે, જે true અથવા false જ values store કરે છે.
- તેમાં ફક્ત બે જ value હોઈ છે, true કે false.

For example : boolean b = true;

Example:

```
class datatypeDemo
{
        public static void main(String args[])
        {
int r=10;
float pi=3.14f,a;                    // તમે અહીં double data type નો પણ ઉપયોગ કરી શકો છો

char ch1=97,ch2='A' ;
boolean x=true;
a = pi*r*r;
System.out.println("Areat of Circle is :: "+a);
System.out.println("Ch1 and Ch2 are :: "+ch1+" "+ch2);
```

```
                System.out.println("Value of X is :: "+x);
            }
        }
```

Output:

```
Area of Circle is :: 314.0
Ch1 and Ch2 are :: a A
Value of X is :: true
```

User Defined Data Type

Class

- Class એ template છે જે object ના attributes અને behavior ને દર્શાવે છે.
- Classએ બ્લુપ્રિન્ટ અથવા પ્રોટોટાઈપ છે જેમાંથી ઘણા objects ને બનાવી શકાય છે.

Example:

```
class Box
{
        double width=1;
        double height=2;
        double depth=3;
        void volume()
        {
                System.out.print("Volume is ");
                System.out.println(width * height * depth);
        }
}
class demo
{
        public static void main(String args[])
        {
                Box  b1 = new Box();
                b1.volume();
        }
}
```

Interface

- Interface એ abstract methods નું collection છે.
- interface ની abstract methods ને inherit કરી ને class એ interface ને implement કરે છે
- Interface એ class નથી. Class એ objectના attributes અને behaviors ને દર્શાવે છે. Interface એ behaviors દર્શાવે છે જે class વડે implement કરવામાં આવે છે.
- જે class interface ને implement કરે છે તો તે class એ interface ની બધી methods ને implement કરે છે.

Example:

```
interface  Animal
{
    public void eat();
```

```
        public void travel();
    }
```

Identifiers and Literals

Identifiers (આઈડેન્ટિફાઇઅર)

- Identifiers નો use class, method અને variable નું name આપવા માટે થાય છે.
- Identifier એ અપરકેસ અને લોઅરકેસ અક્ષરો, સંખ્યાઓ અથવા અન્ડરસ્કોર અક્ષર અને dollar-સાઈન અક્ષર ની કોઈપણ descriptive sequence હોઈ શકે છે.
- Identifiers ને declared કરવા માટે rules છે. જે નીચે પ્રમાણે છે
  1) Identifiers એ letter અથવા અન્ડરસ્કોર (_) થી જ start થતો હોવો જોઈ.
  2) Identifiers ને numbers થી start કરી શકતા નથી.
  3) White space (blank, tab, newline) use થતી નથી.
  4) Identifiers તરીકે Java માં આવેલા કીવર્ડનો ઉપયોગ કરી શકતા નથી.
  5) JAVAમાં Identifiers case-sensitive હોય છે. foo અને Foo એ બે અલગ અલગ Identifiers છે.

Examples :

Valid Identifiers	Not Valid Identifiers
AvgNumber	2number
A1	int
$hello	-hello
First_Name	First-Name

Literals (Constants) (લિટરલ)

- પ્રોગ્રામમાં constant value ને literal કહેવામાં આવે છે. Literal એ numeric (integer અથવા floating-point), character, boolean અથવા string values ને દર્શાવે છે.

Integer	Floating-point	Character	Boolean	String
33, 0, -19	0.3, 3.14	(' 'R' 'r' '{'	(predefined values) true, false	“language”,”0.2” ,“r”

Variables

- Variable એ identifier, type અને optional initializer ના combination થી બને છે.
- બધા variables પાસે scope છે, જે તેમની visibility અને lifetime ને define કરે છે.

Declaring of variable

- બધા variables ને ઉપયોગ કરી તે પહેલાં declared કરવાના હોય છે.

  Syntax:

  type identifier [= value][, identifier [= value] ...];
  - આમાં type એ જાવાનાં Java’s atomic types ,class અથવા interface નું નામ છે.
  - Identifier એ variable નું નામ છે.
  - Equal sign અને value થી variable ને initialize કરી શકો છો.

- એક સરખા typeનાં એક કરતા વધુ variable ને declare કરવા માટે comma થી separate કરવા જોઈએ.

Example: int a, b, c = 10;

Dynamic Initialization

- Java માં variable ને dynamically initialize કરી શકાય છે.
- **Example:**

```
int  a=2, b=3;        // Constants as initializer
int  c = a+b;         // Dynamic initialization
```

Scope of Variables

- જાવા માં કોઈપણ બ્લોકની અંદર variable ને declare કરી શકાય છે.
- Block ચાલુ opening curly brace થી અને close closing curly brace થી થાય છે.
- Block એ scope ને define કરે છે. આમ, દરેક વખતે જ્યારે તમે કોઈ નવો block બનાવો છો, ત્યારે એક નવો scope બને છે.
- Scope એ decide કરે છે કે તમારા પ્રોગ્રામના objects કઈ જગ્યાએ visible થાશે.
- જાવા માં global અને local scope હોય છે.
- Variable જે scope માં declare કરેલ હોય તેમાં જ visible થાશે, outside ના scope માં visible થાશે નહિ.
- જ્યારે તમે કોઈ scope ની અંદર variable ને declare કરો છો, તો તમે તેને ફક્ત તે scope માં જ access કરી શકો છો અને તે unauthorized access ને protect કરે છે.
- Scope ને nested કરી શકાય છે. તેથી, inner scope એ outer scope ની અંદર હોય છે. આનો અર્થ એ છે કે outer scopeમાં declare કરાયેલા objects inner scope માં use કરી શકાય છે.
- જો કે આનું reverse સાચું નથી. Inner scope માં declare કરેલા object ને outer scope માં visible થતા નથી.

Example:

```
class  scopeDemo
{
        public static void main(String args[])
        {
                int  x;            // visible to all code within main
                x = 10;
                if(x == 10) // start new scope
                {
                        int  y = 20;      // Visible only to this block
                                // x and y both are visible here.
                        System.out.println("x and y: " + x + " " + y);
                        x = y * 2;
                }
                // y = 100;                    // Error! y not visible here
                // x is still visible here.
                System.out.println("x is " + x);
}
```

```
}
```

Default values of variables declared

- જો તમે variable ની value આપતા નથી તો java runtime એ variable ને default value આપે છે અને જો ત્યારે variable ની value access કરી તો default value મળે છે.
- નીચેનું table એ variables types અને તેમની default value ને બતાવે છે:

Data type	Default value
boolean	FALSE
char	\u0000
int,short,byte / long	0 / 0L
float /double	0.0f / 0.0d
any reference type	null

- અહીં, char primitive type ની default value \ u0000 છે, જેનો અર્થ થાય છે blank/space charactor.
- જ્યારે તમે કોઈ પણ local/block variable ને declare કરો છો ત્યારે તેને default values મળતી નથી.

Type Conversion and Casting

- તે એક type ના variable ની value ને બીજા type ના variable માં assign કરે છે જો બે types compatible છે તો JAVA આવા type ના conversion automatic કરશે. (Implicit conversion).
- ઉદાહરણ તરીકે, long variable ને int value માં assign કરવું હંમેશાં શક્ય છે. જો કે, બધા types compatible અને તેથી બધા પ્રકારનાં conversions implicitly થતા નથી.
- Double થી byteમાં conversion automatic નથી. Incompatible types નું વચ્ચેનું conversion શક્ય છે. તેના માટે ટાઈપ કાસ્ટિંગ ની જરૂર પડે છે, જે incompatible types વચ્ચે explicit conversion કરે છે.

Implicit Type Conversion (Widening Conversion) (વાઈડનિંગ કન્વર્શન)

- જો નીચેની બે શરતો પૂરી થાય તો automatic type conversion થશે તેમાં એક type નો ડેટા બીજા typeના variable માં assign થાય છે.
    1. **જ્યારે બે type compatible હોય ત્યારે.**
    2. **જ્યારે destination type એ source type કરતા મોટો હોય.**
- જ્યારે આ બે શરતો પૂરી થાય છે, ત્યારે **widening conversion** થાય છે.
- ઉદાહરણ તરીકે, int type હંમેશાં byte values ને store કરવા માટે પૂરતો હોય છે, તેથી કોઈ explicit cast સ્ટેટમેન્ટ ની જરૂર નથી.
- **નીચેના table માં numeric data ના type સાથે ની compatibility બતાવે છે:**

Status	Integer	Floating-Point	Char	Boolean
Compatible	✓	✓		
Incompatible			✓	✓

- ઉપરાંત, Boolean અને char એકબીજા સાથે compatible નથી.

Explicit Type Conversion (Narrowing Conversion) (નેરોઈંગ કન્વર્શન)

- Automatic type conversions helpful છે પરંતુ તે બધી needs ને પૂર્ણ કરશે નહીં. ઉદાહરણ તરીકે, જો આપણે int value ને byte variable માં assign કરવી હોય તો conversion automatic કરવામાં આવશે નહીં, કારણ કે byte type એ Int type કરતા નાનો હોય છે.
- આ પ્રકારના conversion ને narrowing conversion કહેવામાં આવે છે.
- આ પ્રકારના conversion માટે આપણે value ને નાની બનાવવાની જરૂર છે. તેથી, તે target data type માં ફિટ થાય.
- બે incompatible types વચ્ચે conversion કરવા માટે, તમારે cast નો ઉપયોગ કરવો જરૂરી છે.
- cast એ explicit type conversion છે.

  Synatax: **(target-type) value**
- અહીં, target-type ક્યાં type માં convert કરવું તે બતાવે છે.
- ઉદાહરણ તરીકે, નીચે int માંથી byte માં cast થાય છે

```
int  a;
byte  b;
b = (byte) a;
```

- જ્યારે floating-point value ને integer type માં convert કરવામાં આવે તેને truncation કહે છે.
- આપણે જાણીએ છીએ, integerમાં fractional component હોતો નથી.તેથી જ્યારે ફ્લોટિંગ-પોઈન્ટ value ને integer type માં convert કરવામાં આવે ત્યારે fractional component lost થઈ જાય છે.
- ઉદાહરણ તરીકે જો value 1.23 ને integer type માં assign કરવામાં આવે છે ત્યારે value ફક્ત 1 હશે. 0.23 આવશે નહિ.

Example:

```
class  conversionDemo
{
        public static void main(String args[])
        {
                byte b;
                int i = 257;
                double d = 323.142;

                System.out.println("\nConversion of int to byte.");
                b = (byte) i;
                System.out.println("i and b " + i + " " + b);

                System.out.println("\nConversion of double to int.");
                i = (int) d;
                System.out.println("d and i " + d + " " + i);

                System.out.println("\nConversion of double to byte.");
                b = (byte) d;
                System.out.println("d and b " + d + " " + b);
```

```
        }
    }
```

Output:

```
Conversion of int to byte.
i and b 257   1
Conversion of double to int.
d and i 323.142   323
Conversion of double to byte.
d and b 323.142   67
```

- જ્યારે 257 value ને byte વેરિયેબલમાં મુકવામાં આવે છે, ત્યારે 257 ને 256(the range of a byte) થી divide કરવામાં આવે છે અને reminder 1 એ answer છે
- જ્યારે d એ int માં convert થાય છે, ત્યારે fractional component lost થઈ જાય છે.
- જ્યારે d એ byteમાં convert થાય છે, ત્યારે fractional component lost થઈ જાય છે. અને value ને modulo 256 કરવામાં આવે છે , જે આ case માં 67 છે.

Wrapper Class

- Wrapper class એ data type ને wrap કરે છે અને તેને ઑબ્જેક્ટ નો દેખાવ આપે છે.
- Wrapper classes નો ઉપયોગ કોઈપણ data type ને object માં convert કરવા માટે થાય છે.
- primitive data types એ objects નથી અને તે કોઈપણ class સાથે જોડાયેલ નથી.
- કેટલીકવાર જાવામાં data types ને objects માં કન્વર્ટ કરવા જરૂરી છે.
- Wrapper classes માં object ને unwrap કરવા માટે ની methods હોય છે.

  Example:

```
int  k = 100;
Integer  it1 = new Integer(k);
```

  - Integer class નો use કરી ને int k ને it1 object માં convert કરવામાં આવેલો છે.
  - it1 objectનો ઉપયોગ જ્યાં પણ objectની જરૂર હોય ત્યાં થઈ શકે છે.
  - object it1 ને unwrap કરવા (Integer object માંથી int મેળવવા).

  Example:

```
int  m = it1.intValue();
System.out.println(m*m);       // prints 10000
```

  - intValue() એ Integer class ની method છે જે int data type આપે છે.
- java.lang package માં આઠ wrapper classes છે જે 8 data types ને દર્શાવે છે

Primitive Data Type	Wrapper Class	Unwrap Methods
byte	Byte	byteValue()
short	Short	shortValue()
int	Integer	intValue()
long	Long	longValue()
float	Float	floatValue()

double	Double	doubleValue()
char	Character	charValue()
boolean	Boolean	booleanValue()

- Wrapper classes ના main બે ઉપયોગ છે.
  1) Simple data ને objects માં કન્વર્ટ કરવા.
  2) Strings ને data type માં convert કરવા માટે (જેને parsing operations તરીકે ઓળખવામાં આવે છે), અહીં parseX() method નો ઉપયોગ થાય છે. (ઉદા. parseInt())
- Wrapper classes માં એવી methods હોય છે જેનો ઉપયોગ કરી ને String ને primitive data types માં convert કરવા આવે છે જેમાં character data type માટે method હોત્તી નથી.
- આ methods parsex() ફોર્મેટ માં હોય છે જ્યાં x એ char સિવાય કોઈપણ primitive data types ને દર્શાવે છે.
- Primitive data type માંથી string માં convert કરવામાં આપણે String class ની valueOf() method ઉપયોગ કરીએ છીએ.

  Example:

```
int x =  Integer.parseInt("34");          // x=34
double y = Double.parseDouble("34.7");      // y =34.7
String s1= String.valueOf('a');             // s1="a"
String s2=String.valueOf(true);          // s2="true"
```

Comment Syntax

- Comments એ statements છે જે ક્યારેય execute કરવામાં આવતા નથી. (તે non-executable statements છે).
- Comments ઉપયોગ કોડ માં નોંધ ઉમેરવા માટે કરવામાં આવે છે. જેથી source code ના function અને operations ને સમજવા અને સમજાવા સરળ બને.
- જાવા કમ્પાઈલર Comments ને read કરતા નથી. Compilation દરમિયાન Comments ને ignore કરવામાં આવે છે.
- જાવામાં નીચે પ્રમાણે ત્રણ પ્રકારની comments આવેલી છે

1. Single Line Comment
2. Multi Line Comment
3. Documentation Comment

Single Line Comment

- આ comment નો ઉપયોગ જ્યારે પણ આપણે એક લાઈનમાં કંઈપણ લખવાની જરૂર હોય ત્યારે થાય છે.

  Syntax : //<write comment here>

  Example: //This is Single Line Comment.

Multi Line Comment

- જ્યારે પણ આપણે source code ને લગતી વધારે નોંધ (એટલે કે એક લીટી કરતા વધારે અથવા ઘણી લાઈનમાં) લખવા માંગતા હોય ત્યારે આ પ્રકારની commentsનો ઉપયોગ થાય છે.

  Syntax :

```
/*
   <Write comment here>
*/
```

Example :

```
/*
        This Is
        Multi line comment.
*/
```

Documentation Comment

- Documentation commentનો ઉપયોગ સામાન્ય રીતે HTML ફાઈલ બનાવવા માટે થાય છે કે જે આપણા પ્રોગ્રામ નું documents બનાવે છે.
- આ comment / ** થી શરૂ થાય છે અને * / સાથે complete થાય છે.
- Documentation comments નો use કરીને પ્રોગ્રામમાં જ પ્રોગ્રામ વિશેની માહિતી એમ્બેડ કરી શકાય છે.
- પછી javadoc utility program નો ઉપયોગ કરીને માહિતીને HTML ફાઈલમાં મૂકવા આવે છે.
- Documentation commentમાં, પ્રોજેક્ટ અથવા પ્રોગ્રામના author, version, જરૂરી parameters, results ની માહિતી જો કોઈ હોય તો, વગેરે ને ઉમેરી શકીએ છીએ.
- આ સૂચનો ઉમેરવા માટે '@' operator નો use થાય છે. આપણે ફક્ત '@' operator સાથે જરૂરી સૂચન લખવાની જરૂર છે.
- **કેટલાક javadoc tags નીચે આપેલા છે:**
    - @author – પ્રોજેક્ટના author ને describe કરવા માટે.
    - @version – પ્રોજેક્ટના version ને describe કરવા માટે.
    - @param – operations કરવા માટે જરૂરી parameters સમજાવવા માટે.
    - @return – પ્રોજેક્ટના return type ને સમજાવવા માટે.

Syntax :

```
/**
*<write comment/description here>
*@author <write author name>
*@version <write version here>
*/
```

Example:

```
/**
* This is Documentation Comment.
* @author Vishal Makwana
* @version 1.0.0
*/
```

Garbage Collection

- New operatorનો ઉપયોગ કરીને objects ને dynamically memory allocate કરવામાં આવે છે.
- કેટલીક languages માં, જેમ કે C++, dynamically allocated objects ને delete operator ના ઉપયોગ દ્વારા મેન્યુઅલી release કરવાના હોય છે.

- જાવા માં અલગ approach હોય છે તેમાં object નું deallocation automatic થાય છે. આ technique ને garbage collection કહેવાય છે.
- જ્યારે કોઈ objectનો reference હોતો નથી, તે objects ને હવે જરૂરી ગણવામાં આવતા નથી અને objects દ્વારા use કરેલી મેમરીને recovered કરી શકાય છે.
- પ્રોગ્રામના execution દરમિયાન Garbage collection ક્યારેક જ થાય છે.
- વિવિધ Java runtime implementations માં વિવિધ approaches use કરી ને garbage collection કરવામાં આવે છે.

Array

- Array એ same type ના variables નું એક જૂથ છે જેને common name થી ઓળખવામાં આવે છે.
- Array માં element ને access કરવા માટે index નો use થાય છે.
- Array index zero થી start થાય છે.
- કોઈપણ typeનો array બનાવી શકાય છે અને તેમાં એક અથવા વધુ dimensions હોઈ શકે છે.

One Dimensional Array

- **dimensional array બનાવવા માટે ના steps:**
  - Array ના varible ને કોયપણ data type સાથે declare કરવામાં આવે છે.
  - New operator નો ઉપયોગ કરીને array એ hold રાખેલી મેમરીને allocate કરવી આવશ્યક છે અને તેને array variable ને assign કરવી પડશે.
- જાવામાં, બધા arrays dynamically રીતે allocate કરવામાં આવે છે.

  Syntax:

```
data_type  array_var[];
array_var  =  new  data_type[size];
                    OR
data_type  array_var []  =  new  data_type[size];
```

  Example:

```
int  a[]  = new  int[10];
```

- અહીં, datatype એ allocate કરવામાં આવતા ડેટાના type ને દર્શાવે છે, size એ array માં elements ની સંખ્યાને દર્શાવે છે, અને array_var એ array variable છે જે array સાથે જોડાયેલ છે.
- new એ ઓપરેટર છે જે મેમરીને allocate કરે છે.
- new દ્વારા allocate કરેલા arrayમાં રહેલા element માં આપમેળે zero value હોય છે.

  Example:

```
class  demo_array
{
        public static void main (String args[])
        {
                int a[] = new int[3];              // int a[] = {1,2,3};
                a[0] = 1;
                a[1] = 2;
                a[2] = 3;
                System.out.println("Your array elements are :: "+ a[0]+" "+a[1]+" "+a[2]);
```

```
        }
    }
```

Output:

```
Your array elements are :: 1 2 3
```

Multidimensional Array

- Multidimensional arrays ખરેખર arrays નો arrays છે.
- multidimensional array variable ને declare કરવા માટે વધારાની index માટે વધારે square brackets નો use કરવો

Syntax:

```
data_type  array_var [][] = new  data_type[size][size];
```

Example:

```
int  a[][] = new int[3][5];
```

- આ 3*5 array allocate કરે છે અને તેને a માં assign કરે છે.આ matrix ને array નો array તરીકે implement કરવામાં આવશે.

Example:

```
class demo_Tarray
{
    public static void main (String args[])
    {
        int a[][] = new int[2][2];          // int a[][] = {{1,2},{3,4}};
        a[0][0] = 1;
        a[0][1] = 2;
        a[1][0] = 3;
        a[1][1] = 4;
        System.out.println("Your array elements are :: "+ a[0][0]+" "+a[0][1]+"
                "+a[1][0]+" "+a[1][1]);
    }
}
```

Output:

```
Your array elements are :: 1 2 3 4
```

String

- java પ્રોગ્રામિંગમાં Strings નો વધારે ઉપયોગ થાય છે, તે ફક્ત અક્ષરોની sequence નથી તે object દ્વારા defines કરે છે.
- String Class એ java. lang પેકેજમાં define થયેલ છે.
- String type નો ઉપયોગ String નો variable ને declare કરવા માટે થાય છે. પણ આપણે strings નો array પણ declare કરી શકીએ છીએ.
- String type ના વેરિયેબલને બીજા String type ના વેરિયેબલમાં assign કરી શકાય છે.

Example:

```
String s1 = “Welcome To Java String”;
System.out.println(s1);
```

  - અહીં, s1 એ String type નો object છે.

- String objects માં ઘણા special features અને attributes છે જે string powerful બનાવે છે.

String class has following features:

- તે Final class છે
- Final ના કારણે, String class inherits થઈ શકતો નથી.
- તે immutable છે.

Method	Description
charAt(int index)	આપેલી index પર રહેલા character ને આપે છે.
compareTo(String anotherString)	લેક્સિકોગ્રાફિકલી બે strings ને Compare કરે છે.
compareToIgnoreCase(String str)	કેસના તફાવતોને ignore કરીને બે strings ને compare કરે છે.
concat(String str)	આ current string માં સાથે આપેલી string ને જોડે છે.
contentEquals(StringBuffer sb)	આ current string ને આપેલા StringBuffer સાથે compare કરવા માટે થાય છે.
equals(Object anObject)	આ stringને ચોક્કસ object સાથે સરખાવે છે.
isEmpty()	length zero હોય તો જ true value return આપે છે.
length()	આ stringની length આપે છે.
split(String regex)	આ આપેલા regular expression નો use કરી ને string ને split કરે છે.
toLowerCase()	આ String ના તમામ characters ને lower case માં ફેરવે છે.
toString()	આમાં object (જે પહેલાથી જ string છે) પોતે જ return થાય છે.
toUpperCase()	આ String ના તમામ characters ને Upper case માં ફેરવે છે.
trim()	Current string માં આગળ અને પાછળની empty જગ્યા remove કરીને new string return કરે છે.

Example:

```
import java.io.*;
class  stringDemo
{
        public static void main(String args[])
        {
                String str = "Darshan Institute of Engineering & Technology";
                System.out.println(str.length());
                if(str.equals("DIET"))
{
                System.out.println("Same");
}
else
{
                System.out.println("Not Same");
}
if ( str.compareTo("Darshan") > 0)
{
```

```
            System.out.println("Darshan is greater than Darshan Institute of
            Engineering & Technology ");
    }
    else
    {
            System.out.println("Darshan Institute of Engineering &
            Technology is greater than Darshan");
    }
    System.out.println( str.substring(1,3));
            }
    }
```

StringBuffer Class

- JAVA માં StringBuffer class એ થ્રેડ-સેફ અને બદલી શકાય તેવા characters નો સમૂહ છે.
- દરેક string buffer ની capacity હોય છે.
- તેમાં character ની particular sequence છે, પરંતુ તેની length અને content ને method દ્વારા બદલી શકાય છે.

StringBuffer class Constructor

Constructor	Description
StringBuffer()	આ કોઈપણ અક્ષરો વગર અને 16 અક્ષરોની capacity સાથે string buffer બનાવે છે.
StringBuffer(CharSequence seq)	આપેલા CharSequence નો use કરી ને string Buffer બનાવે છે
StringBuffer(int capacity)	આમાં કોઈપણ અક્ષરો વગર અને initial capacity સાથે string buffer બનાવે છે.
StringBuffer(String str)	આપેલી string નો use કરીને string buffer બનાવે છે.

StringBuffer Methods

Method	Description
capacity()	String buffer ની current size આપે છે.
charAt(int index)	આપેલી index પર રહેલા character ને આપે છે.
toString()	આ method stringbuffer નો content ને string માં convert કરે છે..
insert(int offset, char c)	આ method આપેલા char ને string representation માં convert કરીને આપેલા offset પર insert કરે છે.
append(String str)	આપેલી string ને current character sequence માં append કરે છે.
reverse()	આ method string buffer માં આવેલી character sequence ને reverse કરશે.

Example:

```
import java.io.*;
class  stringBufferDemo
{
    public static void main(String args[])
    {
            StringBuffer strBuf1 = new StringBuffer("DIET");
            StringBuffer strBuf2 = new StringBuffer(100);
```

```
        System.out.println("strBuf1 : " + strBuf1);
        System.out.println("strBuf1 capacity : " + strBuf1.capacity());
        System.out.println("strBuf2 capacity : " + strBuf2.capacity());
        System.out.println("strBuf1 reverse : " + strBuf1.reverse());
        System.out.println("strBuf1 charAt 2 : " +strBuf1.charAt(2));
        System.out.println("strBuf1 toString() is : " + strBuf1.toString());
        strBuf1.append("Darshan Institute of Engineering & Tech.");
        System.out.println("strBuf3 when appended with a String : "+ strBuf1);
    }
```

Output:

```
strBuf1 : DIET
strBuf1 capacity : 20
strBuf2 capacity : 100
strBuf1 reverse : TEID
strBuf1 charAt 2 : I
strBuf1 toString() is : TEID
strBuf3 when appended with a String : TEIDDarshan Institute of Engineering & Tech.
```

Difference Between String and StringBuffer

String	StringBuffer
તે immutable છે string માં સુધારો કરી શકતા નથી.	તે mutable છે તેમાં સુધારો કરી શકો છો.
StringBuffer કરતાં String class એ slow છે.	StringBuffer class એ String class કરતા fast છે.
જ્યારે multiple threads નો use કરી ત્યારે string class એ safe નથી	જ્યારે multiple threads નો use કરી ત્યારે string class એ safe છે.
string class માં insert() method નથી.	StringBuffer class માં insert() method હોય છે
string class માં split() method હોય છે	StringBuffer class માં split() method નથી.
String class એ Object classની equals() method ને ઓવરરાઈડ કરે છે.	StringBuffer class એ Object classની equals() method ને ઓવરરાઈડ કરતો નથી.

Operator

Arithmetic Operator

- Arithmetic operators નો ઉપયોગ mathematical expressions માં થાય છે.
- નીચેનું table arithmetic operatorsનું લીસ્ટ દર્શાવે છે: ધારો કે variable A માં 10 value છે અને variable B માં 20 value છે

Operator	Description	Example
+	Addition - Operatorની બંને બાજુ આવેલી value ને ઉમેરે છે	A + B એ 30 આપશે
-	Subtraction - left hand ના operandમાંથી right handના operandને બાદ કરે છે	A - B એ -10 આપશે
*	Multiplication - Operatorની બંને બાજુ આવેલી value ને multiply કરે છે	A * B એ 200 આપશે

/	Division - left handના operandને right handના operand થી Divide કરે છે	B / A એ 2 આપશે
%	Modulus left handના operandને right handના operand થી Divide કરે છે અને reminder આપે છે	B % A એ 0 આપશે
++	Increment - operandની value માં 1 નું increment કરે છે	B++ એ 21 આપશે
--	Decrement - operandની value માં 1 નું decrement કરે છે	B-- એ 19 આપશે

Relational Operator

- નીચેનું table relational operatorsનું લીસ્ટ દર્શાવે છે: ધારો કે variable A માં 10 value છે અને variable B માં 20 value છે

Operator	Description	Example
==	તે બે operands ની values સરખી છે કે નહીં તે check કરે છે, જો values સરખી હોય તો condition true થાય છે.	(A == B) સાચું નથી.
!=	તે બે operands ની values સરખી છે કે નહીં તે check કરે છે, જો values સરખી નથી તો condition true થાય છે.	(A != B) સાચું છે..
>	તે left operand ની value right operand ની value કરતા વધારે છે કે કેમ તે check કરે છે, જો વધારે હોય તો condition true થાય છે.	(A > B) સાચું નથી.
<	તે left operand ની value right operand ની value કરતા ઓછી છે કે કેમ તે check કરે છે, જો ઓછી હોય તો condition true થાય છે.	(A < B) સાચું છે.
>=	તે left operand ની value right operand ની value કરતા વધારે અથવા સરખી છે કે કેમ તે check કરે છે, જો વધારે અથવા સરખી હોય તો condition true થાય છે.	(A >= B) સાચું નથી.
<=	તે left operand ની value right operand ની value કરતા ઓછી અથવા સરખી છે કે કેમ તે check કરે છે, જો ઓછી અથવા સરખી હોય તો condition true થાય છે.	(A <= B) સાચું છે.

Bitwise Operator

- બિટવાઈઝ operator bits પર કામ કરે છે અને બીટ-બાય-બિટ operation કરે છે.
- ધારો કે a = 60 અને b = 13 હોય , હવે બાઈનરી ફોર્મેટમાં તે નીચે મુજબ હશે:

```
a = 0011 1100
b = 0000 1101
-----------------
a&b= 0000 1100
a|b= 0011 1101
a^b = 0011 0001
~a  = 1100 0011
```

Operator	Description	Example

&	Binary AND Operator result માં bit ને copy કરે છે જો તે બંને operandsમાં હોય તો.	(A & B) એ 12 આપશે જે 0000 1100 છે
|	Binary OR Operator result માં bit ને copy કરે છે જો તે બંને માં થી કોઈ એક operandsમાં હોય તો.	(A | B) એ 61 આપશે જે 0011 1101 છે
^	Binary XOR Operator result માં bit ને copy કરે છે જો તે બંને માં થી કોઈ એક operandsમાં હોય તો, બને માં હોય ત્યારે copy કરતુ નથી.	(A ^ B) એ 49 આપશે જે 0011 0001 છે
~	Binary ones Complement Operator એ unary operator છે અને તે bits flip કરે છે.	(~A) એ આપશે -61 જે 1100 0011 છે
<<	Binary Left Shift Operator એ left operands ની value ને right operand ની value ની સંખ્યા જેટલી ડાબી બાજુ ખસેડવામાં આવે છે.	A << 2 એ 240 આપશે જે 1111 0000 છે
>>	Binary right Shift Operator એ left operands ની value ને right operand ની value ની સંખ્યા જેટલી જમણી બાજુ ખસેડવામાં આવે છે.	A >> 2 એ 15 આપશે જે 0000 1111 છે

Logical Operator

- નીચેનું table Logical Operators નું લીસ્ટ દર્શાવે છે: ધારો કે variable A માં true value છે અને variable B માં false value છે

Operator	Description	Example
&&	Logical **AND** Operator: જો left operands અને right operand ની value true હોય તો જ condition true થાય છે.	(A && B) એ false છે.
||	Logical **OR** Operator: જો left operands અથવા right operand ની value true હોય તો condition true થાય છે.	(A || B) એ true છે.
!	Logical **NOT** Operator: operand ના logical state ને reverses કરવા માટે ઉપયોગ થાય છે. જો કોઈ condition true હોય તો Logical NOT operator એ condition ને true કરશે.	!(A && B) એ true છે.

Assignment Operator

- નીચેનું table એ Assignment Operatorનું લીસ્ટ દર્શાવે છે:

Operator	Description	Example
=	Simple assignment operator એ right side ના operand ની value ને left side ના operand માં assign કરે છે.	C = A + B એ A + B ની value ને C માં assign કરેછે
+=	Add and assignment operator એ right side ના operand ની value અને left side ના operand ની value ને add કરે છે અને result ને left operand માં assign કરે છે.	C += A અને C = C + A સરખા છે
-=	Subtract and assignment operator એ right side ના operand ની value	C -= A અને C = C – A

	ને left side ના operand ની value માંથી Subtract કરે છે અને result ને left operand માં assign કરે છે.	સરખા છે
*=	Multiply and assignment operator એ right side ના operand ની value ને left side ના operand ની value સાથે Multiply કરે છે અને result ને left operand માં assign કરે છે.	C *= A અને C = C * A સરખા છે
/=	Divide and assignment operator એ left side ના operand ની value ને right side ના operand ની value થી Divide કરે છે અને result ને left operand માં assign કરે છે.	C /= A અને C = C / A સરખા છે

Conditional (Ternary) Operator

- Conditional operator ને ternary operator તરીકે પણ ઓળખવામાં આવે છે.
- આ operator માં ત્રણ operands છે અને તેનો ઉપયોગ boolean expressions ને evaluate કરવા માટે થાય છે.
- Operator નો use એ નક્કી કરવા માટે થાય છે કે variableને કઈ value assign કરવી જોઈએ.

Syntax:

```
variable x = (expression) ? value if true : value if false
```

Example:

```
class conditionalDemo
{
    public static void main(String args[])
    {
        int a , b;
        a = 10;
        b = (a == 1) ? 20: 30;
        System.out.println( "Value of b is : " +  b );
        b = (a == 10) ? 20: 30;
        System.out.println( "Value of b is : " + b );
    }
}
```

Output:

```
Value of b is : 30
Value of b is : 20
```

Increment and Decrement Operator

- java માં Increment operator અને decrement operator છે : ++ (value ને 1 થી increment કરવું) અને -- (value ને 1 થી decrement કરવું)
- operator ને operand પહેલા use કરવામાં આવે તો તેને prefix increment/decrement કહેવાય છે અને operator ને operand પછી use કરવામાં આવે તો તેને postfix increment/decrement કહેવાય છે

Example: ધારો કે x = 1

```
y = ++x;
System.out.println(y);
z = x++;
System.out.println(z);
```

Output:

2

1

Mathematical Function

- java.lang.math પેકેજમાં બિલ્ટ-ઈન mathematical function આવેલા છે.
- Java માં ઘણાંબધા mathematical function સપોર્ટેડ છે. તેમાંના functionનીચે મુજબ છે:

Function	Description
abs()	Input કરેલા integer argument ની absolute value આપે છે.
round()	તે float argumentની સૌથી નજીકની integer value આપે છે.
ceil()	તે number કરતા વધારે અથવા તેના જેટલી small integer value આપે છે.
floor()	તે number કરતા વધારે અથવા તેના જેટલી large integer value આપે છે.
min()	તે બે argumentsમાંથી નાની value આપે છે.
max()	તે બે argumentsમાંથી મોટી value આપે છે.
sqrt()	તે argumentનું square root આપે છે.
cos()	તે argument (angle) ની trigonometric cosine value આપે છે.
pow()	તે first argument ને બીજી argument ના power સાથે વધારે છે

- **All argument to the function should be double.**

Example:

```
import java.lang.*;
class mathLibraryExample
{
        public static void main(String[] args)
        {
                int j = -9;
                double x = 72.3;

                System.out.println("|" + j + "| is " + Math.abs(j));
                System.out.println("|" + x + "| is " + Math.abs(x));
               System.out.println(x + " is approximately " + Math.round(x));
                System.out.println("The ceiling of " + x + " is " + Math.ceil(x));
               System.out.println("The floor of " + x + " is " + Math.floor(x));

                System.out.println("min(" + j + "," + x + ") is " + Math.min(j,x));
                System.out.println("max(" + j + "," + x + ") is " + Math.max(j,x));
                System.out.println("pow(2.0, 2.0) is "  + Math.pow(2.0,2.0));
                System.out.println("The square root of 9 is " + Math.sqrt(9));
        }
}
```

Output:

|-9| is 9
|72.3| is 72.3
72.3 is approximately 72
The ceiling of 72.3 is 73.0
The floor of 72.3 is 72.0
min(-9,72.3) is -9.0
max(-9,72.3) is 72.3
pow(2.0, 2.0) is 4.0
The square root of 9 is 3.0

Selection Statement

- જાવા બે selection statement ને support કરે છે: if અને switch.
- આ statement નો ઉપયોગ કરી ને program ના execution ને run time એ કન્ટ્રોલ કરી શકાય છે.

If statement

- if statement માં condition હોય છે.

Syntax:

```
if (condition)
{
  // જો Boolean expression સાચું હોય તો જ Statements execute થાશે.
}
```

- જો condition સાચી હશેતો if statement ની અંદર ના સ્ટેટમેન્ટ એક્ઝીક્યુટ થશે

Example:

```
class ifDemo
{
        public static void main(String[] args)
        {
            int marks = 76;
            String grade = null;
            if (marks >= 40)
            {
                    grade = "Pass";
            }
            if (marks < 40)
            {
                    grade = "Fail";
            }
            System.out.println("Grade = " + grade);
        }
}
```

Output:

Grade = Pass

If ... else Statement

- if ...else statement જાવાનું conditional branch statement છે.
- અહીં if..else સ્ટેટમેંટનું general form આપવામાં આવ્યું છે:

Syntax:

```
if (condition)
        statement1;
else
        statement2;
```

- Condition એ કોઈપણ boolean expression છે. else part એ optional છે.
- અહીં curly braces ની અંદર statement એ single statement અથવા compound statement હોય છે.
- If..else માં જો condition સાચી હોય તો statement1 એક્ઝેક્યુટ થાય છે અને જો condition ખોટી હોય તો statement2 (જો હોય તો) એક્ઝેક્યુટ થાય છે.

Example:

```
class ifelseDemo
{
        public static void main(String[] args)
        {
            int marks = 76;
            String grade;
            if (marks >= 40)
            {
                    grade = "Pass";
            }
            else
            {
                    grade = "Fail";
            }
            System.out.println("Grade = " + grade);
        }
}
```

Output:

```
Grade = Pass
```

Switch Statement

- Switch statement જાવાનું multiway branch statement છે. તેનો use expression ની value પ્રમાણે code ના જુદા જુદા part ને execute કરવા માટે થાય છે.
- અહીં switch statement નું general form આપવામાં આવ્યું છે:

Syntax:

```
switch (expression)
{
        case value1:
                // statement sequence
                break;
        case value2:
                // statement sequence
```

```
                    break;
                    ...
                case valueN:
                    // statement sequence
                    break;
                default:
                    // default statement sequence
        }
```

- Expression ની ટાઈપ byte, short, int અને char માંથી કોઈ એક હોવી જોઈ.
- દરેક case ની value unique હોવી જોઈએ છે (એટલે કે, તે constant હોવી જોઈએ, variable હોવી જોઈએ નહીં). Case ની duplicate value allow કરતી નથી.
- Switch statement માં expression ની value ને બધા case ની literal values સાથે compare કરવામાં આવે છે જો case match થાય ત્યારે કેસ સ્ટેટમેન્ટ પછીના કોડ ને એક્ઝેક્યુટ કરવામાં આવે છે.
- જો કોઈ case ની literal value expression સાથે match નથી થતી ત્યારે default સ્ટેટમેંટ એક્ઝેક્યુટ થાય છે. જો કે default સ્ટેટમેન્ટ એ optional છે.
- Statement ની sequence ને terminate કરવા માટે break statement ને switch case ની અંદર use કરવામાં આવે છે.

Example:

```
class switchDemo
{
        public static void main(String[] args)
        {
        int day = 8;
        switch (day)
        {
           case 1:  System.out.println("Monday"); break;
           case 2:  System.out.println("Tuesday"); break;
           case 3:  System.out.println("Wednesday"); break;
           case 4:  System.out.println("Thursday"); break;
           case 5:  System.out.println("Friday"); break;
           case 6:  System.out.println("Saturday"); break;
           case 7:  System.out.println("Sunday"); break;
           default: System.out.println("Invalid Day");break;
         }
    }
}
```

Output:

```
Thursday
```

Iteration Statement

- જાવા માં for, while અને do-while ઈટરેશન સ્ટેટમેન્ટ્સ હોય છે.
- આ statement ને સામાન્ય રીતે loop કહે છે.
- loop એ જ્યાં સુધી condition સાચી પડે ત્યાં સુધી સરખા statement ને repeatedly executes કરે છે.

while

- તે statements block ને repeat executes કરશે જ્યાં સુધી controlling expression ની value true મળે.

Syntax:

```
while(condition)
{
        // body of loop
}
```

- Condition કોઈપણ boolean expression હોઈ શકે છે. લૂપની body જ્યાં સુધી conditional expression true હોય ત્યાં સુધી execute થશે.
- જ્યારે condition ખોટી પડે છે ત્યારે કંટ્રોલ તરત જ loop ની બહાર ની નીચેની line માં જાય છે.

Example:

```
class whileDemo
{
        public static void main(String args[])
        {
            int x = 1;
            while( x < 5 )
            {
                System.out.print(x+" " );
                x++;
            }
        }
}
```

Output:

```
1 2 3 4
```

do-while

- ઘણીવાર conditional ખોટી હોય તો પણ loop ની body minimum એક વખત execute થવી જરૂરી હોય છે. ત્યારે do-while loop use થાય છે.
- do-while loop હંમેશાં body ને minimum એક વાર execute કરે છે કારણ કે તેમાં conditional expression loop ની bottom માં છે.

Syntax:

```
do
{
        // body of loop
} while (condition);
```

- do-while loop માં દરેક iteration માં લૂપની body પહેલા executes કરશે અને પછી conditional check કરે છે. જો આ conditional સાચી હોય તો loop repeat કરશે નહિંતર લૂપ terminate થાય છે. અહી Condition એ boolean expression માં હોવી જોય.

Example:

```
class dowhileDemo
{
        public static void main(String args[])
        {
            int x = 1;
            do
```

```
                    {
                        System.out.print(x+" ");
                        x++;
                    }while( x < 5);
                }
        }
```

Output:

```
1  2  3  4
```

for

Syntax:

```
for(initialization; condition; iteration)
{
        // body
}
```

- જ્યારે લૂપ પહેલીવાર શરૂ થાય છે ત્યારે લૂપનો initialization ભાગ execute થાય છે. initialization ભાગ ફક્ત એક જ વાર execute થાય છે.
- બીજા ભાગમાં condition આવે છે. આમાં boolean expression હોવી જોઈએ. તે loop control variable ને target value સાથે check કરે છે જો expression સાચું હોય તો લૂપની body execute થાય છે અને જો તે expression ખોટું હોયતો લૂપ terminate થાય છે.
- ત્રીજો ભાગ loop નો iteration portion છે. આમાં loop control variable ને increment અથવા decrement કરવામાં આવે છે.
- પ્રથમ conditional expression evaluate થાય છે પછી લૂપ ની body execute થાય છે. અને પછી દરેક iteration માટે iteration expression execute થાય છે. Controlling expression ખોટી ન થાય ત્યાં સુધી આ process repeats થાય છે.

Example:

```
class forDemo
{
        public static void main(String args[])
        {
                for(int x = 1; x < 5; x++ )
                {
                    System.out.print(x+" ");
                }
        }
}
```

Output:

```
1   2  3  4
```

Jump Statement

- આ statements પ્રોગ્રામ નો કન્ટ્રોલ બીજા ભાગમાં transfer કરે છે.

break

- break કીવર્ડનો ઉપયોગ લૂપને stop કરવા માટે થાય છે. બ્રેક કીવર્ડ કોઈપણ લૂપ અથવા સ્વીચ સ્ટેટમેન્ટની અંદર use કરવામાં આવે છે.
- બ્રેક કીવર્ડ એ innermost લૂપને stop કરશે અને બ્લોક પછી ના કોડ ને એક્ઝેક્યુટ કરવાનું શરૂ કરશે.

Example:

```
class breakDemo
{
        public static void main(String args[])
        {
            int [] numbers = {10, 20, 30, 40, 50};
            for(int x=0;x<5;x++ )
            {
                if( numbers[x] == 30 )
                {
                    break;
                }
                System.out.print( numbers[x] +"  ");
            }
        }
    }
```

Output:

```
10  20
```

Continue

- Continue statement નો use loop નો કોયપણ iteration માં આવેલા statements ને સ્કીપ કરવા માટે થાય છે અને new iteration ચાલુ કરે છે.

Example:

```
class  continueDemo
{
        public static void main(String args[])
        {
            int [] numbers = {10, 20, 30, 40, 50};
            for(int x=0;x<5;x++ )
            {
                if( numbers[x] == 30 )
                {
                    continue;
                }
                System.out.print( numbers[x] +"  ");
            }
        }
    }
```

Output:

```
10  20  40  50
```

return

- return સ્ટેટમેન્ટનો ઉપયોગ method માંથી explicit રીતે પાછા આવવા માટે થાય છે. તે પ્રોગ્રામ ના કન્ટ્રોલ ને પાછો caller method પાસે transfer કરે છે.
- return સ્ટેટમેન્ટ તરત જ method ને સ્ટોપ કરે છે જેમાં તે execute કરવામાં આવે છે.

Example:

```
class Test
{
        public static void main(String args[])
        {
            int a=10, b=20, c;
            c=add(a,b);
            System.out.println("Addition="+c);
        }
        public static int add(int x, int y)
        {
                return x+y;
        }
}
```

Output:

```
Addition=30
```

Class

- Class એ template છે જે object ના attributes અને behavior ને દર્શાવે છે.
- Classએ બ્લુપ્રિન્ટ અથવા પ્રોટોટાઈપ છે જેમાંથી ઘણા objects ને બનાવી શકાય છે.
- Class એ abstract data type (ADT) નું implementation છે. જે attributes અને methods ને define કરે છે તેમજ ADTના data structure અને operationsને implement કરે છે.

  Syntax:

```
class classname
{
    Datatype variable1;
    Datatype variable2;
    // ...
    Datatype variableN;

    return_type   methodname1(parameter-list)
    {
        // body of method
    }

    return_type   methodname2(parameter-list)
    {
        // body of method
    }

        return_type   methodnameN(parameter-list)
        {
            // body of method
        }
}
```

  - class ને class keyword નો ઉપયોગ કરીને declare કરવામાં આવે છે.
  - class માં declare કરેલા ડેટા અને વેરીએબલ્સને ઈન્સ્ટન્સ વેરીએબલ્સ કહેવામાં આવે છે કારણ કે classના દરેક object માટે આ variables ની પોતાની copy હોય છે તેથી એક object નો ડેટા બીજા object ના ડેટાથી અલગ અને unique હોય છે.
  - methods ની અંદર કોડ ને લખવામાં આવે છે.
  - class માં defined કરેલ methods અને variables એ class ના members છે.

Methods

Syntax:

```
return_type   method_name(parameter-list)
{
    // body of method
```

```
}
```

- અહીં return_type એ method માંથી return થયેલ ડેટાની type દર્શાવે છે. તે કોઈપણ data type નો હોઈ શકે છે. જો method કોઈ value return કરતી નથી, તો તેનો રીટર્ન type void હોય છે.
- method_name એ methodનું નામ દર્શાવે છે. આ કોઈપણ valid identifier હોયશકે જે current scope માં પહેલાથી declare કરેલ ન હોવા જોઈએ. parameter-list એ type અને identifier ની comma separate કરેલ sequence છે.
- Parameters એ variables છે. જે method call કરવામાં આવે ત્યારે argument તરીકે pass કરવામાં આવે છે. જો method માં કોઈપણ parameters ન હોય તો parameter list ને empty રાખવામાં આવે છે.

Return value

- જયારે method માં void સિવાય કોઈપણ return type હોય ત્યારે calling method ને નીચે આપેલ form દ્વારા value return કરવામાં આવે છે.

  Syntax:

  ```
  return  value;
  ```

  - અહીં, value એ return કરવાની હોય તેને દર્શાવે છે.

  Example:

```
class  Box
{
        double width = 1;
        double height = 2;
        double depth = 3;
        double volume()
        {
                return  (width * height * depth);
        }
}
```

Method Call

Syntax:

```
var_name  =  object_name.method_name(parameter-list);
```

Example:

```
vol = b1.volume();
```

- ઉપરના ઉદાહરણમાં, b1 એ એક object છે અને જ્યારે volume() ને call કરવામાં આવે છે, ત્યારે તે એસાઈનમેન્ટ સ્ટેટમેન્ટની જમણી બાજુ મૂકવામાં આવે છે. ડાબી બાજુ માં variable હોય છે અહી vol માં volume() દ્વારા return થયેલ value store થાય છે.

Declaring Object

- class નો object બનાવાવા માટે class type નો variable declare કરવામાં આવે છે અને આ variable કોઈ object ને define કરતો નથી. તે ફક્ત variable જ છે જે કોઈ object ને refer કરી શકે છે.
- new operator નો use કરીને object ની original copy લેવામાં આવે છે અને variable ને assign કરવામાં આવે છે

- new operator એ object માટે dynamically memory allocate કરે છે અને allocated કરેલી memory નું address આપે છે.

  Syntax:

  Step - 1 : class_name class_var;
  Step - 2: class_var = new class_name( );

  - અહીં, class_var એ class type નો variable create બનાવામાં આવે છે.class_name એ class નું નામ છે જે ના object બનાવામાં આવે છે.

Assigning Object Reference Variables

- Object ના રેફરન્સ વેરીએબલ્સ ને assignment operator સાથે ઉપયોગ કરવામાં આવે ત્યારે તે જુદી રીતે act કરે છે.

  Example:

  ```
  Box  b1 = new Box();
  Box  b2 = b1;
  ```

  - અહીં b1 અને b2 બંને એક જ object ને refer કરે છે. જયારે b1 થી b2 માં assignment થાય ત્યારે કોઈ પણ memory allocate થતી નથી.
  - તેથી b2 દ્વારા કોઈપણ changes કરવામાં આવે તો તે b1 ને effect કરશે કારણકે b2 અને b1 બને same object ને refer કરે છે.

  Example:

```
class Box
{
        double width = 1.0;
        double height = 2.0;
        double depth = 3.0;

        void volume()
        {
                System.out.print("Volume is ");
                System.out.println(width * height * depth);
        }
}
class BoxDemo
{
        public static void main(String args[])
        {
                Box b1 ;                    // declare reference to object
                b1 = new Box()              // allocate a Box object
                //Box b1 = new Box();       We can also combine above two statement.
                b1.volume();
        }
}
```

Output:

```
Volume is 6.0
```

Visibility Controls of JAVA

- classes, variables, methods અને constructors ના access level ને કન્ટ્રોલ કરવા માટે java માં જુદા જુદા access modifier આવેલ છે.JAVA માં four access levels છે:
  1) **Package/friendly (default) –** પેકેજ માં access થાય છે. કોઈ modifiers ની જરૂર નથી.
  2) **Private** – class માં જ access થાય છે.
  3) **Public**- class માં અને class ની બહારના part માં પણ access થાય છે.
  4) **Protected**- પેકેજ માં અને બધા sub class માં access થાય છે.

Default Access Modifier – No Keyword

- Default access modifier નો meaning એ છે કે class, field, method માટે access modifier declare કરવામાં આવતા નથી.
- કોઈપણ access control મોડિફાયર નો use કર્યા વગર declare કરેલ variable અથવા method ને same package માં રહેલા બીજા class access કરી શકે છે. ઈન્ટરફેસમાં methods માટે ડિફોલ્ટ મોડિફાયરનો ઉપયોગ કરી શકાતો નથી કારણ કે (interface)ઈન્ટરફેસ માં રહેલી methods by default public હોય છે.

Example:

```
String  str = “Hi”;
void a()
{
        System.out.println(str);
}
```

Private Access Modifier – private

- private declare કરેલ Methods, Variables અને Constructors ને જે class માં declare કરેલા હોય તે જ classમાં જ access કરી શકાય છે.
- Private access modifier એ સૌથી વધારે restrictive (રિસ્ટ્રિક્ટિવ) access level આપે છે. Class and interfaces એ private હોઈ શકે નહિ.
- જો public getter methods class માં હોય તો private declare કરાયેલ Variables ને class ની બહાર access કરી શકાય છે.
- private મોડિફાયરનો ઉપયોગ કરીને object encapsulate થાય છે અને બહારના world થી ડેટા ને hide કરે છે.

Example:

```
class  A
{
private String s1 = “Hello”;
public String getName()
{
        return  this.s1;
}
}
```

- અહીં, A class નો s1 variable private છે, તેથી બીજા class માં access થઈ શકે નહિ. જો આ variable ને class ની બહાર access કરવો હોય તો getName() નામ ની public method declare કરેલ છે જે s1 ની value return કરે છે.

Public Access Modifier - public

- Public keyword એ access specifier છે જે પ્રોગ્રામરને class ના members ની visibility ને કન્ટ્રોલ કરવા માં મદદ કરે છે.
- જ્યારે class ના member ને public declare કરેલ હોય ત્યારે તે class ની બહાર access થઈ શકે છે.
- Class, method, constructor અને interface વગેરે ને public declares કરેલ હોય તો તેને બીજા class દ્વારા access કરી શકાય છે.
- જેથી Public class માં declare કરેલ methods અથવા blocks ને Java world માં આવેલ કોઈપણ class દ્વારા access કરી શકાય છે.
- જો આપણે કોઈબીજા package માં રહેલા class ને access કરવો હોય તો તેને import કરવો પડે છે કારણકે Class માં આવેલ public methods અને variables ને inheritance નો use કરી ને sub class દ્વારા inherit કરવામાં આવે છે.

  Example:

```
public static void main(String[] args)
{
    // ...
}
```

- એપ્લિકેશનની main () method public હોવી જરૂરી છે. નહિંતર, જાવા ઈન્ટરપ્રીટર main method ને call કરી શકે નહિ.

Protected Access Modifier – Protected

- Variables, methods અને constructors ને super class માં protected declare કરવામાં આવ્યા છે હોય તેને ફક્ત બીજા પેકેજ અથવા same પેકેજ માં રહેલ sub class માં access કરી શકાય છે.
- Protected access મોડિફાયર class અને interfaces સાથે use કરી શકાય નહિ. Methods ને protected declare કરી શકાય છે.જયારે ઈન્ટરફેસમાં રહેલ Methods ને protected declare કરી શકાતી નથી.
- Protected access મોડિફાયર એ subClass ને super class ની helper method અથવા variable use કરવા દે છે જયારે non-related class ને use કરતા અટકાવે છે.

  Example:

```
package p1;
class A
{
    float f1;
    protected int i1;
}

// note that class B belongs to the same package as class A
package p1;
class B
```

```
{
        public void getData()
        {
                // create an instance of class A
                A  a1 = new A();
                a1.f1 = 19;
                a1.i1 = 12;
        }
}
```

- ઉપરના ઉદાહરણમાં, class A અને B same પેકેજ p1 માં છે. Class A નો i1 variable છે જે protected તરીકે declare કરેલ છે. તેથી, તે સંપૂર્ણ પેકેજ અને તેના subclasses દ્વારા access કરી શકાય છે. આમ, class B ની getData() methodમાં આપણે variable f1 અને આઈ i1 નો ઉપયોગ કરી શકીએ છીએ.

this Keyword

- ક્યારેક કોઈ method જે object દ્વારા invoke થયેલ હોય તેને refer કરવાની જરૂર પડે છે.
- તેના માટે this keyword નો use થાય છે. this keyword ને method અને constructor માં use કરી શકાય છે જે current object ને refer કરે છે.
- તેનો અર્થ એ કે, this હંમેશા તે object refer કરે છે કે જેનો use કરીને તે method call કરવામાં આવેલ હોય.
- Variable hide થતા હોય તે case માં this keyword ખૂબ ઉપયોગી થઈ શકે છે.
- class ના current object ને refer કરવામાટે કોઈપણ જગ્યાએ use કરી શકાય છે.
- એક જ નામ ના બે Instance/Local variables બનાવી શકય નહિ. પરંતુ એક જ નામ ના instance variable અને local variable અથવા method parameter declare કરી શકાય છે.
- લોકલ વેરીએબલ એ ઈન્સ્ટન્સ વેરિયેબલને hide કરે છે જેને Variable Hiding કહે છે.

  Example:

```
class A
{
        int v = 5;
        public static void main(String args[])
        {
                A a1 = new A();
                a1.method(20);
                a1.method();
        }
        void method(int variable)
        {
                int  v = 10;
                System.out.println("Value of Instance variable :" + this.v);
                System.out.println("Value of Local variable :" + v);
        }
        void method()
        {
                int v = 40;
                System.out.println("Value of Instance variable :" + this.v);
                System.out.println("Value of Local variable :" + v);
        }
```

```
}
```

Output:

```
Value of Instance variable :5
Value of Local variable :10
Value of Instance variable :5
Value of Local variable :40
```

static Keyword

- જ્યારે member ને static declare કરવામાં આવે છે, ત્યારે તેના class ના કોઈપણ objects બનાવવામાં આવે તે પહેલાં, અને કોઈપણ objects ને refer કરે તે પહેલા.
- methods અને variablesને static declare કરી શકાય. static method નું સૌથી best ઉદાહરણ main() method છે.
- main() ને static declare કરવામાં આવે છે કારણ કે કોઈપણ object create થાય તે પહેલાં તેને call કરવામાં આવે છે.
- જ્યારે class નો object create કરવામાં આવે ત્યારે static variable ની new copy બનાવવામાં આવતી નથી. બધા class same static variable ને share કરે છે.
- static method માં નીચે મુજબ restriction(રિસ્ટ્રિક્શન) છે:
  1) તે બીજા static method ને જ call કરી શકે છે.
  2) તે static data ને જ access કરી શકે છે.
  3) તે માં this અને super keyword ને ઉપયોગ થઈ શકે નહિ.

Example:

```
class staticDemo
{
        static int count=0;      //will get memory only once and retain its value

        staticDemo()
        {
                count++;
                System.out.println(count);
        }
        static
        {
                System.out.println("Static block initialized...");
        }
        static void display()
        {
                System.out.println("Static method call…");
        }
        public static void main(String args[])
        {
                staticDemo s1=new staticDemo();
                staticDemo s2=new staticDemo();
```

```
                staticDemo s3=new staticDemo();
                display();
            }
        }
```

Output:

```
Static block initialized...
1
2
3
Static method call...
```

- જો class ની બહારથી static methodને call કરવી હોય તોતે નીચે મુજબ કરી શકાય:

  classname.method( );

- અહીં, classname એ class નું નામ છે જેમાં static method ને declare કરેલ છે. Static method ને class ના object દ્વારા call કરવાની જરૂર નથી.

final Keyword

- variable ને final declare કરી શકાય છે. final variable ની value ને બદલી શકાય નહિ. તેનો અર્થ એ છે કે variable એ constatnt declare થશે જેની value change ન થઈ શકે.
- જો કોઈપણ method ને final declare કરવામાં આવેલ હોય તો તેને ઓવરરાઈડ ન કરી શકાય.
- જો તમે કોઈપણ class ને final declare કરેલ હોય તો તેને inherit ન કરી શકાય.

Example:

```
class finalDemo
{
    final int b = 100;
    void  m1()
    {
        b = 200;   /* Error generate because we cannot change the value of
                      final variable*/
    }
    public static void main(String args[])
    {
        finalDemo f1 = new finalDemo();
        f1.m1();
    }
}
```

Output:

```
Compile Time Error
```

Method Overloading

- જો class માં એક જ નામની અને જુદા જુદા parameters સાથે ઘણી methods હોય તો તેને મેથડ ઓવરલોડિંગ તરીકે ઓળખવામાં આવે છે.
- મેથડ ઓવરલોડિંગને કમ્પાઈલ ટાઈમ (સ્ટેટિક) પોલીમોર્ફિઝમ તરીકે પણ ઓળખવામાં આવે છે.

- અકે જ નામ ની method માં જુદા જુદા parameters અને જુદી જુદી data type ના parameters નો ઉપયોગ કરવામાં આવે છે.
- જુદી જુદી return types સાથે method નું ઓવરલોડિંગ થઈ શકે નહિ.
- એક જ નામ થી declare થયેલ એક કરતા વધારે method માંથી કઈ method call કરવી તે parameters ની data type અને parameters ની સંખ્યા પર આધાર રાખે છે.
- એક જ નામ ની બે methods નીચે આપેલ કોઈ એક point થી different હોવી જોઈએ,
  1) parameters ની સંખ્યા
  2) parameters ના ડેટા type
  3) parameter નો order

Example:

```
class overloadingDemo
{
        void  sum(int a,int b)
        {
                System.out.println("Sum of (a+b) is:: "+(a+b));
        }
        void  sum(int a,int b,int c)
        {
                System.out.println("Sum of (a+b+c) is:: "+(a+b+c));
        }
        void  sum(double a,double b)
        {
                System.out.println("Sum of double (a+b) is:: "+(a+b));
        }
        public static void main(String args[])
        {
                overloadingDemo o1 = new overloadingDemo();
                o1.sum(10,10);          // call method1
                o1.sum(10,10,10);       // call method2
                o1.sum(10.5,10.5);      // call method3
        }
}
```

Output:

```
Sum of (a+b) is:: 20
Sum of (a+b+c) is:: 30
Sum of double (a+b) is:: 21.0
```

Constructor

- કન્સ્ટ્રક્ટર એ special method છે જેનો ઉપયોગ object ને initialize કરવા માટે થાય છે.
- તે object બનાવતી વખતે automatic call થાય છે.
- **કન્સ્ટ્રક્ટરને define કરવા માટેના નીચે પ્રમાણે બે નિયમો છે:**

1) કન્સ્ટ્રક્ટર નું નામ અને class નું નામ સરખુ જ હોવું જોઈએ.
2) કન્સ્ટ્રક્ટરને કોઈ return type હોતી નથી.

- class constructor ની રીટર્ન type એ class type હોય છે.
- **constructor ના બે type છે.**
  1) Default Constructor
  2) Parameterized constructor

Default Constructor

- જે કન્સ્ટ્રક્ટર માં એકપણ પેરામીટર ન હોય તે કન્સ્ટ્રક્ટર ને default કન્સ્ટ્રક્ટર તરીકે ઓળખવામાં આવે છે
- જો એકપણ class માં કન્સ્ટ્રક્ટર ન બનાવામાં આવે તો compiler દ્વારા તે class માટે default કન્સ્ટ્રક્ટર બનાવે છે.

Example:

```
class A
{
        A()                 //Default constructor
        {
                System.out.println("Default constructor called..");
        }
        public static void main(String args[])
        {
                A a = new A();
        }
}
```

Output:

```
Default constructor called..
```

Parameterized Constructor

- જે કન્સ્ટ્રક્ટર માં પેરામીટર હોય છે તે કન્સ્ટ્રક્ટર ને Parameterized કન્સ્ટ્રક્ટર તરીકે ઓળખવામાં આવે છે
- તેનો ઉપયોગ જુદા જુદા object ને જુદી જુદી values આપવા માટે થાય છે.
- objects બનાવામાં આવે ત્યારે parameter પાસ કરવા જરૂરી છે.
- જો ફક્ત પેરામીટરાઈઝડ કન્સ્ટ્રક્ટરને જ define કરેલ હોય, તો default constructor થી object બનાવી શકાય નહીં. કારણ કે કમ્પાઈલર default constructor બનાવશે નહીં પરંતુ આપણે default constructor ને બનાવવું પડશે.

Example:

```
class A
{
        int a;
String s1;
A(int b,String s2)        //Parameterized constructor
{
        b = a;
        s2 = s1;
}
void display()
{
        System.out.println("Value of parameterized constructor is :: "+a+" and
```

```
"+b);
        }
        public static void main(String args[])
        {
            A a = new A(10,"Hello");
            a.display();
        }
    }
```

Output:

```
Value of parameterized constructor is :: 10 and Hello
```

Copy Constructor

- copy કન્સ્ટ્રક્ટર એક કન્સ્ટ્રક્ટર છે જે માં ફક્ત એક જ parameter pass કરવામાં આવે છે અને તે જ class માં કન્સ્ટ્રક્ટર define કરેલ હોય તે class ની type નો હોય છે.
- Copy constructorનો ઉપયોગ parameter તરીક પાસ કરેલા object ની new copy બનાવવા માટે થાય છે પરંતુ new copy એ original object થી independent હોય છે.
- તે independent નો meaning એ છે કે નવા object નું memory location original object ના momory location થી અલગ હોય છે.

Overloading Constructor

- એક class ની માં એક કરતા વધારે Constructor define કરી શકાય છે તેને Constructor overloading તરીકે ઓળખવામાં આવે છે.
- તે મેથડ ઓવરલોડિંગ જેવુજ છે. કન્સ્ટ્રક્ટર ઓવરલોડિંગમાં તમારી પાસે different signature સાથે multiple constructors define કરેલા હોય છે અને difference એ છે કે કન્સ્ટ્રક્ટર પાસે રીટર્ન type હોતી નથી.
- આ types ના કન્સ્ટ્રક્ટર ને overloaded constructor તરીકે ઓળખાય છે.

Passing object as a parameter

- જો કોઈ નવો object બનાવવો હોય જે existing object જેવો જ હોય ત્યારે class માં constructor બનાવામાં આવે છે જેમાં paramater તરીકે class નો object pass કરવામાં આવે છે.

Example:

```
class Box
{
    double width;
    double height;
    double depth;
    // It takes an object of type Box. Copy constructor
    Box(Box ob)
    {
        // pass object to constructor
        width = ob.width;
        height = ob.height;
        depth = ob.depth;
    }
```

```
// Parameterized constructor
Box(double w, double h, double d)
{
        width = w;
        height = h;
        depth = d;
}
// Default constructor
Box()
{
        width = -1; // use -1 to indicate
        height = -1; // an uninitialized
        depth = -1; // box
}
// constructor used when cube is created
Box(double len)
{
        width = height = depth = len;
}
// compute and return volume
double volume()
        {
                return width * height * depth;
        }
}
class DemoAllCons
{
        public static void main(String args[])
        {
                Box mybox1 = new Box(10, 20, 15);
                Box mybox2 = new Box();
                Box mycube = new Box(7);
                Box myclone = new Box(mybox1); // create copy of mybox1
                double vol;
// get volume of first box
vol = mybox1.volume();
System.out.println("Volume of mybox1 is " + vol);
// get volume of second box
vol = mybox2.volume();
System.out.println("Volume of mybox2 is " + vol);
// get volume of cube
vol = mycube.volume();
```

```
                System.out.println("Volume of cube is " + vol);
                // get volume of clone
                vol = myclone.volume();
                System.out.println("Volume of clone is " + vol);
            }
        }
```

Output:

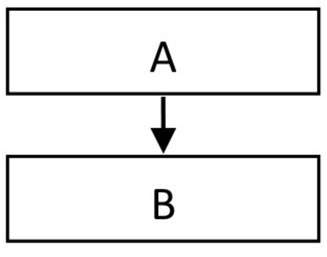
```
Volume of mybox1 is 3000.0
Volume of mybox2 is -1.0
Volume of cube is 343.0
Volume of clone is 3000.0
```

Inheritance

- Inheritance એ process છે, જેનાથી class તેના parent class ની properties અને methods ને inherit કરી શકે છે.
- Old parent class માંથી નવો child class બનાવવાની process ને inheritance કહેવામાં આવે છે.
- નવા class ને derived class અને old class ને base class કહેવામાં આવે છે.
- જ્યારે તમે existing class માંથી inherit કરો છો, ત્યારે તમે parent class ની methods and fields નો reuse કરી શકો છો, અને તમે નવી methods અને fields ને add કરી શકો છો.
- Private properties સિવાય super class ની તમામ properties તેના subclass માં extends કીવર્ડનો ઉપયોગ કરીને inherit કરી શકાય છે.

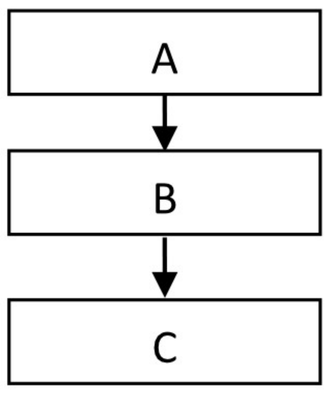
Types of Inheritance

Single Inheritance



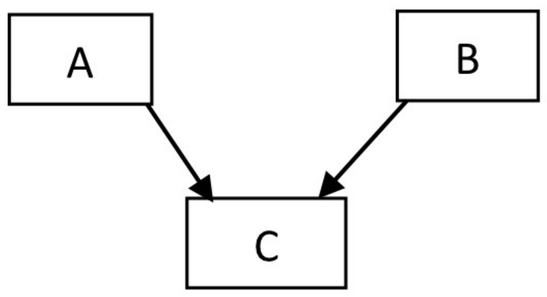
- જો કોઈ class એક જ classમાં થી derived થતો હોય તો તેને single inheritance કહેવામાં આવે છે.
- Class B એ class A માંથી derived થાય છે

Multilevel Inheritance



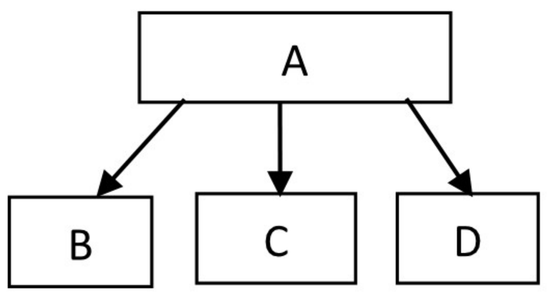
- જો class એ બીજા class માંથી derived થતો હોય અને તે class પણ બીજા class માંથી derived થતો હોય તો તેને multilevel inheritance કહેવામાં આવે છે.
- અહીં, class C એ class B માંથી derive થયો છે અને class B એ class A માંથી derived થયો છે, તેથી તેને multilevel inheritance કહેવામાં આવે છે.

Multiple Inheritance



- જો class એક કરતા વધારે class માંથી derived થાય તો તે multiple inheritance કહે છે.
- તે જાવા માં class દ્વારા સપોર્ટેડ નથી.

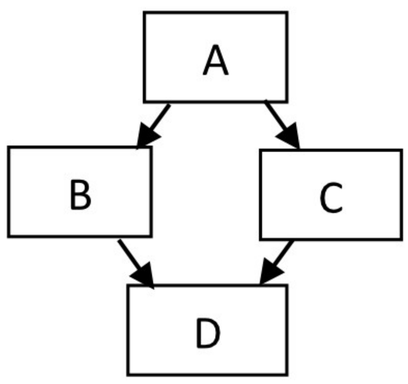
Hierarchical Inheritance



- જો એક કરતા વધુ class એક class માંથી derived કરવામાં આવે તો તેને hierarchical inheritance કહેવામાં આવે છે.
- અહીં class B, class C અને class D એ class A માંથી derived થયા છે.

Hybrid Inheritance

- Hybrid inheritance એ single અને multiple inheritanceનું combination છે.
- જાવા multiple inheritance ને support નથી કરતુ , તેથી hybrid inheritance શક્ય નથી.



Example

```
class A
{
    public  void displayA()
    {
        System.out.println("class A method");
    }
}
class B extends A                    //Single Inheritance - class B is derived from class A
{
    public  void displayB()
    {
        System.out.println("class B method");
    }
}
class C extends B   // Multilevel Inheritance - class C is derived from class B
{
    public  void displayC()
    {
        System.out.println("class C method");
    }
}
class D extends A  //Hierarchical Inheritance - Class B and Class D are derived from Class A
{
    public  void displayD()
    {
        System.out.println("class D method");
    }
}
class  Trial
{
    public  static  void  main(String []args)
    {
        B b=new B();
        C c=new C();
```

```
        D d=new D();
        b.displayB();
        c.displayC();
        d.displayD();
    }
}
```

- Class B અને class D એ class A માંથી derive થયા છે તેથી તે Hierarchical Inheritance નું ઉદાહરણ છે.

Method Overriding (મેથડ ઓવરાઈડિંગ)

- જ્યારે subclass માં કોઈ method declare કરવામાં આવે જેનું નામ અને parameters તેના સુપરક્લાસમાં declare કરેલ method જેવા જ હોય, ત્યારે sub class ની method super class ની method ને ઓવરરાઈડ કરે છે.
- ઓવરરાઈડિંગનો ફાયદો એ છે: subclass specific behavior ને define કરી શકાય છે. જેનો અર્થ છે કે subclass તેની જરૂરિયાતને આધારે superclass ની method ને implement કરી શકે છે.
- રનટાઈમ પોલિમોર્ફિઝમ માટે મેથડ ઓવરરાઈડિંગનો ઉપયોગ થાય છે.
- Object oriented ની language માં , ઓવરરાઈડિંગનો અર્થ કોઈપણ existing method ની functionality ને ઓવરરાઈડ કરવી એવો થાય છે.

Example:

```
class A
{
    public void display()
    {
        System.out.println("Class A");
    }
}
class B extends A
{
    public void display()
    {
        System.out.println("Class B");
    }
}
class Trial
{
    public static void main(String args[])
    {
        A a = new A();          // A reference and object
        A b = new B();          // A reference but B object
        a.display();            // Runs the method in A class
        b.display();            // Runs the method in B class
    }
}
```

Output:

```
Class A
```

Class B

- ઉપરના ઉદાહરણમાં, તમે જોઈ શકો છો કે b એ A type નો હોવા છતાં તે class B ની display () method ને call કરે છે.
- કારણ કે, કમ્પાઈલ સમયે object નો type check કરવામાં આવે છે. જો કે, રનટાઈમ એ JVM object નો type નક્કી કરે છે અને તે જ method ને call કરે જે particular object ને અનુરૂપ હોય.

Rules for method overriding

- Overridden થયેલ methods માં arguments list same હોવા જોયે.
- રીટર્ન type એ સુપરક્લાસમાં original overridden methodમાં declared કરેલા રીટર્ન type જેવો હોવો જોઈએ.
- જો subclass દ્વારા inherit કરવામાં આવે તો જ Instance methods ઓવરરાઈડ કરી શકાય છે.
- Final declared કરાયેલ method ને ઓવરરાઈડ કરી શકાતી નથી.
- Static declare કરાયેલ method ને ઓવરરાઈડ કરી શકાતી નથી પરંતુ ફરીથી declare કરી શકાય છે.
- જો કોઈ method inherit કરી શકાતી નથી, તો તેને ઓવરરાઈડ કરી શકશે નહીં.
- Same package માં subclass એ super class ની કોઈ પણ instance method જે private અને final declare કરેલ ના હોય તેને override કરી શકે છે.
- Different package માં subclass એ super class ની કોઈ પણ method જે final declare કરેલ ના હોય અને public અથવા protected declare કરેલ હોય તેને override કરી શકે છે.
- Constructors ને ઓવરરાઈડ કરી શકાતા નથી.

super keyword

- જાવામાં સુપર કીવર્ડ નો use immediate parent class ના object ને refer કરવા માટે થયા છે.

super To Access super-class Members

- Method override થાય ત્યારે overridden method ને super keyword નો use કરી ને call કરી શકાય છે

Example:

```
class A
{
    String name = "Class A";
    public  void  display()
    {
            System.out.println("Class A display method called..");
    }
}
class B extends A
{
    String name = "Class B";
    public  void  display()
    {
            System.out.println("Class B display method called..");
    }
    void  printName()
    {
```

```
//this will print value of name from subclass(B)
System.out.println("Name from subclass : " + name);

// this will print value of name from superclass(A)
System.out.println("Name from Superclass: " + super.name);

//invoke display() method of Class B method
display();

                // invoke display() method of class A(superclass) using super
                super.display();
        }
    }
    class SuperDemo
    {
        public static void main(String args[])
        {
                B b1 = new B();
                b1.printName();
        }
    }
```

Output:

```
Name from subclass : Class B
Name from Superclass: Class A
Class B display method called..
Class A display method called..
```

- અહીં, સબક્લાસની display() method તેના સુપર-ક્લાસની display() method ને ઓવરરાઈડ કરે છે.
- તેથી, સબક્લાસ (B) ની અંદર સુપરક્લાસ (A) members ને call કરવા માટે સુપર કીવર્ડ વપરાય છે.

super To Call super-class Constructor

- દરેક વખતે સબક્લાસનું પેરામીટરાઈઝ્ડ અથવા નોન-પેરામેટરાઈઝ્ડ કન્સ્ટ્રક્ટર બનાવવામાં આવે છે, by default સુપરક્લાસના default કન્સ્ટ્રક્ટરને implicitly call કરવામાં આવે છે.
- સુપરક્લાસ કન્સ્ટ્રક્ટરને call કરવા માટે syntax:

```
super();
```
OR
```
super(parameter list);
```

- નીચે આપેલ ઉદાહરણ બતાવે છે કે સુપરક્લાસ ના કન્સ્ટ્રક્ટરને call કરવા માટે સુપર કીવર્ડનો ઉપયોગ કેવી રીતે કરવો.

Example:

```
class A
{
    A()
    {
            System.out.println("Super class default constructor called..");
    }
    A(String s1)
```

```
    {
        System.out.println("Super class parameterized constructor called: "+s1);
    }
}
class B extends A
{
    /* Implicitly default constructor of superclass(A) will be called. Whether you define super or not in subclass(B) constructor */
    B()
    {
        System.out.println("Sub class default constructor called..");
    }
    /* To call a parameterized constructor of superclass(A) you must write super() with same number of arguments*/
    B(String s1)
    {
        super("Class A");
        System.out.println("Sub class parameterized constructor called: " + s1);
     }
}
class SuperConDemo
{
    public static void main(String args[])
        {
            B b1 = new B();
            B b2 = new B("Class B");
        }
    }
```

Output:

```
Super class default constructor called..
Sub class default constructor called..
Super class parameterized constructor called: Class A
Sub class parameterized constructor called: Class B
```

- અહી superclass નું default constructor automatic call થાય છે જયારે superclass ના parameterized constructor ને call કરવા માટે super keyword અને matching parameters નો use કરવામાં આવે છે.

Dynamic Method Dispatch

- Dynamic method dispatch નો use overridden method ના call ને run time એ resolve કરે છે. Compile time નહિ.
- તે રન-ટાઈમ પોલિમોર્ફિઝમ તરીકે પણ ઓળખાય છે.
- સુપરક્લાસ એ સબક્લાસ ના object ને refer કરી શકે છે. તે અપકાસ્ટિંગ તરીકે ઓળખાય છે.
- જ્યારે ઓવરરાઈડ method ને સુપરક્લાસ ના reference દ્વારા call કરવામાં આવે છે, ત્યારે કઈ method call થાશે તેનું decision reference variable કયા object ને refer કરે છે તેના પર આધાર રાખે છે.
- આ decision રનટાઈમ પર કરવામાં આવે છે.

Example:

```
class   A
{
    void callme()
    {
            System.out.println("Inside A's callme method");
    }
}
class B extends A
{
    void callme()
    {
            System.out.println("Inside B's callme method");
    }
}
class C extends A
{
    void callme()
    {
            System.out.println("Inside C's callme method");
    }
}
class DispatchDemo
{
    public static void main(String args[])
    {
            A a = new A();          // object of type A
            B b = new B();          // object of type B
            C c = new C();          // object of type C
            A r;                    // obtain r a reference of type A
            r = a;                  // r refers to an A object
            r.callme();             // calls A's version of callme()
            r = b;                  // r refers to a B object
            r.callme();             // calls B's version of callme()
            r = c;                  // r refers to a C object
            r.callme();             // calls C's version of callme()
    }
}
```

Output:

```
Inside A's callme method
Inside B's callme method
Inside C's callme method
```

Object Class

- Object class (java.lang.Object) એ java માં બધા class નો super class છે

- Object class એ બધા class ને common behaviors આપે છે જેવા કે object ને convert કરવા, object ને string માં convert કરવા, object ને notified કરવા etc..
- **જાવા object class ની methods નીચે આપેલ છે:**

Method	Description
equals()	બે object ને compare કરવામાટે
getClass()	object ના class નું રનટાઈમ representation આપે છે. આનો ઉપયોગ કરીને આપણે class નું નામ, તેના સુપરક્લાસ વગેરે જેવી class વિશેની માહિતી મેળવી શકીએ છીએ.
toString()	object નું string representation આપે છે.
notify()	આ object's ના મોનિટરની રાહ જોતા, એક થ્રેડ ને Wake up કરે છે.
notifyAll()	આ object's ના મોનિટરની રાહ જોતા,બધા થ્રેડ ને Wake up કરે છે.
wait()	અન્ય થ્રેડ notifies ન કરે ત્યાં સુધી, current thread ને રાહ જોવાનું કહે છે.

Example:

```
class parent
{
    inti = 10;
    Integer i1 = new Integer(i);
    void PrintClassName(Object obj)      // Pass object of class as an argument
    {
        System.out.println("The Object's class name is ::" + obj.getClass().getName());
    }
}
class ObjectClassDemo
{
    public static void main(String args[])
    {
        parent a1 = new parent();
        a1.PrintClassName(a1);
        System.out.println("String representation of object i1 is:: "+a1.i1.toString());

    }
}
```

Output:

```
The Object's class name is :: parent
String representation of object i1 is:: 10
```

Packages

- જાવા પેકેજ એ same type ના classes, interfaces અને sub-packages નું group છે.
- Packages નો ઉપયોગ naming conflicts ને રોકવા માટે કરવામાં આવે છે અને access protection આપે છે.
- તેનો ઉપયોગ classes અને interfaces ના categorize માટે પણ થાય છે. જેથી તેઓ સરળતાથી maintain કરી શકાય.
- subpackage નો use કરી ને package ને categorize કરી શકાય છે . પેકેજની અંદરના પેકેજને subpackage કહેવામાં આવે છે.
- પેકેજને બે form માં categorized કરી શકાય છે: built-in package અને user-defined package.

- **built-in packages :** હાલના જાવા પેકેજ જેમ કે **java.lang, java.util, java.io, java.net, java.awt.**
- **User-defined-package :** classes અને interface ને categorize કરવા માટે user દ્વારા બનાવેલ જાવા પેકેજ.

- Programmers પોતાના classes અને interfaces નું પોતાનું package બનાવી શકે છે.

Creating a package

- પેકેજ બનાવવા માટે, package statement પછી package નું name આપવાનું હોય છે.
- package statement એ source file માં પ્રથમ લીટી હોવું જોઈએ. દરેક source file માં ફક્ત એક જ પેકેજ સ્ટેટમેન્ટ હોઈ શકે છે.
- જો પેકેજ સ્ટેટમેન્ટનો ઉપયોગ ન કરવામાં આવે તો class, interface વગેરે unnamed package માં મૂકવામાં આવશે.

Example:

```
package mypack;
class Book
{
    String bookname;
    String author;
    Book()
    {
        bookname = "Complete Reference";
        author = "Herbert";
    }
    void show()
    {
        System.out.println("Book name is :: "+bookname+"\nand author name is :: "+
author);
    }
}
class DemoPackage
{
    public static void main(String[] args)
    {
        Book b1 = new Book();
        b1.show();
    }
}
```

Compile: javac -d . DemoPackage.java
To run the program : java mypack.DemoPackage
Output:

```
Book name is :: Complete Reference
and author name is :: Herbert
```

- -d એ સ્વીચ છે જે કમ્પાઈલરને ક્લાસ ફાઈલ ક્યાં મૂકવી તે કહે છે. જેમ કે, /home (Linux), d:/abc (windows) etc.
- . (ડોટ) current folder ને represent કરે છે.

Import Package

- import keyword નો ઉપયોગ java source file માં built-in અને user-defined packages ને import કરવા માટે થાય છે.
- જો class same પેકેજમાં બીજો class use કરવા માંગે છે, તો પેકેજ import કરવાની જરૂર નથી.
- પરંતુ, જો કોઈ class બીજા class નો use કરવા માંગે છે જે એક જ પેકેજમાં નથી, તો import કીવર્ડનો ઉપયોગ તે પેકેજને તમારી java source file import કરવા માટે કરવામાં આવે છે.
- class માં એક કરતા વધારે import statements હોય શકે છે.
- **પેકેજની બહારથી access કરવાની ત્રણ રીતો છે:**
  1) import package.*;
  2) import package.classname;
  3) fully qualified name

1) Using packagename.*:

- જો તમે packagename.* નો use કરો છો. તો પછી આ પેકેજના બધા classes અને interfaces એક્સેસ કરી શકાય પરંતુ subpackages એક્સેસ થશે નહિ
  Syntax: import packagename.*;

2) Using packagename.classname :

- જો તમે packagename.classname નો ઉપયોગ કરો છો તો આ પેકેજનો ફક્ત declared class જ access થશે.
  Syntax: import packagename.**classname**;

3) Using fully qualified name :

- જો તમે fully qualified name નો ઉપયોગ કરો છો તો આ પેકેજનો ફક્ત declared class જ access થશે. તેથી, પેકેજ import કરવાની જરૂર નથી.
- પરંતુ, જ્યારે તમે class અને interface ને access કરો ત્યારે તમારે દર વખતે fully qualified name વાપરવાની જરૂર છે.
- સામાન્ય રીતે જ્યારે બે પેકેજોના નામ same હોય છે ત્યારે તેનો ઉપયોગ થાય છે દા.ત. java.util અને java.sql પેકેજોમાં Date ક્લાસ હોય છે.

Example:

File 1: **A.java**

```
package pack;
public class A
{
        public void display()
        {
                System.out.println("Welcome to package pack...");
        }
}
```

File 2: **Q.java**

```
package pack;
public class Q
{
        public void Q_display()
        {
                System.out.println("Welcome to package pack through qualified
name...");
        }
```

```
}
```

File 3: **B.java**

```
package  mypack;
import  pack.*;            // Here, you can also use    import pack.A ;
class  B
{
        public static void main(String args[])
        {
            A a1 = new A();
            pack.Q  q1= new  pack.Q();
        /*Here Q  is class in package A. If you want to access Q_display() method of class Q
        using fully qualified name then no need to import package named pack */
            a1.display();
            q1.Q_display();
        }
}
```

Output: java mypack.B

```
Welcome to package pack...
Welcome to package pack through qualified name...
```

- ઉપરના ઉદાહરણમાં, class A, class Q અને તેની બધી methods ને public તરીકે declared કરવી આવશ્યક છે નહીં તો તેઓ class Bમાં access થશે નહિ.

Visibility and Access Rights

class \ have access to	Private	Default	Protected	Public
own class	yes	yes	yes	yes
subclass - same package	no	yes	yes	yes
subclass - another package	no	no	yes	yes
class - another package	no	no	no	yes

Interfaces

- ઇંટરફેસ એ abstract methods નું collection છે. ઈન્ટરફેસ એ class નથી.
- જ્યારે તમે ઇંટરફેસ બનાવો છો ત્યારે તે class કેવી રીતે કરશે તેના વિશે કંઈ પણ કહ્યા વિના class શું કરી શકે છે તે defines કરે છે.
- ઈન્ટરફેસમાં ફક્ત static constants and abstract methods ઓ હોય છે.
- જાવામાં ઇંટરફેસ એ fully abstraction આપે છે. જાવા ઈન્ટરફેસમાં abstract methods ઓ હોઈ શકે છે.
- By default (Implicitly) ઈન્ટરફેસ એબ્સ્ટ્રેક્ટ હોય છે, ઈન્ટરફેસ માં fields(data members) એ public, static અને final હોય છે અને methods public અને abstract હોય છે.
- તેનો ઉપયોગ જાવામાં fully abstraction અને multiple inheritance કરવા માટે થાય છે.
- **Class અને ઈન્ટરફેસ વચ્ચે** similarity **નીચે આપેલ છે:**
  - ઈન્ટરફેસમાં ઘણી બધી methods હોઈ શકે છે
  - ઇંટરફેસ ને .java ના extension થી store કરવામાં આવે છે. ઇંટરફેસ file નું નામ અને ઇંટરફેસ નું નામ સરખા હોય છે.

- ઈન્ટરફેસનો bytecode ને .class ફાઈલમાં save કરવામાં આવે છે.

- **class અને ઈન્ટરફેસ વચ્ચેનો તફાવત નીચે આપેલ છે:**

Class	Interface
• તમે class instantiate કરી શકો છો.	• તમે ઈન્ટરફેસ instantiate કરી શકતા નથી.
• તેમાં ડિફોલ્ટ તેમજ પેરામીટરાઈઝડ કન્સ્ટ્રક્ટર છે.	• તેમાં કોઈપણ constructors નથી.
• બધી methods ની definition હોવી જોઈએ અથવા methodsને abstract તરીકે declare કરો.	• ઈન્ટરફેસમાં બધી method એ by default રીતે abstract હોય છે.
• બધા variables by default રીતે instance છે.	• બધા variables by default રીતે static અને final હોય છે, અને definition સમયે value આપવી જરૂરી છે.
• class ફક્ત એક જ class ને inherit કરી શકે છે અને ઘણા ઇંટરફેસ ને implement કરી શકે છે.	• ઇંટરફેસ કોઈપણ class ને inherit કરી શકે નહિ જ્યારે તે ઘણા ઇંટરફેસને extend કરી શકે.

Declaring Interfaces

- ઈન્ટરફેસ કીવર્ડનો ઉપયોગ ઈન્ટરફેસ declare કરવા માટે થાય છે.

Syntax: NameOfInterface.java

```
import java.lang.*;
public interface NameOfInterface
{
        //Any number of final, static fields
        //Any number of abstract method declarations
}
```

Example: DemoInterface.java

```
interface DemoInterface
{
        int i = 10;
        void  demo();
}
```

- ઉપરના ઉદાહરણમાં, ઈન્ટરફેસનું નામ DemoInterface છે અને તેમાં integer typeનો variable i અને demo() નામની abstract method છે.

Implementing Interfaces

- class ઈન્ટરફેસને implement કરવા માટે implements કીવર્ડનો ઉપયોગ થાય છે.
- class એ ઈન્ટરફેસ ને implements કરે છે તેનો અર્થ class એ contract પર sign કરે છે જે ઈન્ટરફેસ નું specific behaviors ને perform કરવા agree હોય છે.
- જો કોઈ class એ interface ના બધા behaviors ને perform કરતો નથી, તો class પોતાને abstract તરીકે declare કરે છે.

Example: DemoInterfaceImp.java

```
public class DemoInterfaceImp implements DemoInterface
{
    public void demo()
    {
            System.out.println("Value of i is :: "+i);
```

```
        }
        public static void main(String args[])
        {
                DemoInterfaceImp  d = new DemoInterfaceImp();
                d.demo();
        }
    }
```

Output:

```
Value of i is :: 10
```

Inheritance on Interfaces

- આપણે બધા જાણીએ છીએ કે એક class બીજા class ને extend કરી શકે છે. એ જ રીતે ઈન્ટરફેસ બીજા ઈન્ટરફેસને extend કરી શકે છે.
- Extends કીવર્ડનો ઉપયોગ ઈન્ટરફેસને extends કરવા માટે થાય છે, અને ચાઈલ્ડ ઇંટરફેસ પેરન્ટ ઈન્ટરફેસની methods ને inherit કરે છે.

Example:

```
public interface  A
{
    void getdata(String name);
}
public  interface  B  extends  A
{
    void setdata();
}
class  InheritInterface  implements  B
{
    String display;
    public void getdata(String name)
    {
            display = name;
    }
    public void setdata()
    {
            System.out.println(display);
    }
    public static void main(String args[])
    {
            InheritInterface obj = new InheritInterface();
            obj.getdata("Welcome TO Heaven");
            obj.setdata();
    }
}
```

Output:

Welcome TO Heaven

- ઈન્ટરફેસ B ની એક method છે, પરંતુ તે ઈન્ટરફેસ A માંથી inherits થાય છે; આમ, B ને implements કરતો class InheritInterface બે method ને implement કરવાની જરૂર છે.

Multiple Inheritance using Interface

- જો class મલ્ટીપલ interfaces ને implements કરે છે, અથવા ઇંટરફેસ મલ્ટીપલ ઈન્ટરફેસને extends કરે છે તો આને multiple inheritance તરીકે ઓળખાય છે.
- class ફક્ત એક parent class ને extend કરી શકે છે. Multiple inheritances allowed નથી. જો કે, ઈન્ટરફેસ એકથી વધુ પેરન્ટ ઈન્ટરફેસને extend કરી શકે છે.
- extends કીવર્ડનો ઉપયોગ એકવાર થાય છે, અને parent interfaces ને (,) થી separated list declared કરવામાં આવે છે.

Example:

```
public interface  A
{
    void  getdata(String name);
}
public  interface  B     /* Here we can also extends multiple interface like  interface B extends A,C*/
{
    void  setdata();
}
class  InheritInterface  implements A, B
{
    String  display;
    public  void getdata(String  name)
    {
            display = name;
    }
    public  void setdata()
    {
            System.out.println(display);
    }
    public  static  void  main(String args[])
    {
            InheritInterface obj = new  InheritInterface();
            obj.getdata("Welcome TO Heaven");
            obj.setdata();
        }
    }
```

Output:

Welcome TO Heaven

Abstract Class

- class જેને abstract keyword સાથે declared કરવામાં આવે છે, તે class ને એબ્સ્ટ્રેક્ટ class તરીકે ઓળખાય છે. તેમાં abstract અને non-abstract methods (method with body) હોઈ શકે છે.

- તેને extended કરવાની અને તેની method ને implement કરવાની જરૂર છે. તેને instantiated કરી શકાતું નથી એટલે આપણે તેનો object બનાવી શકીએ નહીં.
- કોઈપણ class કે જે abstract class ને extends કરે છે તેણે સુપર class દ્વારા declared કરેલી બધી abstract methods ને define કરવી જરૂરી છે.

Syntax:

```
abstract classclass_name
{
        //Number of abstract as well as non-abstract methods.
}
```

- method જેને abstract તરીકે declared કરવામાં આવે છે અને તેમાં implementation નથી, તે abstract method તરીકે ઓળખાય છે.
- method ની body તેના subclass દ્વારા defined કરવામાં આવશે. Abstract method ક્યારેય final અને static હોઈ શકતી નથી.

Syntax:

```
abstract return_typefunction_name ();        // No definition
```

Example:

```
abstract class A
{
    abstract void abs_method();
    public void display()
    {
        System.out.println("This is concrete method..");
    }
}
class DemoAbstract extends A
{
    void abs_method()
    {
        System.out.println("This is an abstract method..");
    }
    public static void main(String[] args)
    {
        DemoAbstract abs = new DemoAbstract();
        abs.abs_method();
        abs.display();
    }
}
```

Output:

```
This is an abstract method..
This is concrete method..
```

Final Keyword

- જાવામાં final કીવર્ડનો ઉપયોગ user ને restrict કરવા માટે થાય છે. જાવા final કીવર્ડ આની સાથે ઉપયોગ કરી શકાય છે:
  1) variable
  2) method
  3) class

- **Final Variable:** જો તમે કોઈપણ variable ને final તરીકે declare કરો છો, તો તમે તે final variable નું value બદલી શકતા નથી (It will be constant).
  - Variable કે જે final તરીકે declared કરવામાં આવે છે અને initialized નથી, તેને blank final variable કહેવામાં આવે છે. એક blank final variable કન્સ્ટ્રકટરોને તેને initialize કરવા માટે forces કરે છે.
- **Final Method:** final તરીકે declared કરાયેલ Methods ને ઓવરરાઈડ કરી શકાતી નથી.
- **Final Class:** જાવા classes ને final તરીકે declared કરી શકાતા નથી, કારણ કે તેનો inherit કરી શકાતો નથી.
- જો તમે કોઈપણ parameter ને final તરીકે declare કરો છો, તો તમે તેની value બદલી શકતા નથી.

Example:

```
class  DemoBase
{
    final  inti = 1;              //final variable must be initialize.
    final  void  display()
    {
            System.out.println("Value of i is :: "+ i);
    }
}
class  DemoFinal  extends  DemoBase
{
    /*void display()        // Compilation error final method cannot override.
    {
            System.out.println("Value of i is :: "+ i);
    }*/
            public  static  void  main(String args[])
            {
                    DemoFinal  obj = new  DemoFinal();
                    obj.display();
            }
        }
```

Output:

Value of iis :: 1

Difference between Method Overloading and overriding

Overriding	Overloading
• argument list ઓવરરાઈડ method ની exactly match થતી હોવી જોઈએ.	• Overloaded methods માં argument list ને બદલવી જરૂરી છે.
• રીટર્ન type સુપર ક્લાસમાં ઓવરરાઈડ method જેવો જ હોવો જોઈએ.	• ઓવરલોડ methods રીટર્ન type બદલી શકે છે.
• Private અને final methods ઓવરરાઈડ કરી શકાતી	• Private અને final methods ઓવરલોડ થઈ શકે છે

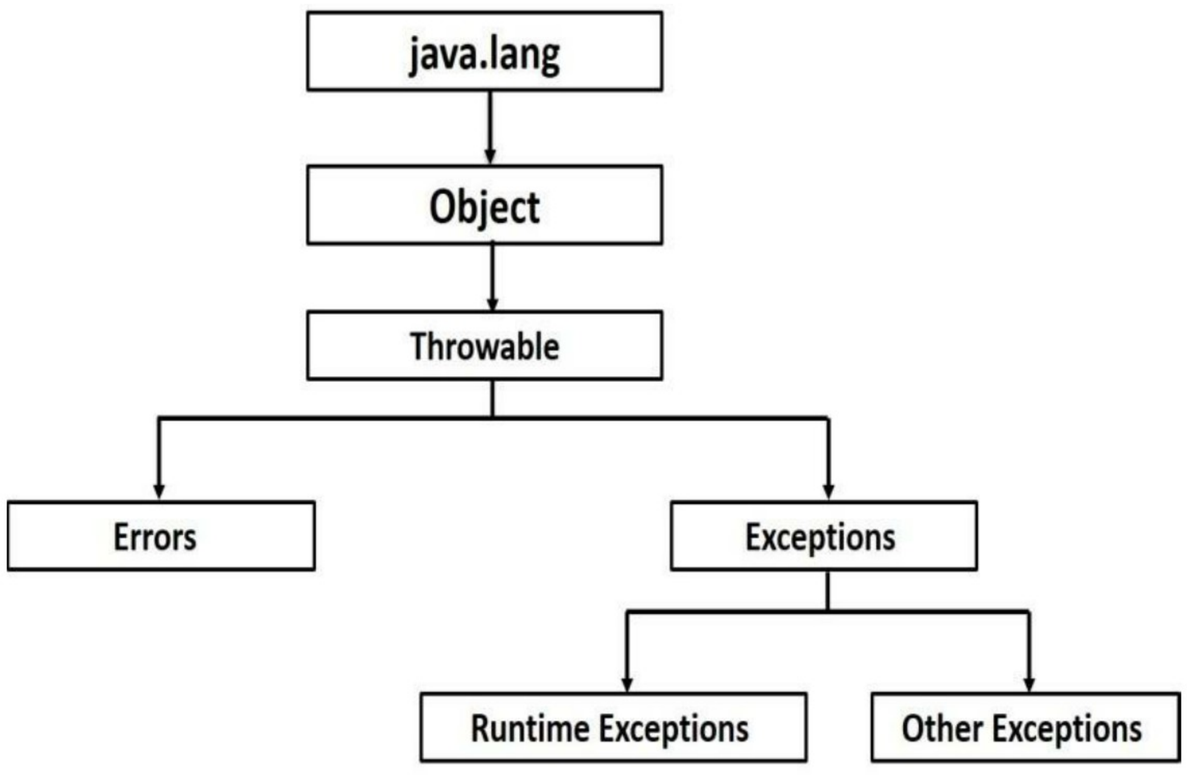
નથી.	
• મેથડ ઓવરરાઈડિંગ એ બે classes માં થાય છે જેમાં inheritance હોય છે.	• Method overloading એ class માં કરવામાં આવે છે.
• Dynamic binding એ overridden methods માટે ઉપયોગમાં લેવાય છે.	• Static binding એ overloaded methods માટે ઉપયોગમાં લેવાય છે.
• મેથડ ઓવરરાઈડિંગ એ રન ટાઈમ પોલિમોર્ફિઝમનું ઉદાહરણ છે.	• મેથડ ઓવરલોડિંગ એ કમ્પાઈલ ટાઈમ પોલિમોર્ફિઝમનું ઉદાહરણ છે.

Difference between Interface and Abstract class

Interface	Abstract class
• ઈન્ટરફેસમાં ફક્ત abstract methods જ હોઈ શકે છે.	• એબ્સ્ટ્રેક્ટ class માં abstract અને non-abstract methods હોઈ શકે છે.
• ઈન્ટરફેસ એ multiple inheritance ને supports કરે છે.	• Abstract class એ multiple inheritance ને support કરતો નથી.
• ઈન્ટરફેસમાં static methods, main method અને constructor હોઈ શકે નથી.	• Abstract class માં static methods, main method અને constructor હોઈ શકે.
• ઇંટરફેસ એ abstract class નું implementation કરી શકતું નથી.	• એબ્સ્ટ્રેક્ટ class એ ઈન્ટરફેસ નું implementation કરી શકે છે.

Exceptions Handling

- Exception (અથવા Exception event) એ એક problem છે જે પ્રોગ્રામના execution દરમિયાન generate થાય છે.
- જ્યારે કોઈ Exception generate થાય છે ત્યારે પ્રોગ્રામનો સામાન્ય flow disrupted થાય છે અને પ્રોગ્રામ / એપ્લિકેશન terminate થાય છે, તેથી આ Exception ને handled કરવાની જરૂર છે.
- જાવા માં Exception એ એક object છે જે પ્રોગ્રામમાં થતાં exception ને describes કરે છે. જ્યારે જાવામાં કોઈ exceptional generate થાય છે, ત્યારે exception ને thrown કરવામાં આવે છે.
- Exception generate ઘણા જુદા જુદા કારણોસર થઈ શકે છે, તેમાંથી કેટલાક નીચે મુજબ છે:
  - User એ invalid ડેટા enter કરે ત્યારે છે.
  - જે ફાઈલને open કરવાની જરૂર છે તે શોધી શકાતી નથી.
  - Communications માં નેટવર્ક કનેક્શન lost થઈ ગયું છે, અથવા JVM પાસે પુરતી મેમરી નથી.
- Users, programmer's અથવા physical resources failed થાવાથી Exception generate થાય છે.
- જાવામાં Exception Handling એ Exception (runtime errors) ને handle કરવા માટે એક powerful method છે, જેથી એપ્લિકેશનનો normal flow જાળવી શકાય.
- જાવામાં Exceptions ની ત્રણ categories છે:
  1) **Checked exceptions:** checked exception એ exception છે જે કમ્પાઈલ સમયે આવે છે, આને compile time exceptions પણ કહેવામાં આવે છે. ઉદાહરણ, IOException, SQLException વગેરે.
  2) **Runtime exceptions:** Unchecked exception એ exception છે જે execution દરમિયાન આવે છે, એને Runtime Exceptions તરીકે પણ ઓળખવામાં આવે છે. આમાં પ્રોગ્રામિંગ bugs હોય છે, જેમ કે logic errors અથવા API નો improper ઉપયોગ.
     - Compilation સમયે રનટાઈમ exception ને ignored કરવામાં આવે છે.
     - **Example:** ArithmeticException, NullPointerException, Array Index out of Bound exception.
  3) **Errors:** આ કોઈ પણ exceptions નથી, પરંતુ problems કે જે user અથવા programmer ના control બહાર છે. **Example:** OutOfMemoryError, VirtualMachineErrorException.
- બધા exception classes એ java.lang.Exception class ના subtypes છે. Exception class એ Throwable class નો subclass છે. exception class સિવાય અન્ય એક subclass છે જેને Error કહેવામાં આવે છે જે Throwable class માંથી derived કરવામાં આવ્યો છે.



- Errors સામાન્ય રીતે જાવા પ્રોગ્રામ્સમાં હોતી નથી. આ conditions સામાન્ય રીતે failuresના caseમાં થાય છે, જે જાવા programs દ્વારા handle કરવામાં આવતી નથી. Runtime environment દ્વારા generate થતી Errors સૂચવવા માટે Errors generated થાય છે. ઉદાહરણ: JVM ને મેમરી નથી છે. સામાન્ય રીતે પ્રોગ્રામ્સ errors માંથી recover થઈ શકતા નથી.
- Exception class માં બે main subclasses છે: IOException અને and RuntimeException class

Exception Handling Mechanism

- Exception handling પાંચ કીવર્ડ્સનો ઉપયોગ કરીને કરવામાં આવે છે:
  1) try
  2) catch
  3) finally
  4) throw
  5) throws

Using try and catch

1) try block :

- જાવા માં try block નો ઉપયોગ કોડને લખવામાં થાય છે જે કદાચ exception throw કરી શકે.
- જાવા માં try block પછી કેચ અથવા finally block કરવું જરૂરી છે.

2) catch block:

- Exceptionને હેન્ડલ કરવા માટે catch block નો ઉપયોગ થાય છે. તેનો ઉપયોગ ફક્ત try block પછી જ થવો જોઈએ.
- કેચ બ્લોકને checked કરવામાં આવે છે, જો કે જે exception આવે છે તે type કેચ બ્લોકમાં આપવામાં આવે છે, તો exception તે કેચ બ્લોકને સોંપવામાં આવે છે.
- તમે single try ની સાથે multiple catch blockનો ઉપયોગ કરી શકો છો.

Syntax:

```
try
{
        //Protected code
}
catch(ExceptionName1  e1)
{
        //Catch block 1
}
catch(ExceptionName2  e2)
{
        //Catch block 2
}
```

- ઉપરની syntaxમાં, ત્યાં બે કેચ બ્લોક્સ છે. ટ્રાય બ્લોકમાં, કોડ લખીએ છીએ જે exception generate કરે છે. જો protected code દ્વારા generate થયેલ exception ને first catch block સાથે match કરવામાં આવશે.
- જો thrown exception નો ડેટા type ExceptionName1 સાથે match થાય છે, તો તે ત્યાં catch થઈ જાય છે અને કેચ બ્લોક execute થાય છે.
- જો નહીં, તો exception ને second કેચ બ્લોકમાં check કરવામાં આવે છે.
- exception catch થાય ત્યાં સુધી અથવા તમામ catch check ન થાય ત્યાં સુધી ચાલુ રહે છે, તે case માં current Method નું execution અટકાવે છે.

Example:

```
class  demoTry
{
        public static void main(String[]  args)
        {
                try
                {
                        int arr[]={1,2,3};
                        arr[3]=3/0;
                }
                catch(ArithmeticException  ae)
                {
                        System.out.println("Divide by zero  :: " + ae);
                }
                catch(ArrayIndexOutOfBoundsException  e)
                {
                        System.out.println("Array index out of bound exception  :: "+e);
                }
        }
    }
```

Output:

divide by zero :: java.lang.ArithmeticException: / by zero

Nested Try-Catch Blocks

- જાવામાં, try block use બીજા try block ની અંદર કરવામાં આવે તેને નેસ્ટેડ ટ્રાય બ્લોક તરીકે ઓળખવામાં આવે છે.
- નેસ્ટેડ try block નો ઉપયોગ ત્યારે થાય છે જ્યારે બ્લોકના એક ભાગમાં એક error આવે છે ત્યારે આખા બ્લોકમાં બીજી error generate થાય છે.
- જો અંદરના try blockમાં કોઈ particular exception માટે કેચ હેન્ડલર નથી, તો બહારના try ના catch ને check કરવામાં આવે છે.
- આ checking કેચ સ્ટેટમેન્ટ્સમાંથી એક match થાય ત્યાં સુધી ચાલુ રહે છે, અથવા ત્યાં સુધી entire નેસ્ટેડ ટ્રાય સ્ટેટમેન્ટ્સ પૂર્ણ ન થાય ત્યાં સુધી. જો કોઈ કેચ સ્ટેટમેન્ટ્સ મેચ થતું નથી, તો જાવા રન-ટાઈમ સિસ્ટમ exceptionને હેન્ડલ કરે છે.

Syntax:

```
try
{
        Statement 1;
        try
        {
                //Protected code
        }
        catch(ExceptionName  e1)
        {
                //Catch block1
        }
}
catch(ExceptionName1  e2)
```

```
{
        //Catch block 2
}
```

Example:

```
class demoTry1
{
        public static void main(String[]  args)
        {
                try
{
        int  arr[]={5,0,1,2};
        try
        {
                arr[4] = arr[3]/arr[1];
        }
        catch(ArithmeticException e)
        {
                System.out.println("divide by zero :: "+e);
        }
}
catch(ArrayIndexOutOfBoundsException e)
{
        System.out.println("array index out of bound exception :: "+e);
}
                catch(Exception e)
                {
                        System.out.println("Generic exception :: "+e);
                }
                System.out.println("Out of try..catch block");
        }
}
```

Output:

```
divide by zero :: java.lang.ArithmeticException: / by zero
Out of try..catch block
```

3) finally:

- Finally keywordનો ઉપયોગ કોડનો બ્લોક બનાવવા માટે થાય છે જે try અથવા catch block ને follow કરે છે.
- Finally block હંમેશાં execute થાય છે exception આવે કે નહીં.
- કેચ બ્લોક પછી માં finally block આવે છે.

Syntax:

```
try
{
        //Protected code
}
catch(ExceptionType1  e1)
{
```

```
            //Catch block 1
}
catch(ExceptionType2 e2)
{
            //Catch block 2
}
finally
{
            //The finally block always executes.
}
```

Example:

```
class  demoFinally
{
        public static void main(String args[])
        {
                int  a[] = new int[2];
                try
                {
                        System.out.println("Access element three :" + a[3]);
                }
                catch(ArrayIndexOutOfBoundsException e)
                {
                        System.out.println("Exception thrown  :" + e);
                }
                finally
                {
                        a[0] = 10;
                        System.out.println("First element value: " +a[0]);
                        System.out.println("The finally block is always executed");
                }
                System.out.println("Out of try...catch...finally... ");
        }
}
```

Output:

```
Exception thrown  :java.lang.ArrayIndexOutOfBoundsException: 3
First element value: 10
The finally block is always executed
Out of try...catch...finally...
```

Key points to keep in mind:

- try statement વગર catch block use થઈ શકે નહિ.
- દરેક try/catch માટે finally block હોવો ફરજીયાત નથી.
- try block એ catch block અથવા finally block વગર use થઈ શકે નહિ.

- કોઈપણ કોડ try, catch, finally blocks ની વચ્ચે present હોઈ શકતો નથી.

4) throw :

- throw keyword નો ઉપયોગ explicitly exception ને throw કરવા માટે થાય છે.
- throw કીવર્ડનો ઉપયોગ કરીને આપણે કાં તો checked અથવા uncheked exception throw કરી શકીએ.
- ફક્ત Throwable class અથવા તેના subclasses નો object ને thrown કરી શકાય છે.
- પ્રોગ્રામ નું એક્ઝેક્યુશન થ્રો સ્ટેટમેન્ટ પર અટકે છે, અને closest catch statement ને exception માટે match કરવામાં આવે છે.

Syntax:

```
throw  ThrowableInstance;
```

Example:

```
class demoThrow
{
        static void demo()
        {
                try
                {
                        throw new ArithmeticException("demo");
                }
                catch(ArithmeticException  e)
                {
                        System.out.println("Exception caught");
                }
        }
        public static void main(String args[])
        {
                demo();
        }
}
```

Output:

```
Exception caught
```

5) throws:

- throws keywordનો ઉપયોગ exception ને declare કરવા માટે થાય છે
- જો કોઈ method એ checked exceptionને હેન્ડલ કરતી નથી, તો method એ throws keywordનો ઉપયોગ કરીને તેને declare કરવી જોઈએ. throws keyword એ method's signature ની અંતમાં આવે છે.
- તમે multiple exceptions declare કરી શકો છો.

Syntax:

```
return_type  method_name()  throws  exception_class_name_list
{
        //method code
}
```

Example:

```
class  demoThrows
```

```
{
        static void display() throws ArithmeticException
        {
                System.out.println("Inside check function");
                throw new ArithmeticException("Demo");
        }
        public static void main(String args[])
        {
                try
                {
                        display();
                }
                catch(ArithmeticException e)
                {
                        System.out.println("caught :: " + e);
                }
        }
}
```

Output:

```
Inside check function
caught :: java.lang.ArithmeticException: Demo
```

User Defined Exception

- જાવામાં, આપણે આપણું પોતાનું exception બનાવી શકીએ જે કસ્ટમ exception અથવા user-defined exception તરીકે ઓળખાય છે.
- આપણો પોતાનો exception અને message હોઈ શકે છે.

Key points to keep in mind:

- બધા exceptions એ Throwable class ના child હોવા જરૂરી છે.
- જો તમે checked exception લખવા માંગતા હોવ કે જે Declare Rule દ્વારા automatically લાગુ કરવામાં આવે છે, તો તમારે Exception class extend કરવાની જરૂર છે.
- જો તમે runtime exception ને write કરવા માંગતા હો, તો તમારે RuntimeException class ને extend કરવાની જરૂર છે.

Example:

```
class demoUserException extends Exception
{
private int ex;
demoUserException(int a)
{
        ex=a;
}
public String toString()
{
        return "MyException[" + ex +"] is less than zero";
}
```

```
}
class demoException
{
        static void sum(int a,int b) throws demoUserException
        {
                if(a<0)
                {
                        throw new demoUserException (a);
                }
                else
                {
                        System.out.println(a+b);
                }
}
public static void main(String[] args)
{
        try
        {
                sum(-10, 10);
        }
        catch(demoUserException e)
        {
                System.out.println(e);
        }
    }
```

Output:

MyException[-10] is less than zero

List of Java Exception (Built-In Exception)

- જાવા standard પેકેજ java.lang અંદર ઘણા બિલ્ટ-ઈન exception classes ને defines કરે છે.

Checked Exception:

Exception	Description
ClassNotFoundException	આ Exception ત્યારે આવે છે જ્યારે જાવા રન-ટાઈમ સિસ્ટમ પ્રોગ્રામમાં specified class ને શોધવામાં નિષ્ફળ જાય છે.
IllegalAccessException	આ Exception ત્યારે આવે છે જ્યારે તમે કોઈ abstract class અને interfaceનો object બનાવો છો.
NotSuchMethodException	આ Exception ત્યારે આવે છે જ્યારે તમે call કરો છો તે method class માં અસ્તિત્વમાં નથી.
NoSuchFieldException	requested કરેલ field અસ્તિત્વમાં નથી.

Unchecked Exception:

Exception	Description
ArithmeticException	આ Exception આવે છે, જ્યારે તમે કોઈ સંખ્યાને શૂન્યથી divide કરો છો ત્યારે

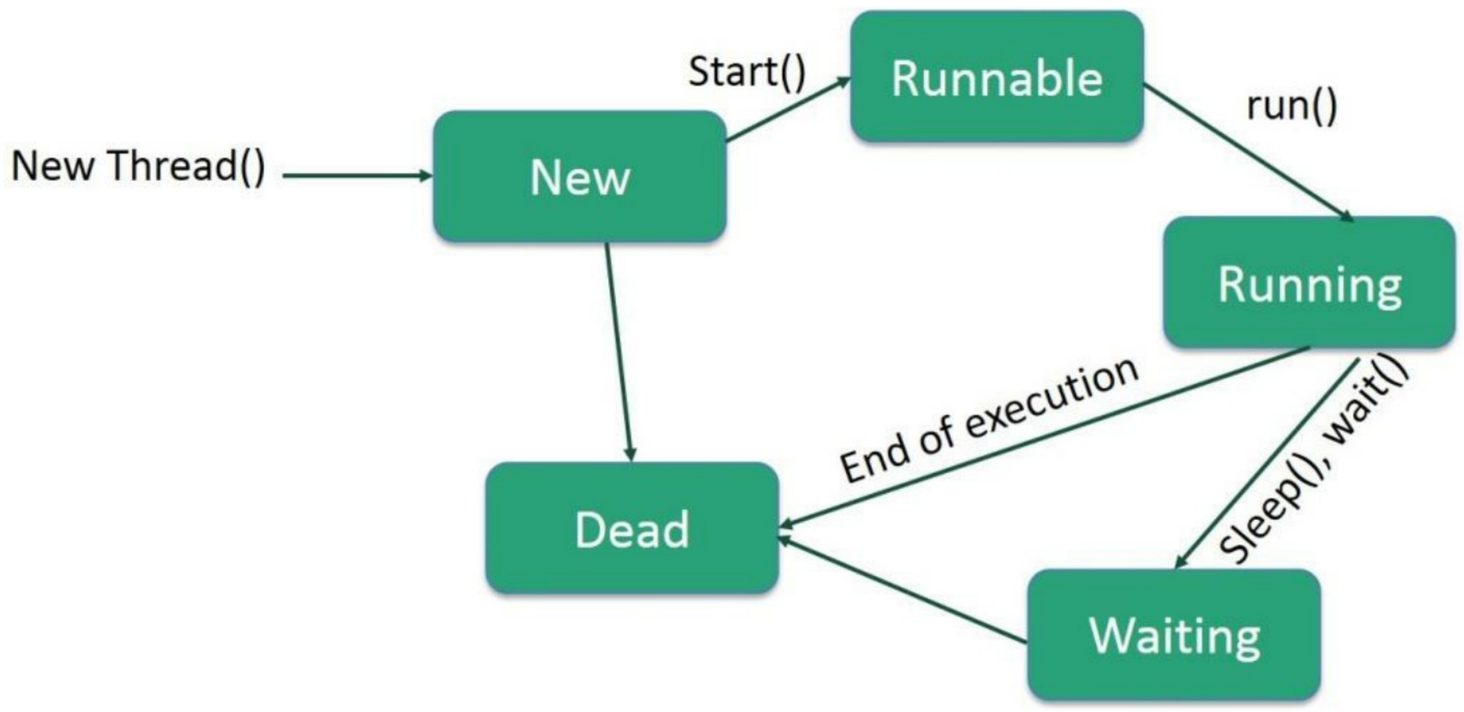
	Arithmetic Exception આવે છે.
ArrayIndexOutOfBounds Exception	આ Exception આવે છે, જ્યારે તમે કોઈ array ની size કરતા વધારે index થી array ના element ને access કરો છો.
NumberFormatException	આ Exception આવે છે, જ્યારે તમે ખોટા format વાળા string variable ને integer (numeric forma) માં convert કરવાનો પ્રયાસ કરો છો જે એકબીજા સાથે compatible નથી.
ClassCastException	Invalid cast.
NullPointerException	null reference નો Invalid ઉપયોગ.

Multithreading

- જાવા એ મલ્ટિથ્રેડેડ પ્રોગ્રામિંગ લેંગ્વેજ છે, જેનો અર્થ છે કે આપણે જાવાનો ઉપયોગ કરીને મલ્ટિ થ્રેડેડ પ્રોગ્રામ બનાવી શકીએ.
- મલ્ટિથ્રેડેડ પ્રોગ્રામ્સમાં બે અથવા વધુ થ્રેડો હોય છે જે એક સાથે run થઈ શકે છે. આનો અર્થ એ છે કે એક જ પ્રોગ્રામ એક સાથે બે અથવા વધુ કાર્યો કરી શકે છે.
- થ્રેડ એ lightweight sub-process છે, જે processing નો સૌથી small unit છે.
- મલ્ટિપ્રોસેસીંગ અને મલ્ટિથ્રેડિંગ, બંનેનો ઉપયોગ મલ્ટિટાસ્કિંગ કરવા માટે થાય છે.
- પરંતુ મલ્ટિપ્રોડ્રિસંગ કરતા મલ્ટિથ્રિડિંગનો ઉપયોગ કરીએ છીએ કારણ કે થ્રેડો common memory area ને share કરે છે. તેઓ અલગ memory area ને allocate કરતા નથી તે મેમરીને બચાવે છે, અને થ્રેડો વચ્ચે context-switching એ process કરતા ઓછો સમય લે છે.
- Threads independent છે. તેથી, જો કોઈ એક થ્રેડમાં exception આવે તો તે અન્ય થ્રેડોને અસર કરતું નથી.
- જાવા મલ્ટિથ્રેડિંગનો ઉપયોગ મોટે ભાગે games, એનિમેશન વગેરેમાં થાય છે.
- ઉદાહરણ તરીકે, એક થ્રેડ ફાઈલ પર content લખી રહ્યો છે તે જ સમયે બીજો થ્રેડ spelling check કરી રહ્યો છે.
- મલ્ટિપ્રોસેસીંગમાં, દરેક process ને મેમરીમાં પોતાનું address હોય છે. તેથી, દરેક process ને અલગ મેમરી area allocates થાય છે.
- એક processમાંથી બીજી process માં સ્વિચ કરવા માટે રજિસ્ટર, મેમરી maps, updating lists વગેરેને કરવા માટે સમય જરૂરી છે તેથી processes વચ્ચેના communication નો cost વધુ છે.
- Disadvantage: જો તમે ઘણા બધા થ્રેડ બનાવો છો, તો તમે ખરેખર તમારા પ્રોગ્રામના performance વધારવાને બદલે degrade કરો છો.
- ઓવરહેડ context switching સાથે સંકળાયેલા છે. જો તમે ઘણા બધા થ્રેડ બનાવો છો, તો તમારા પ્રોગ્રામને ચલાવવા કરતાં વધુ CPU સમય context switchingમાં ખર્ચવામાં આવશે

Life Cycle of Thread

- એક **thread** તેના **life cycle** માં વિવિધ stages માંથી પસાર થાય છે. ઉદાહરણ **thread** એ **born, started, runs, and then dies.**
- **Diagram:**



- **New:**
  - એક નવો થ્રેડ New state થી તેની life cycle ની શરૂઆત કરે છે. જ્યાં સુધી પ્રોગ્રામ Start () methodનો ઉપયોગ કરીને થ્રેડ શરૂ ન કરે ત્યાં સુધી તે આ stateમાં રહે છે. તેને born thread તરીકે પણ ઓળખવામાં આવે છે.
- **Runnable:**
  - start () methodના invocation પછી થ્રેડ runnable stateમાં છે, પરંતુ થ્રેડ શેડ્યૂલેરે તેને running thread તરીકે પસંદ કર્યો નથી.
  - જો thread scheduler એ select કર્યો હોય તો થ્રેડ running stateમાં છે.
- **Waiting:**
  - કેટલીકવાર થ્રેડ waiting state માં transitions કરે છે જ્યારે થ્રેડ કોઈ કાર્ય કરવા માટે બીજા થ્રેડની રાહ જુએ છે. જ્યારે બીજો થ્રેડ signal આપે છે ત્યારે જ થ્રેડ continue executing state માં પાછો ફરો.
- **Timed waiting:**
  - runing થ્રેડ time ના specified interval માટે timed waiting stateમાં જાય છે. જ્યારે તે સમયગાળો સમાપ્ત થાય છે અથવા જ્યારે તે ઈવેન્ટની રાહ જોતા હોય તે આવે છે ત્યારે runnable stateમાં પાછો આવે છે.
- **Terminated:**
  - જ્યારે ચાલી રહેલ થ્રેડ terminated stateમાં પ્રવેશ કરે છે જ્યારે તે ((run() method exits) તેનું કાર્ય પૂર્ણ કરે છે.

Thread Priorities

- દરેક જાવા થ્રેડ ને priority હોય છે જે operating system ને કયા order માં થ્રેડ schedule કરવા તેમાં હેલ્પ કરે છે.
- Priorities એ MIN_PRIORITY (1 ની constant) અને MAX_PRIORITY (10 ની constant) વચ્ચેની રેન્જમાં છે.
- default રૂપે, દરેક થ્રેડને NORM_PRIORITY (5 ની constant) priority આપવામાં આવે છે
- પ્રોગ્રામ માટે higher priority થ્રેડો વધુ important છે અને lower-priority થ્રેડો પહેલાં higher priority થ્રેડો ને પ્રોસેસર સમય allocate કરવો જોઈએ.
- થ્રેડ શેડ્યૂલર મુખ્યત્વે થ્રેડોને શેડ્યૂલ કરવા માટે preemptive અથવા time slicing scheduling નો ઉપયોગ કરે છે.

Creating a Thread

- થ્રેડ ક્લાસ થ્રેડ પર create અને operations માટે કન્સ્ટ્રક્ટર અને methods provide કરે છે.

1) Constructor of Thread class:

- Thread ()
- Thread (String name)
- Thread (Runnable r)

- Thread (Runnable r, String name)

2) Methods of Thread Class:

Method	Description
public void **run**()	થ્રેડ માટે Entry point
public void **start**()	run() મેથડને call કરીને થ્રેડ શરૂ કરી શકો છો
public String **getName**()	થ્રેડનું નામ આપે છે.
public void **setName**(String **name**)	થ્રેડને નામ આપવું.
public int **getPriority**()	થ્રેડની priority આપે છે.
public int **setPriority**(int **priority**)	આ થ્રેડ object ની priority સેટ કરે છે. possible values 1 અને 10 ની વચ્ચે છે.
public final boolean **isAlive**()	થ્રેડ હજી ચાલુ છે કે નહીં તે Checks કરે છે.
public static void **sleep**(long **millisec**)	specified સમય માટે થ્રેડને Suspend કરે છે
public final void **join**(long **millisec**)	થ્રેડ end થાય ત્યાં સુધી રાહ જુઓ.

- જાવા બે રીતે defines કરે છે જેના દ્વારા થ્રેડ બનાવી શકાય છે.
  - By implementing the **Runnable** interface.
  - By extending the **Thread** class.

1) By implementing the Runnable interface:

- થ્રેડ બનાવવાનો સૌથી સહેલો રસ્તો class બનાવવાનો છે જે runnable interface ને implements કરે છે.
- runnable interface ને implementing કર્યા પછી, class ને run () method ને implement કરવાની હોય છે, જે નીચે આપેલ ફોર્મ ધરાવે છે:

public void run( )

- આ method થ્રેડ માટે entry point provides કરે છે અને તમે આ method ની અંદર તમારું business logic ને લખી શકો છો.
- તે પછી, તમે નીચેના કન્સ્ટ્રક્ટરનો ઉપયોગ કરીને થ્રેડ object નો ઈન્સ્ટન્ટ બનાવશો:

Thread (Runnable threadObj**, String** threadName**) ;**

- જ્યાં, threadObj એ class નો instance છે જે Runnable interface ને implements કરે છે અને threadName એ નવા થ્રેડને આપેલ નામ છે.
- એકવાર થ્રેડ object બનાવવામાં આવે, પછી તમે start () method ને call કરીને થ્રેડ start કરી શકો છો, જે run( ) method ને executes કરે છે

void start ( );

Example:

```
class demoThread3 implements Runnable
{
        public void run()
        {
                System.out.println("Thread is running...");
        }
```

```
        public static void main(String args[])
        {
                demoThread3 d1=new demoThread3 ();
                Thread t1 =new Thread(d1);
                t1.start();
        }
}
```

Output:

```
Thread is running...
```

Example:

```
class RunnableDemo implements Runnable
{
        Thread t;
        String threadName;
        RunnableDemo( String name)
        {
                threadName = name;
                System.out.println("Creating " +  threadName );
        }
        public void run()
        {
                System.out.println("Running " +  threadName );
                try
                {
                        for(int i = 2; i >= 0; i--)
                        {
                                System.out.println("Thread: " + threadName + ", " + i);
                                // Let the thread sleep for a while.
                                Thread.sleep(1000);
                        }
                }
                catch (InterruptedException e)
                {
                        System.out.println("Thread " +  threadName + " interrupted.");
                }
                System.out.println("Thread " +  threadName + " exiting.");
        }
        public void start ()
        {
                System.out.println("Starting " +  threadName );
                if (t == null)
                {
                        t = new Thread (this, threadName);
                        t.start ();
                }
        }
```

```
}
public class ThreadDemo
{
        public static void main(String args[])
        {
                RunnableDemo R1 = new RunnableDemo( "Thread-1");
                R1.start();
        }
}
```

Output:

```
Creating Thread-1
Starting Thread-1
Running Thread-1
Thread: Thread-1, 2
Thread: Thread-1, 1
Thread: Thread-1, 0
Thread Thread-1 exiting.
```

2) By extending the Thread class.

- થ્રેડ બનાવવાની બીજી રીત એ છે કે નવો class બનાવવો જે Thread ક્લાસને extends કરે અને પછી તે class નો instance બનાવવો.
- Extending class એ run() method ને ઓવરરાઈડ કરવી જરૂરી છે, જે નવા થ્રેડ માટે entry point છે.
- એકવાર થ્રેડ object બનાવવામાં આવે, પછી તમે start () method ને call કરીને થ્રેડ start કરી શકો છો, જે run( ) method ને executes કરે છે

Example:

```
class demoThread2 extends Thread
{
        public void run()
        {
                System.out.println("Thread is running...");
        }
        public static void main(String args[])
        {
                demoThread2  t1 = new demoThread2();
                t1.start();
        }
}
```

Output:

```
Thread is running...
```

Example:

```
class ThreadDemo extends Thread
{
```

```
Thread t;
String threadName;
ThreadDemo( String name)
{
        threadName = name;
        System.out.println("Creating " +  threadName );
}
public void run()
{
        System.out.println("Running " +  threadName );
        try
        {
                for(int i = 2; i >= 0; i--)
                {
                        System.out.println("Thread: " + threadName + ", " + i);
                        // Let the thread sleep for a while.
                        Thread.sleep(1000);
                }
        }
        catch (InterruptedException e)
        {
                System.out.println("Thread " +  threadName + " interrupted.");
        }
        System.out.println("Thread " +  threadName + " exiting.");
}
public void start ()
{
        System.out.println("Starting " +  threadName );
        if (t == null)
        {
                t = new Thread (this, threadName);
                t.start ();
        }
}
public class ThreadDemo2
{
        public static void main(String args[])
        {
                ThreadDemo T1 = new ThreadDemo( "Thread-1");
                T1.start();
```

```
        }
    }
```

Output:

```
Creating Thread-1
Starting Thread-1
Running Thread-1
Thread: Thread-1, 2
Thread: Thread-1, 1
Thread: Thread-1, 0
Thread Thread-1 exiting.
```

Thread Synchronization (થ્રેડ સિંક્રોનાઈઝેશન)

- જ્યારે બે કે તેથી વધુ થ્રેડોને shared resource ને access કરવાની જરૂર હોય, ત્યારે એક સમયે ફક્ત એક જ થ્રેડ દ્વારા resource નો ઉપયોગ કરવામાં આવે, તે ensure કરવા માટે કેટલીક રીતની જરૂર હોય છે.
- તે process કે જેના દ્વારા આ સિંક્રોનાઈઝેશન કરવામાં આવે છે તેને થ્રેડ સિંક્રોનાઈઝેશન કહે છે
- જાવામાં synchronized કીવર્ડ થી કોડનો એક બ્લોક બનાવે છે, જેને critical section તરીકે ઓળખવામાં આવે છે.
- કોડના critical section માં દરેક જાવા object ને object સાથે સંકળાયેલ lock મળે છે.

Syntax:

```
synchronized(object)
{
        // statements to be synchronized
}
```

- અહીં, **object** એ **synchronized** થઈ રહેલા **object**નો **reference** છે.
- સિંક્રનાઈઝડ **block** એ **ensures** કરે છે કે **current** થ્રેડ સફળતાપૂર્વક **object** ના **critical section**માં **enter** થયા પછી જ **object** ની **method** ને **call** કરી શકે છે.

Example:

```
class PrintDemo
{
        public void printCount()
        {
                try
                {
                        for(int i = 3; i > 0; i--)
                        {
                                System.out.println("Counter   ---   " + i );
                        }
                }
                catch (Exception e)
                {
                        System.out.println("Thread  interrupted.");
                }
        }
}
class ThreadDemo extends Thread
```

```
{
        Thread t;
        String threadName;
        PrintDemo  PD;

        ThreadDemo( String name,  PrintDemo pd)
        {
                threadName = name;
                PD = pd;
        }
        public void run()
        {
                synchronized(PD)
                {
                        PD.printCount();
                }
                System.out.println("Thread " +  threadName + " exiting.");
        }
        public void start ()
        {
                System.out.println("Starting " +  threadName );
                if (t == null)
                {
                        t = new Thread (this, threadName);
                        t.start ();
                }
        }
}
public class ThreadSchro
{
        public static void main(String args[])
        {
                PrintDemo PD = new PrintDemo();
                ThreadDemo T1 = new ThreadDemo( "Thread - 1 ", PD );
                ThreadDemo T2 = new ThreadDemo( "Thread - 2 ", PD );

                T1.start();
                T2.start();
                }
    }
```

Output:

```
Starting Thread - 1
Starting Thread - 2
Counter   ---   3
Counter   ---   2
Counter   ---   1
```

```
Thread Thread - 1  exiting.
Counter   ---   3
Counter   ---   2
Counter   ---   1
Thread Thread - 2  exiting.
```

Inter-thread Communication

- ઈન્ટર-થ્રેડ કમ્યુનિકેશન અથવા Co-operation, સિંક્રનાઈઝ થ્રેડોને એકબીજા સાથે કમ્યુનિકેશન કરવાની allow કરે છે
- ઈન્ટર-થ્રેડ કમ્યુનિકેશન એક મિકેનિઝમ છે જેમાં થ્રેડને તેના critical section માં paused કરવામાં આવે છે અને બીજા થ્રેડને એક્ઝિક્યુટ કરવા માટે same critical sectionમાં (અથવા lock) દાખલ કરવાની મંજૂરી છે.
- avoid polling(It is usually implemented by loop), ઈન્ટર-થ્રેડ communication એ Object class ની નીચેની method ઓં દ્વારા લાગુ કરવામાં આવે છે:
  - **wait( ):** આ method calling થ્રેડને કહે છે કે critical section છોડી દો અને sleep થઈ જાઓ ત્યાં સુધી કેટલાક અન્ય થ્રેડ same critical sectionમાં આવે નહીં અને notify () method call કરે.
  - **notify( ):** આ method પ્રથમ થ્રેડને wakes up કરે છે જેને same object પર wait() કરે છે.
  - **notifyAll( ):** આ method તે બધા થ્રેડો ને wakes up કરે છે જેને same object પર wait () કરે છે. સૌથી વધુ priority હોય તે થ્રેડ પ્રથમ run થાય છે.
- ઉપરની બધી methods final તરીકે object class માં implement કરેલ છે.
- ત્રણેય methods ફક્ત synchronized context માંથી જ call કરી શકાય.

Example:

```
class Customer
{
        int amount=10000;
        synchronized void withdraw(int amount)
        {
                System.out.println("going to withdraw...");

                if(this.amount<amount)
                {
                        System.out.println("Less balance; waiting for deposit...");
                        try
                        {
                                wait();
                        }
                        catch(Exception e){}
                }
                this.amount-=amount;
                System.out.println("withdraw completed...");
        }
        synchronized void deposit(int amount)
        {
                System.out.println("going to deposit...");
                this.amount+=amount;
```

```
                System.out.println("deposit completed... ");
                notify();
            }
}
class InterThreadDemo
{
            public static void main(String args[])
            {
                final Customer c = new Customer();
                new Thread()
                {
                    public void run(){c.withdraw(15000); }
                }.start();
                new Thread()
                {
                    public void run(){c.deposit(10000);}
                }.start();
            }
    }
```

Output:

```
going to withdraw...
Less balance; waiting for deposit...
going to deposit...
deposit completed...
withdraw completed...
```

Streams (સ્ટ્રીમ્સ)

- stream એ ડેટાની sequence છે. જાવામાં stream એ bytesથી બનેલી હોય છે.
- જાવામાં, આપમેળે 3 સ્ટ્રીમ્સ બનાવવામાં આવે છે.
    1. **System.out :** standard output stream
    2. **System.in :** standard input stream
    3. **System.err :** standard error stream

Byte Streams

- જાવા બાઇટ સ્ટ્રીમ્સનો ઉપયોગ બાઈનરી ફાઇલમાંથી એક સમયે ૪-બિટ્સ / 1 બાઇટના ઇનપુટ અને આઉટપુટને કરવા માટે થાય છે.
- InputStream અને OutputStream class એ બાઇટ સ્ટ્રીમનો સૌથી વધુ ઉપયોગમાં લેવાતા class છે.

Byte Streams classes

Stream class	Description
BufferedInputStream	Buffered input stream માટે use થાય છે.
BufferedOutputStream	Buffered output stream માટે use થાય છે.
DataInputStream	જાવા સ્ટાન્ડર્ડ ડેટા type reading કરવા માટેની method હોય છે.
DataOutputStream	જાવા સ્ટાન્ડર્ડ ડેટા type writing કરવા માટેની method હોય છે.
FileInputStream	ઇનપુટ સ્ટ્રીમ જે ફાઇલમાંથી read કરે છે.
FileOutputStream	આઉટપુટ સ્ટ્રીમ જે ફાઇલમાં લખે છે.
InputStream	એબ્સ્ટ્રેક્ટ class કે જે સ્ટ્રીમ ઇનપુટ ને describe કરે છે.
OutputStream	એબ્સ્ટ્રેક્ટ class કે જે સ્ટ્રીમ આઉટપુટ ને describe કરે છે.
PrintStream	આઉટપુટ સ્ટ્રીમ જેમાં print () અને println () method હોય છે.

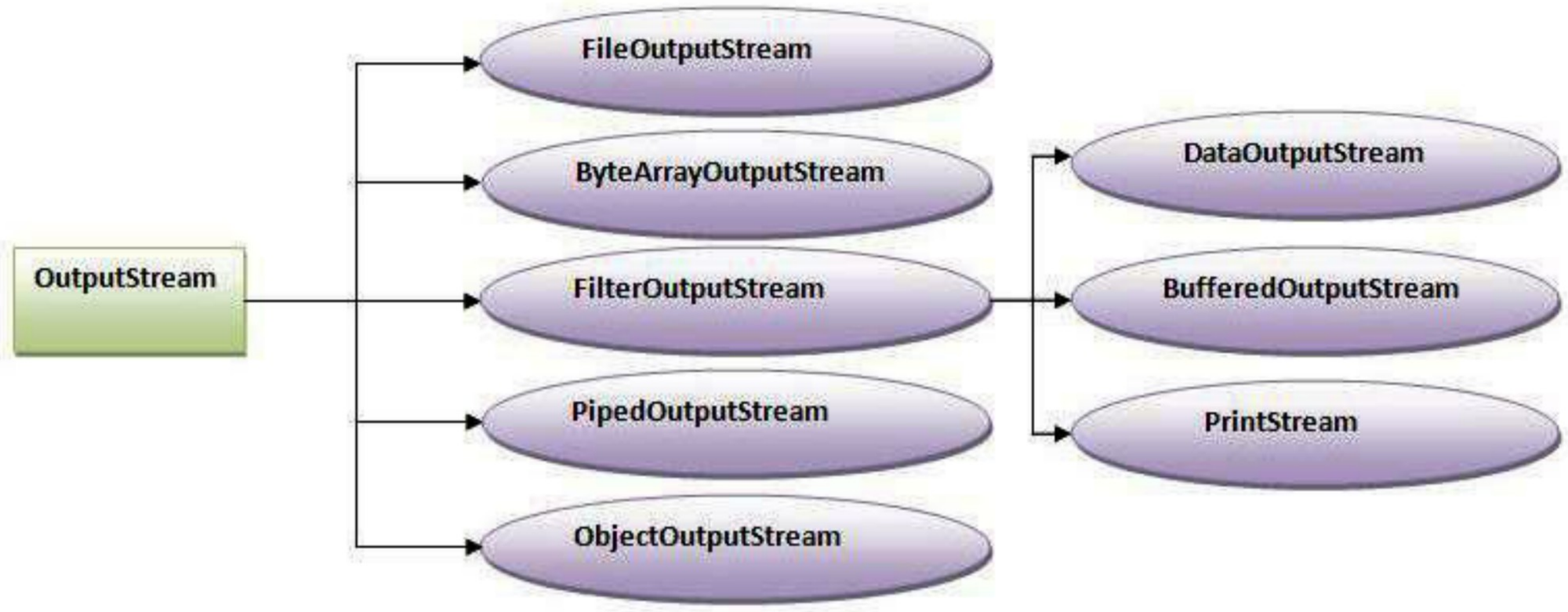
FileOutputStream

- FileOutputStreamનો ઉપયોગ ફાઇલ બનાવવા અને તેમાં ડેટા લખવા માટે થાય છે.
- OutputStream એ file open કરતા પહેલા check કરે છે કે file already exist છે કે નહિ જો નથી તો તે file ને create કરે છે.
- FileOutputStream સાથે ફાઇલનો ઉપયોગ કરવાની બે રીત છે.

1. **FileOutputStream f = new FileOutputStream("C:/java/hello.txt");**

✓ અહીં, FileOutputStream નો object બનાવતી વખતે સીધા ફાઇલ પાથ specified કરવામાં આવે છે.

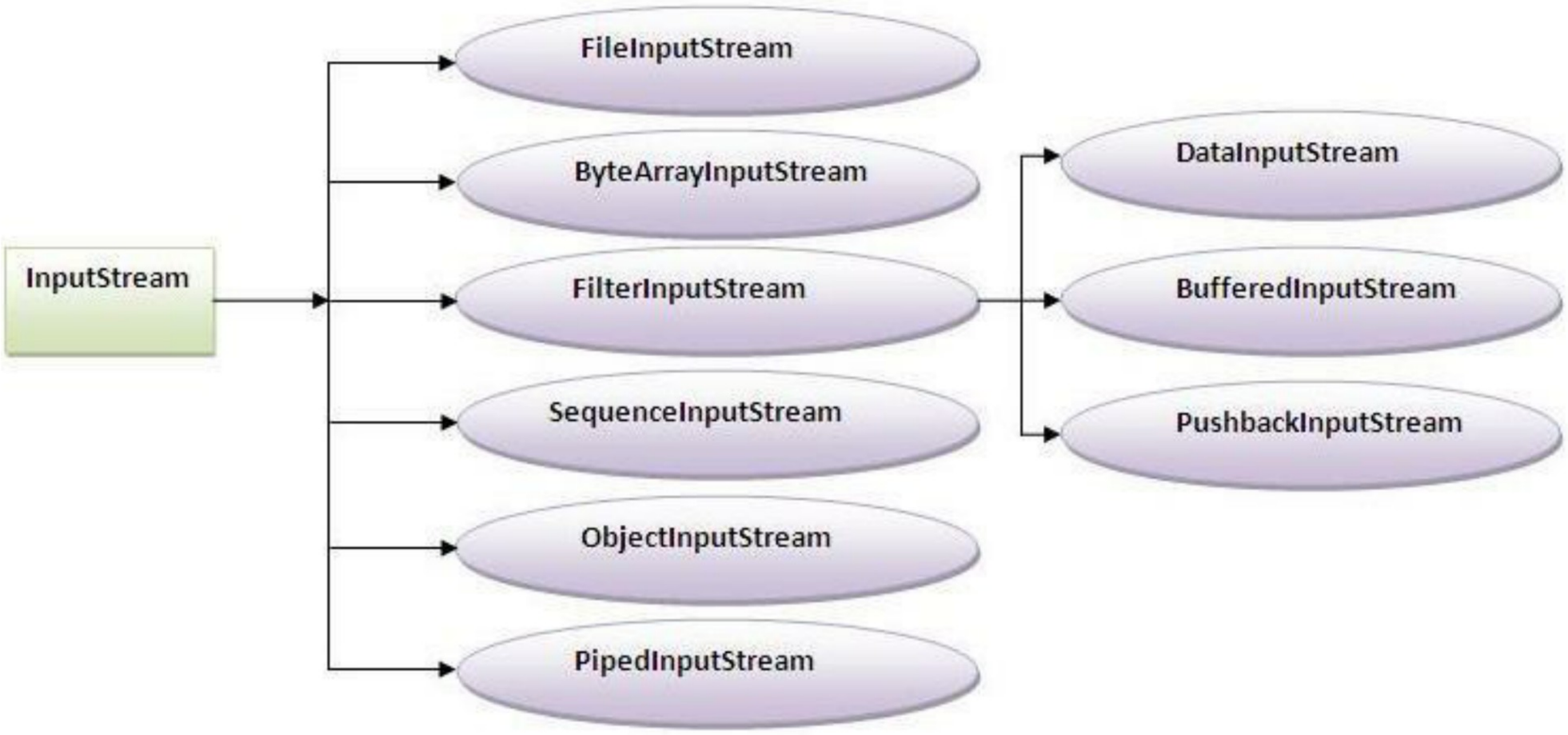
2. **File f = new File("C:/java/hello.txt");**
   FileOutputStream f1 = new FileOutputStream(f);
   ✓ બીજી method, પ્રથમ File નો object બનાવવામાં આવે છે અને પછી ફાઇલ FileOutputStream નો object બનાવવામાં આવે છે અને File નો object ને FileOutputStream પર argument તરીકે pass કરવામાં આવે છે.



FileInputStream

- FileInputStream નો use ફાઇલોમાંથી ડેટા read કરવા માટે થાય છે.
- FileInputStream class નો ઉપયોગ કરતી વખતે, ફાઇલ exist હોવી આવશ્યક છે જો ફાઇલ exist ન હોય તો તે error generate કરશે.
- FileInputStream સાથે ફાઇલનો ઉપયોગ કરવાની બે રીત છે.

1. **FileInputStream f1 = new FileInputStream("C:/java/hello.txt");**
   ✓ અહીં, FileInputStream નો object બનાવતી વખતે સીધા ફાઇલ પાથ specified કરવામાં આવે છે.
2. **File f = new File("C:/java/hello.txt");**
   FileInputStream f1 = new FileInputStream(f);
   ✓ બીજી method, પ્રથમ File નો object બનાવવામાં આવે છે અને પછી ફાઇલ FileInputStream નો object બનાવવામાં આવે છે અને File નો object ને FileInputStream પર argument તરીકે pass કરવામાં આવે છે.



Example : Write a java program that writes the Hello.txt file into Hello1.txt using FileInputStream and FileOutputStream.

Solution: File_1.java

```
import java.io.*;
public  class  File_1
{
        public static void main(String[] args)
        {
                try
                {
                        FileInputStream in = new FileInputStream("C:/temp/Hello.txt ");
                        FileOutputStream out = new FileOutputStream("C:/temp/Hello1.txt ");
                        int c = 0;
                        while ((c = in.read())!=-1)
                        {
                                out.write(c);
                        }
                        in.close();
                        out.close();
                        System.out.println("Successfully Write");
                }
                catch (FileNotFoundException e)
                {
                        e.printStackTrace();
                }
                catch (IOException e)
```

```
                {
                        e.printStackTrace();
                }
            }
        }
```

Output:

Successfully Write

Character Streams

- Character ફાઇલમાંથી એક સમયે 16-બીટ્સ / 2 બાઇટ્સ નું ઇનપુટ અને આઉટપુટ કરવા માટે Character streams નો ઉપયોગ થાય છે.
- Reader અને Writer એ character stream ના સૌથી વધુ ઉપયોગમાં લેવાતા class છે.

Character Stream classes

Stream class	Description
BufferedReader	Handle buffered input ને હેન્ડલ કરે છે.
BufferedWriter	Handle buffered output ને હેન્ડલ કરે છે.
FileReader	Input stream જે ફાઇલમાંથી read કરે છે.
FileWriter	Output stream જે ફાઇલમાંથી read કરે છે.
InputStreamReader	Input stream જે byte characterમાં translate કરે છે.
OutputStreamReader	Output stream જે byte characterમાં translate કરે છે.
Reader	Abstract class કે જે character stream input ને define કરે છે.
Writer	Abstract class કે જે character stream output ને define કરે છે.

FileWriter

- જાવા માં FileWriter ક્લાસનો ઉપયોગ ફાઇલમાં કેરેક્ટર ડેટા ને write કરવા માટે થાય છે.
- FileWriter ના Object ને બનાવવાની બે રીત.
  1. FileWriter f = new FileWriter(String fname); **//File path is specified in argument.**
  2. FileWriter f = new FileWriter(File f1); **//1st file is created and it is passed as argument.**

Method of FileWriter Class

void write(int ch)	ફાઇલ પર single character ને write કરે છે.
void write(String s)	FileWriter માં string ને લખે છે.

void write(char[] c)	FileWriter માં char array ને લખે છે.
void close()	FileWriter ને બંધ કરે છે.

FileReader

- જાવા માં FileReader ક્લાસનો ઉપયોગ ફાઇલમાં કેરેક્ટર ડેટા ને read કરવા માટે કરવામાં આવે છે.
- FileReader ના Object ને બનાવવાની બે રીત.

1. FileReader f = new FileReader(String name); **//File path is specified in argument.**
2. FileReader f = new FileReader(File f1); **//1st file is created and it is passed as argument.**

Method of FileReader Class

int read()	ASCII formમાં character Return કરે છે. તે ફાઈલના અંતે -1 Return કરે છે.
int read(char[] ch)	ફાઇલમાંથી ડેટાને char array માં read કરવા માટે.
void close()	FileReader ને બંધ કરે છે.

Example :Write a java program that writes the Hello.txt file into Hello1.txt using FileReader and FileWriter.

Solution: File_2.java

```
import java.io.*;
public class File_2
{
	public static void main(String[] args) throws IOException
	{
		File f = new File("d:/temp/Hello.txt");
		File f1 = new File("d:/temp/Hello1.txt");
		FileReader fr = new FileReader(f);
		FileWriter fw = new FileWriter(f1);
		int c = 0;
		while ((c = fr.read())!=-1)
		{
			fw.write(c);
		}
		fr.close();
		fw.close();
		System.out.println("Successfully Write");
	}
}
```

Output:

Successfully Write

BufferedReader

- BufferedReader class નો ઉપયોગ readLine () methodનો ઉપયોગ કરીને line by line ડેટા read કરવા માટે થાય છે.

BufferedOutputStream

- BufferedOutputStream class ડેટા સ્ટોર કરવા માટે internal buffer નો ઉપયોગ કરે છે.
- તે સીધા જ streamમાં ડેટા લખવા કરતાં વધુ efficiency add કરે છે. તેથી, તે performance ને fast બનાવે છે.

BufferedInputStream

- BufferedInputStream class સ્ટ્રીમમાંથી information ને read કરવા માટે વપરાય છે. તે performance ને fast બનાવવા માટે internally buffer mechanismનો ઉપયોગ કરે છે.

Example : Write a program in java that take character as a input from console and write it in file develop this program using Bufered Classes.

Solution: File_3.java

```
import java.io.*;
public class File_3
{
        public static void main(String[] args)
        {
                InputStreamReader in = new InputStreamReader(System.in);
                BufferedReader br = new BufferedReader(in);
                File f = new File("d:/temp/Hello.txt");
try
{
                OutputStream out = new FileOutputStream(f);
                System.out.print("Type your text to write in file===");
                String str;
                str = br.readLine();
                byte b[] = str.getBytes();
                out.write(b);
                System.out.println("File Successfully Write..");
                br.close();
                in.close();
                out.close();
}
```

```
                    catch (IOException e)
                    {
                            e.printStackTrace();
                    }
            }
    }
```

Output:

Type your text to write in file===i like an ice cream
File Successfully Write..

Enrollment No./Seat No.:

GUJARAT TECHNOLOGICAL UNIVERSITY

DIPLOMA IN ENGINEERING - SEMESTER - III EXAMINATION - SUMMER 2026

Subject Code: DI03032021 **Date: 20-05-2026**

Subject Name: Java Programming

Time: 02:30 PM TO 05:00 PM **Total Marks: 70**

Instructions:

1. **Attempt all questions.**
2. **Make Suitable assumptions wherever necessary.**
3. **Figures to the right indicate full marks.**
4. **Use of simple calculators and non-programmable scientific calculators are permitted.**
5. **English version is authentic.**

			Marks
Q.1	**(a)**	List out features of JAVA and explain any two of them.	**03**
	(અ)	JAVA ની વિશેષતાઓની યાદી બનાવો અને તેમાંથી કોઈપણ બે સમજાવો.	૦૩
	(b)	Explain JDK, JRE, JIT and JVM.	**04**
	(બ)	JDK, JRE, JIT અને JVM સમજાવો.	૦૪
	(c)	Write syntax of Java while loop statement. Develop a Java program to print first 50 even numbers.	**07**
	(ક)	Java while loop સ્ટેટમેન્ટનું વાક્યરચના લખો. પહેલા 50 બેકી સંખ્યાઓ છાપવા માટે જાવા પ્રોગ્રામ વિકસાવો.	૦૭
		OR	
	(c)	List relational operators in Java. Develop a Java program using any three relational operators and show the output of program.	**07**
	(ક)	Java માં રિલેશનલ ઓપરેટરોની યાદી બનાવો. કોઈપણ ત્રણ રિલેશનલ ઓપરેટરોનો ઉપયોગ કરીને જાવા પ્રોગ્રામ વિકસાવો અને પ્રોગ્રામનું આઉટપુટ બતાવો.	૦૭
Q.2	**(a)**	Explain this in Java with example.	**03**
	(અ)	Java માં this ઉદાહરણ સાથે સમજાવો.	૦૩
	(b)	Explain equals(), toString() and finalize() methods with its usage in JAVA.	**04**
	(બ)	JAVA માં તેના ઉપયોગ સાથે equals(), toString() અને finalize() પદ્ધતિઓ સમજાવો.	૦૪
	(c)	Define Constructor. List types of Constructors. Develop a java code to explain Default constructor.	**07**
	(ક)	કન્સ્ટ્રક્ટર વ્યાખ્યાયિત કરો. કન્સ્ટ્રક્ટરના પ્રકારોની યાદી બનાવો. ડિફોલ્ટ કન્સ્ટ્રક્ટર સમજાવવા માટે જાવા કોડ વિકસાવો.	૦૭
		OR	
	(a)	Explain final keyword with example.	**03**
	(અ)	ઉદાહરણ સાથે Final કીવર્ડ સમજાવો.	૦૩

	(b)	Explain static in java with example.	**04**
	(બ)	Java માં static ને ઉદાહરણ સાથે સમજાવો.	**૦૪**
	(c)	Write a program in Java which has a class Shape having 2 overloaded methods area(float radius) and area(float length, float width). Display the area of circle and rectangle using overloaded methods.	**07**
	(ક)	Java માં એક પ્રોગ્રામ લખો જેમાં Shape ક્લાસ હોય જેમાં 2 ઓવરલોડેડ મેથડ્સ area(float radius) અને area(float length, float width) હોય. ઓવરલોડેડ મેથડ્સનો ઉપયોગ કરીને વર્તુળ અને લંબચોરસનું ક્ષેત્રફળ દર્શાવો.	**૦૭**
Q.3	**(a)**	Define package in JAVA. Write a Java program to create package.	**03**
	(અ)	JAVA માં પેકેજ વ્યાખ્યાયિત કરો. પેકેજ બનાવવા માટે Java પ્રોગ્રામ લખો.	**૦૩**
	(b)	Explain Abstract class in Java with suitable example.	**04**
	(બ)	Java માં એબ્સ્ટ્રેક્ટ ક્લાસને યોગ્ય ઉદાહરણ સાથે સમજાવો.	**૦૪**
	(c)	Explain Inheritance and its types in Java with suitable diagrams.	**07**
	(ક)	યોગ્ય આકૃતિઓ સાથે Java માં Inheritance અને તેના પ્રકારો સમજાવો.	**૦૭**
		OR	
	(a)	Write a Java program to demonstrate single inheritance.	**03**
	(અ)	સિંગલ inheritance દર્શાવવા માટે Java પ્રોગ્રામ લખો.	**૦૩**
	(b)	Explain Final class in Java with suitable example.	**04**
	(બ)	યોગ્ય ઉદાહરણ સાથે Java માં ફાઇનલ ક્લાસ સમજાવો.	**૦૪**
	(c)	Define Interface in Java. Write a Java program to demonstrate Multiple inheritance using interfaces in Java.	**07**
	(ક)	Java માં ઇન્ટરફેસ વ્યાખ્યાયિત કરો. Java માં ઇન્ટરફેસનો ઉપયોગ કરીને બહુવિધ inheritance દર્શાવવા માટે Java પ્રોગ્રામ લખો.	**૦૭**
Q.4	**(a)**	Explain try catch block with example in Java.	**03**
	(અ)	Java માં ઉદાહરણ સાથે ટ્રાય કેચ બ્લોક સમજાવો.	**૦૩**
	(b)	List any four inbuilt exceptions in Java.	**04**
	(બ)	Java માં કોઈપણ ચાર ઇનબિલ્ટ અપવાદોની યાદી બનાવો.	**૦૪**
	(c)	What is Exception? Write a program that shows the use of Arithmetic Exception.	**07**
	(ક)	Exception શું છે? એક પ્રોગ્રામ લખો જે અંકગણિત અપવાદનો ઉપયોગ બતાવે છે.	**૦૭**
		OR	
	(a)	List and explain types of errors in Java.	**03**
	(અ)	Java માં ભૂલોના પ્રકારોની યાદી બનાવો અને સમજાવો.	**૦૩**
	(b)	What is thread ? Explain thread life cycle.	**04**
	(બ)	Thread શું છે? Thread ના જીવન ચક્રને સમજાવો.	**૦૪**
	(c)	Write a program in Java to develop user defined exception for 'Divide by Zero' error.	**07**

	(ક)	'Divide by Zero' એરર માટે યુઝર ડિફાઈન્ડ અપવાદ વિકસાવવા માટે જાવામાં એક પ્રોગ્રામ લખો.	**૦૭**
Q.5	**(a)**	List Different Stream Classes in Java.	**03**
	(અ)	Java માં વિવિધ સ્ટ્રીમ વર્ગોની યાદી બનાવો.	**૦૩**
	(b)	Write a Java program to create and display ArrayList.	**04**
	(બ)	ArrayList બનાવવા અને પ્રદર્શિત કરવા માટે Java પ્રોગ્રામ લખો.	**૦૪**
	(c)	Write a java program to create a text file and perform read operation on the text file.	**07**
	(ક)	ટેક્સ્ટ ફાઇલ બનાવવા માટે જાવા પ્રોગ્રામ લખો અને ટેક્સ્ટ ફાઇલ પર રીડ ઓપરેશન કરો.	**૦૭**

OR

	(a)	List different file operations in Java.	**03**
	(અ)	Java માં વિવિધ ફાઇલ ઓપરેશન્સની યાદી બનાવો.	**૦૩**
	(b)	Write a Java program to create and display LinkedList.	**04**
	(બ)	LinkedList બનાવવા અને પ્રદર્શિત કરવા માટે Java પ્રોગ્રામ લખો.	**૦૪**
	(c)	Write a java program to create a file and perform write operation on this file.	**07**
	(ક)	ફાઇલ બનાવવા માટે Java પ્રોગ્રામ લખો અને આ ફાઇલ પર લખવાનું કાર્ય કરો.	**૦૭**

***

Enrollment No./Seat No.:

GUJARAT TECHNOLOGICAL UNIVERSITY

DIPLOMA IN ENGINEERING - SEMESTER - III EXAMINATION - WINTER 2025

Subject Code: DI03032021 **Date: 10-12-2025**

Subject Name: Java Programming

Time: 10:30 AM TO 01:00 PM **Total Marks: 70**

Instructions:

1. **Attempt all questions.**
2. **Make Suitable assumptions wherever necessary.**
3. **Figures to the right indicate full marks.**
4. **Use of simple calculators and non-programmable scientific calculators are permitted.**
5. **English version is authentic.**

			Marks
Q.1	**(a)**	List all Java features and explain any two of them in details.	**03**
	(અ)	JAVA ના બધા ફીચર્સની યાદી બનાવો અને તેમાંથી કોઈપણ બે ફીચર્સ વિગતવાર સમજાવો.	૦૩
	(b)	Explain the role of the compiler and interpreter in Java program execution.	**04**
	(બ)	જાવા પ્રોગ્રામ એક્ઝેક્યુશનમાં કમ્પાઇલર અને ઇન્ટરપ્રીટરની ભૂમિકા સમજાવો.	૦૪
	(c)	Illustrate different categories of operators in Java with code examples and explain how operator precedence affects output.	**07**
	(ક)	ઉદાહરણો સાથે જાવામાં ઓપરેટર્સની વિવિધ શ્રેણીઓ સમજાવો અને ઓપરેટર પ્રાયોરિટી આઉટપુટને કેવી રીતે અસર કરે છે તે સમજાવો.	૦૭
		OR	
	(c)	Write a Java program to demonstrate all types of type casting (implicit and explicit). Explain each conversion with output.	**07**
	(ક)	બધા પ્રકારના ટાઇપ કાસ્ટિંગ (ઇમ્પ્લીસીટ અને એક્સ્પ્લીસીટ) દર્શાવવા માટે જાવા પ્રોગ્રામ લખો. દરેક રૂપાંતરણને આઉટપુટ સાથે સમજાવો.	૦૭
Q.2	**(a)**	Define access modifiers. List the four types used in Java.	**03**
	(અ)	એક્સેસ મોડિફાયર વ્યાખ્યાયિત કરો. જાવામાં વપરાતા ચાર પ્રકારોની યાદી બનાવો.	૦૩
	(b)	Explain method overloading in JAVA with example.	**04**
	(બ)	JAVA માં METHOD ઓવરલોડિંગ પદ્ધતિને ઉદાહરણ સાથે સમજાવો.	૦૪
	(c)	Create a Java class BankAccount with private fields (accountNo, balance). Implement methods deposit(), withdraw(), and displayBalance().	**07**
	(ક)	Private Field એકાઉન્ટ નંબર, બેલેન્સ સાથે જાવા ક્લાસ બેંક એકાઉન્ટ બનાવો. ડિપોઝિટ(), વિથડ્રો(), અને ડિસ્પ્લેબેલેન્સ() પદ્ધતિઓ લાગુ કરો.	૦૭
		OR	
	(a)	Explain the role of the final keyword in variables and methods.	**03**
	(અ)	વેરીએબલ અને મેથડમાં ફાયનલ કીવર્ડની ભૂમિકા સમજાવો.	૦૩

	(b)	Write a short note on constructor overloading with an example program.	**04**
	(બ)	પ્રોગ્રામનો ઉપયોગ કરીને કન્સ્ટ્રક્ટર ઓવરલોડિંગ પર એક ટૂંકી નોંધ લખો.	૦૪
	(c)	Explain concept of static word and design a Java program that uses static blocks, static variables, and static methods.	**07**
	(ક)	સ્ટેટિક શબ્દનો ખ્યાલ સમજાવો અને એક જાવા પ્રોગ્રામ ડિઝાઇન કરો જે સ્ટેટિક બ્લોક્સ, સ્ટેટિક વેરીએબલ્સ અને સ્ટેટિક પદ્ધતિઓનો ઉપયોગ કરે છે.	૦૭
Q.3	**(a)**	Write a program to demonstrate single and multilevel inheritance using the extends keyword.	**03**
	(અ)	extends કીવર્ડનો ઉપયોગ કરીને સિંગલ અને મલ્ટિલેવલ ઇનહેરિટન્સ દર્શાવવા માટે એક પ્રોગ્રામ લખો.	૦૩
	(b)	Explain method overriding with Java example.	**04**
	(બ)	જાવાના ઉદાહરણ સાથે મેથડ ઓવરરાઇડિંગ સમજાવો.	૦૪
	(c)	Explain the concept of inheritance hierarchy with an example of Person → Employee → Manager classes. Write a Java program showing single, multilevel, and hierarchical inheritance.	**07**
	(ક)	વ્યક્તિ → કર્મચારી → મેનેજર વર્ગોના ઉદાહરણ સાથે inheritance ખ્યાલ સમજાવો. સિંગલ, મલ્ટિલેવલ અને હાયરાર્કિકલ inheritance દર્શાવતો જાવા પ્રોગ્રામ લખો.	૦૭
		OR	
	(a)	Define an interface and mention how it differs from a class.	**03**
	(અ)	ઇન્ટરફેસને વ્યાખ્યાયિત કરો અને તે ક્લાસથી કેવી રીતે અલગ પડે છે તેનો ઉલ્લેખ કરો.	૦૩
	(b)	Explain use of packages in java with java programme.	**04**
	(બ)	જાવા પ્રોગ્રામ સાથે જાવામાં પેકેજોનો ઉપયોગ સમજાવો.	૦૪
	(c)	Discuss in detail the Object class methods (toString(), equals(), hashCode()) and show their overriding in a Java class Student.	**07**
	(ક)	ઓબ્જેક્ટ ક્લાસ પદ્ધતિઓ (toString(), equals(), hashCode()) ની વિગતવાર ચર્ચા કરો અને જાવા ક્લાસ Student માં તેમનું ઓવરરાઇડિંગ બતાવો.	૦૭
Q.4	**(a)**	Explain the purpose of the finally block in Java.	**03**
	(અ)	જાવામાં finally બ્લોકનો હેતુ સમજાવો.	૦૩
	(b)	Write a program to demonstrate multiple catch blocks for different exception types.	**04**
	(બ)	વિવિધ એક્સેપ્સન માટે મલ્ટીપલ કેચ બ્લોક્સ દર્શાવવા માટે એક પ્રોગ્રામ લખો.	૦૪
	(c)	Write a Java program that implements Runnable interface to perform two concurrent tasks (e.g., printing odd and even numbers).	**07**
	(ક)	એક જાવા પ્રોગ્રામ લખો જે બે Concurrent કાર્યો (દા.ત., એકી અને સમ સંખ્યાઓ છાપવા) કરવા માટે રનેબલ ઇન્ટરફેસનો અમલ કરે.	૦૭
		OR	
	(a)	Define multi-threading and mention two of its advantages.	**03**

	(અ)	મલ્ટિથ્રેડીંગને વ્યાખ્યાયિત કરો અને તેના બે ફાયદાઓનો ઉલ્લેખ કરો.	૦૩
	(b)	Write a Java program to demonstrate creation of threads by extending Thread class.	04
	(બ)	થ્રેડ ક્લાસને વિસ્તૃત કરીને થ્રેડો બનાવવાનું દર્શાવવા માટે જાવા પ્રોગ્રામ લખો.	૦૪
	(c)	Explain the concept of multi-threading with a program that creates two threads printing numbers and alphabets simultaneously.	07
	(ક)	એક પ્રોગ્રામ દ્વારા મલ્ટિથ્રેડીંગનો ખ્યાલ સમજાવો જે બે થ્રેડ બનાવે છે અને એકસાથે સંખ્યાઓ અને મૂળાક્ષરો છાપે છે.	૦૭
Q.5	(a)	List any three classes in the Collections Framework and their use.	03
	(અ)	કલેક્શન ફ્રેમવર્કમાં કોઈપણ ત્રણ વર્ગો અને તેમના ઉપયોગની યાદી બનાવો.	૦૩
	(b)	Differentiate between HashMap and TreeMap with suitable examples.	04
	(બ)	યોગ્ય ઉદાહરણો સાથે હેશમેપ અને ટ્રીમેપ વચ્ચે તફાવત દર્શાવો.	૦૪
	(c)	Develop a program using ArrayList and LinkedList to store and display a list of student names. Compare both lists in terms of performance and usage.	07
	(ક)	વિદ્યાર્થીઓના નામોની યાદી સંગ્રહિત કરવા અને પ્રદર્શિત કરવા માટે ArrayList અને LinkedList નો ઉપયોગ કરીને એક પ્રોગ્રામ વિકસાવો. કામગીરી અને ઉપયોગની દ્રષ્ટિએ બંને યાદીઓની તુલના કરો.	૦૭
		OR	
	(a)	Define serialization and state its purpose.	03
	(અ)	Serialization વ્યાખ્યાયિત કરો અને તેનો હેતુ જણાવો.	૦૩
	(b)	Write a Java program to read and write data from/to a text file using character streams.	04
	(બ)	કેરેક્ટર સ્ટ્રીમ્સનો ઉપયોગ કરીને ટેક્સ્ટ ફાઇલમાંથી/માં ડેટા વાંચવા અને લખવા માટે જાવા પ્રોગ્રામ લખો.	૦૪
	(c)	Write a program to demonstrate serialization and deserialization of a Student class object using ObjectOutputStream and ObjectInputStream.	07
	(ક)	ObjectOutputStream અને ObjectInputStream નો ઉપયોગ કરીને Student ક્લાસ ઑબ્જેક્ટના સીરીયલાઇઝેશન અને ડિસેરિયલાઇઝેશન દર્શાવવા માટે એક પ્રોગ્રામ લખો.	૦૭

***

Seat No. / Enrolment No.:

GUJARAT TECHNOLOGICAL UNIVERSITY

Diploma Engineering – SEMESTER – 4 (NEW) – EXAMINATION – Summer-2025

Subject Code: 4343203 **Date: 13-05-2025**

Subject Name: Java Programming

Time: 10:30 AM TO 01:00 PM **Total Marks: 70**

Instructions:

1. **Attempt all questions.**
2. **Make Suitable assumptions wherever necessary.**
3. **Figures to the right indicate full marks.**
4. **Use of simple calculators and non-programmable scientific calculators are permitted.**
5. **English version is authentic.**

			Marks
Q.1	**(a)**	List out the rules to name an identifier in Java with valid and invalid examples.	**03**
પ્રશ્ન.1	(અ)	JAVA મા આઇડેન્ટીફાયરને નામ આપવાના નિયમોની યાદી માન્ય અને અમાન્ય ઉદાહરણો સાથે બનાવો.	૦૩
	(b)	List out different types of operators in Java. Explain Arithmetic and Logical Operators in detail.	**04**
	(બ)	JAVA ના ઓપરેટર્સની યાદી બનાવો. Arithmetic અને Logical ઓપરેટર્સને સમજાવો.	૦૪
	(c)	Write a program in Java to reverse the digits of a number for number having three digits. Like reverse of 653 is 356.	**07**
	(ક)	3 આંકડાની સંખ્યાને વિપરીત કરવા માટે JAVA પ્રોગ્રામ લખો. ઉદાહરણ તરીકે, સંખ્યા 653 છે તો તેની વિપરીત સંખ્યા 356 છે.	૦૭
		OR	
	(c)	Write a program in Java to add two 3*3 matrices.	**07**
	(ક)	3*3 મેટ્રિક્સનો સરવાળો કરવા માટેનો JAVA પ્રોગ્રામ લખો.	૦૭
Q.2	**(a)**	Write a program in Java that shows the use of parameterized Constructor.	**03**
પ્રશ્ન.2	**(અ)**	પેરામીટરાઇઝ્ડ ક્ન્સ્ટ્રકટર નો ઉપયોગ દર્શાવતો JAVA પ્રોગ્રામ લખો.	**૦3**
	(b)	Give the basic syntax of the following terms with an example: (1) To create a Class, (2) To create an Object, (3) To define a Method, (4) To declare a Variable.	**04**
	(બ)	ઉદાહરણ સાથે નીચેના શબ્દોની બેસીક સીન્ટેક્સ આપો: (1) ક્લાસ બનાવવા માટે, (2) ઑબ્જેકટ બનાવવા માટે, (3) મેથડને વ્યાખ્યાયિત કરવા માટે, (4) ચલ(વેરીએબલ)ને ડીકલેર કરવા માટે.	૦૪
	(c)	Write a program in Java which has a class Student having two instance variables enrollmentNo and name. Create 3 objects of Student class in main method and display student's name.	**07**
	(ક)	JAVA માં એક પ્રોગ્રામ લખો જેમાં Student નામનો Class છે. જેમાં enrollmentNo અને name નામ ના બે ઇન્સટ્ન્ટ વેરીએબલ હોય. Student Class ના 3 ઓબ્જેકટ main મેથડમાં બનાવો અને Student's ના નામ દર્શાવો.	૦૭
		OR	

Q.2	**(a)**	Write a program in Java that shows the use of Default Constructor.	**03**
પ્રશ્ન.**2**	(અ)	ડિફોલ્ટ કન્સ્ટ્રકટર નો ઉપયોગ દર્શાવતો JAVA પ્રોગ્રામ લખો.	૦૩
	(b)	Give four Difference between Procedure Oriented Programming and Object-Oriented Programming.	**04**
	(બ)	પ્રોસિજર-ઓરિએન્ટેડ પ્રોગ્રામિંગ અને ઑબ્જેક્ટ-ઓરિએન્ટેડ પ્રોગ્રામિંગ વચ્ચે ચાર તફાવત આપો.	૦૪
	(c)	Write a program in Java which has a class Shape having 2 overloaded methods area (float radius) and area (float length, float width). Display the area of circle and rectangle using overloaded methods.	**07**
	(ક)	JAVA માં એક પ્રોગ્રામ લખો જેમાં Shape નામનો Class હોય જે 2 ઓવરલોડ મેથડ area (ફ્લોટ radius) અને area (ફ્લોટ length, ફ્લોટ width) ધરાવતો હોય. ઓવરલોડ પદ્ધતિઓનો ઉપયોગ કરીને વર્તુળ અને લંબચોરસનો area (વિસ્તાર) દર્શાવો.	૦૭
Q. 3	**(a)**	Write a program in Java to demonstrate single inheritance.	**03**
પ્રશ્ન.**3**	(અ)	સિંગલ ઇનહેરીટેન્સ દર્શાવવા માટે JAVA પ્રોગ્રામ લખો.	૦૩
	(b)	Define abstract class in JAVA with example.	**04**
	(બ)	JAVA માં એબ્સ્ટેક્ટ ક્લાસને ઉદાહરણ સાથે વ્યાખ્યાયિત કરો.	૦૪
	(c)	Write a program in Java to implement multiple inheritance using interfaces.	**07**
	(ક)	ઇન્ટરફેસનો ઉપયોગ કરીને મલ્ટીપલ ઇનહેરીટન્સને ઇમપ્લિમેંટ કરવા માટે JAVAમાં પ્રોગ્રામ લખો.	૦૭
		OR	
Q. 3	**(a)**	Write a program in Java to demonstrate multilevel inheritance.	**03**
પ્રશ્ન.**3**	(અ)	મલ્ટીલેવલ ઇનહેરીટેન્સ દર્શાવવા માટે JAVA પ્રોગ્રામ લખો.	૦૩
	(b)	Define package and write the syntax to create a package with example.	**04**
	(બ)	પેકેજ વ્યાખ્યાયિત કરો અને ઉદાહરણ સાથે પેકેજ બનાવવા માટે સીન્ટેક્સ લખો.	૦૪
	(c)	Write a program in Java to demonstrate method overriding.	**07**
	(ક)	મેથડ ઓવરરાઇડિંગ દર્શાવવા માટે JAVA પ્રોગ્રામ લખો.	૦૭
Q. 4	**(a)**	List and explain different types of errors in Java.	**03**
પ્રશ્ન.4	(અ)	JAVAમાં વિવિધ પ્રકારની એરરની યાદી બનાવો અને સમજાવો.	૦૩
	(b)	What is wrapper class? Explain use of any two wrapper class.	**04**
	(બ)	રેપર ક્લાસ શું છે? કોઈપણ બે રેપર ક્લાસનો ઉપયોગ સમજાવો.	૦૪
	(c)	Write a program in Java to develop Banking Application in which user deposits the amount Rs 25000 and then start withdrawing of Rs 20000, Rs 4000 and it throws exception "Not Sufficient Fund" when user withdraws Rs. 2000 thereafter.	**07**
	(ક)	બેંકિંગ એપ્લિકેશન વિકસાવવા માટે JAVA માં એક પ્રોગ્રામ લખો જેમાં વપરાશકર્તા 25000 રૂપિયા જમા કરે અને પછી 20000, 4000 રૂપિયા ઉપાડવાનું શરૂ કરે અને ત્યારબાદ જ્યારે વપરાશકર્તા રૂ. 2000 ઉપાડે તો પ્રોગ્રામ "પુરતુ ભંડોળ નથી" એક્સેપશન(exception) આપે.	૦૭
		OR	
Q. 4	**(a)**	Describe the complete lifecycle of a thread.	**03**
પ્રશ્ન.4	(અ)	થ્રેડના સંપૂર્ણ જીવનચક્રનું વર્ણન કરો.	૦૩
	(b)	List access specifiers and describe their purpose in JAVA.	**04**

	(બ)	એક્સેસ સ્પેસીફાયરની યાદી બનાવો અને JAVA માં તેમના હેતુનું વર્ણન કરો.	૦૪
	(c)	Write a program that executes two threads. One thread displays "Thread1" every 1000 milliseconds, and the other displays "Thread2" every 2000 milliseconds. Create the threads by extending the **Thread** class.	**07**
	(ક)	JAVA પ્રોગ્રામ લખો જે 2 થ્રેડો ચલાવે છે દર થ્રેડ એક .1000 મીલીસેકન્ડે "થ્રેડ –I" પ્રિન્ટ કરે છે, બીજો થ્રેડ દર 2000 મીલીસેકન્ડે – થ્રેડ"II" પ્રિન્ટ કરે છે. થ્રેડ Classને એક્સ્ટેન્ડ કરીને થ્રેડ બનાવો.	૦૭
Q.5	**(a)**	What is stream class? How are the stream classes classified?	**03**
પ્રશ્ન.5	(અ)	સ્ટ્રીમ ક્લાસ શું છે? સ્ટ્રીમ ક્લાસ કેવી રીતે વર્ગીકૃત કરવામાં આવે છે?	૦૩
	(b)	Illustrate purpose of method overriding with example.	**04**
	(બ)	ઉદાહરણ સાથે મેથડ ઓવરરાઇડિંગનો હેતુ સમજાવો.	૦૪
	(c)	Write a program in Java to perform read and write operations on a Text file named Abc.txt.	**07**
	(ક)	Abc.txt નામની ટેક્સ્ટ ફાઇલ પર વાંચવા અને લખવાની કામગીરી કરવા માટે નો JAVA પ્રોગ્રામ લખો.	૦૭
		OR	
Q.5	**(a)**	Explain InputStream.	**03**
પ્રશ્ન.5	(અ)	ઇન્પુટ સ્ટ્રીમ સમજાવો.	૦૩
	(b)	Define package in JAVA. Write how package can be implemented in Java with proper syntax and one example.	**04**
	(બ)	JAVA માં પેકેજ વ્યાખ્યાયિત કરો. યોગ્ય સીન્ટેક્સ અને એક ઉદાહરણ સાથે JAVA માં પેકેજ કેવી રીતે ઇમપ્લીમેન્ટ કરી શકાય તે લખો.	૦૪
	(c)	Write a program in Java to demonstrate use of List. 1) Create ArrayList and add weekdays (in string form), 2) Create LinkedList and add months (in string form). Display both List.	**07**
	(ક)	List નો ઉપયોગ દર્શાવવા માટે JAVA મા પ્રોગ્રામ લખો .1) ArrayList બનાવો અને અઠવાડિયાના દિવસો ઉમેરો (સ્ટ્રીંગ સ્વરૂપમા) 2) LinkedList બનાવો અને મહિના ઉમેરો)સ્ટ્રીંગ સ્વરૂપમા (બંને List ડિસ્પ્લે કરો.	૦૭

Seat No. / Enrolment No.:

GUJARAT TECHNOLOGICAL UNIVERSITY

Diploma Engineering – SEMESTER – 4 (NEW) – EXAMINATION – Winter-2025

Subject Code: 4343203 **Date: 06-12-2025**

Subject Name: Java Programming

Time: 02:30 PM TO 05:00 PM **Total Marks: 70**

Instructions:

1. **Attempt all questions.**
2. **Make Suitable assumptions wherever necessary.**
3. **Figures to the right indicate full marks.**
4. **Use of simple calculators and non-programmable scientific calculators are permitted.**
5. **English version is authentic.**

			Marks
Q.1	**(a)**	Differentiate between Procedure-Oriented Programming (POP) and Object-Oriented Programming (OOP).	**03**
પ્રશ્ન.1	(અ)	પ્રોસિજર-ઓરિએન્ટેડ પ્રોગ્રામિંગ (POP) અને ઑબ્જેક્ટ-ઓરિએન્ટેડ પ્રોગ્રામિંગ (OOP) વચ્ચે તફાવત કરો.	૦૩
	(b)	List different features of Java. Explain any four Features of Java.	**04**
	(બ)	જાવાના વિવિધ લક્ષણોની યાદી બનાવો. જાવાની કોઈપણ ચાર વિશેષતાઓ સમજાવો.	૦૪
	(c)	Write a program in java to find out maximum from any ten numbers using command line argument.	**07**
	(ક)	કમાન્ડ લાઈન આર્ગુમેન્ટનો ઉપયોગ કરીને કોઈપણ દસ સંખ્યાઓમાંથી મહત્તમ શોધવા માટે જાવામાં પ્રોગ્રામ લખો.	૦૭
		OR	
	(c)	Write a Java program to find out prime number between 1 to 200.	**07**
	(ક)	1 થી 200 ની વચ્ચે અવિભાજ્ય સંખ્યા શોધવા માટે જાવા પ્રોગ્રામ લખો.	૦૭
Q.2	**(a)**	Explain Garbage collection in java.	**03**
પ્રશ્ન.2	**(અ)**	જાવામાં ગાર્બેજ કલેક્શન સમજાવો.	**૦૩**
	(b)	Explain final keyword with example.	**04**
	(બ)	ફાઈનલ કીવર્ડ ઉદાહરણ સાથે સમજાવો.	૦૪
	(c)	Define Inheritance. List out types of it. Explain multilevel and hierarchical inheritance with suitable example.	**07**
	(ક)	ઇનહેરીટંન્સને વ્યાખ્યાયિત કરો. તેના પ્રકારોની યાદી બનાવો. મલ્ટિલેવલ અને હાયરાર્કિકલ ઇનહેરીટંન્સને યોગ્ય ઉદાહરણ સાથે સમજાવો.	૦૭
		OR	
Q.2	**(a)**	Explain the Java Program Structure with example.	**03**
પ્રશ્ન.2	(અ)	ઉદાહરણ સાથે Java પ્રોગ્રામ સ્ટ્રક્ચર સમજાવો.	૦૩
	(b)	Explain super keyword with example.	**04**
	(બ)	ઉદાહરણ સાથે સુપર કીવર્ડ સમજાવો.	૦૪
	(c)	Define: Constructor. List out types of it. Explain Parameterized and copy constructor with suitable example.	**07**

	(ક)	વ્યાખ્યાયિત કરો: કન્સ્ટ્રકટર. તેના પ્રકારોની યાદી બનાવો. પેરામીટરાઇઝ્ડ અને કોપી કન્સ્ટ્રકટરને યોગ્ય ઉદાહરણ સાથે સમજાવો.	૦૭
Q. 3	**(a)**	Explain static keyword with example.	**03**
પ્રશ્ન.3	(અ)	`Static` કીવર્ડ ઉદાહરણ સાથે સમજાવો.	૦૩
	(b)	Explain abstract class with suitable example.	**04**
	(બ)	યોગ્ય ઉદાહરણ સાથે `abstract class` સમજાવો.	૦૪
	(c)	Define: Thread. List different methods used to create Thread. Explain Thread life cycle in detail.	**07**
	(ક)	વ્યાખ્યાયિત કરો: થ્રેડ. થ્રેડ બનાવવા માટે ઉપયોગમાં લેવાતી વિવિધ પધ્ધતિઓની સૂચિ બનાવો. થ્રેડ લાઈફ સાઇકલને વિગતવાર સમજાવો.	૦૭
		OR	
Q. 3	**(a)**	Explain Type Conversion and Casting in java.	**03**
પ્રશ્ન.3	(અ)	જાવામાં ટાઈપ કન્વર્ઝન અને કાસ્ટિંગ સમજાવો.	૦૩
	(b)	Explain different access controls in Java.	**04**
	(બ)	જાવામાં વિવિધ એક્સેસ કંટ્રોલ સમજાવો.	૦૪
	(c)	Explain Synchronization in Thread with suitable example.	**07**
	(ક)	થ્રેડમાં સિંક્રોનાઇઝેશનને યોગ્ય ઉદાહરણ સાથે સમજાવો.	૦૭
Q. 4	**(a)**	Differentiate between String class and String Buffer class.	**03**
પ્રશ્ન.4	(અ)	સ્ટ્રિંગ ક્લાસ અને સ્ટ્રિંગ બફર ક્લાસ વચ્ચે તફાવત કરો.	૦૩
	(b)	Explain multiple catch with suitable example.	**04**
	(બ)	યોગ્ય ઉદાહરણ સાથે બહુવિધ કેચ સમજાવો.	૦૪
	(c)	What is interface? Explain multiple inheritance with example.	**07**
	(ક)	ઈન્ટરફેસ શું છે? ઉદાહરણ સાથે મલ્ટીપલ ઇનહેરીટેન્સ સમજાવો.	૦૭
		OR	
Q. 4	**(a)**	Explain the Primitive Data Types and User Defined Data Types.	**03**
પ્રશ્ન.4	(અ)	`Primitive Data` પ્રકારો અને `User Defined` ડેટા પ્રકારો સમજાવો.	૦૩
	(b)	Write a Java Program to find sum and average of 10 numbers of an array.	**04**
	(બ)	એરેની 10 સંખ્યાઓનો સરવાળો અને સરેરાશ શોધવા માટે જાવા પ્રોગ્રામ લખો.	૦૪
	(c)	Write a program to explain method overloading and method overriding.	**07**
	(ક)	મેથડ ઓવરલોડિંગ અને મેથડ ઓવરરાઇડિંગ સમજાવવા માટે એક પ્રોગ્રામ લખો.	૦૭
Q.5	**(a)**	Explain Divide by Zero Exception in Java with example.	**03**
પ્રશ્ન.5	(અ)	`Java`માં `Divide by Zero Exception`ને ઉદાહરણ સાથે સમજાવો.	૦૩
	(b)	Explain throw and finally in Exception Handling with example.	**04**
	(બ)	થ્રો અને ફાઇનલી એક્સેપ્શન હેન્ડલિંગ ઉદાહરણ સાથે સમજાવો.	૦૪
	(c)	Write a java program to create a text file and perform read operation on the text file.	**07**
	(ક)	ટેક્સ્ટ ફાઇલ બનાવવા માટે જાવા પ્રોગ્રામ લખો અને ટેક્સ્ટ ફાઇલ પર રીડ ઑપરેશન કરો.	૦૭
		OR	
Q.5	**(a)**	Explain Array Index Out Of Bound Exception in Java with example.	**03**
પ્રશ્ન.5	(અ)	`Java`માં એરે ઈન્ડેક્સ આઉટ ઓફ બાઉન્ડ એક્સેપ્શનને ઉદાહરણ સાથે સમજાવો.	૦૩

(b)	Explain the different I/O Classes available with Java.	**04**
(બ)	`Java` સાથે ઉપલબ્ધ વિવિધ `I/O` કલાસીસ સમજાવો.	૦૪
(c)	Write a java program to create a text file and perform write operations on a Text File.	**07**
(ક)	ટેક્સ્ટ ફાઇલ બનાવવા માટે જાવા પ્રોગ્રામ લખો અને ટેક્સ્ટ ફાઇલ પર વ્રાઈટ ઑપરેશન કરો.	૦૭

Seat No. / Enrolment No.:

GUJARAT TECHNOLOGICAL UNIVERSITY

Diploma Engineering – SEMESTER – 4 (NEW) – EXAMINATION – Summer-2024

Subject Code: 4343203 **Date: 15-06-2024**

Subject Name: Java Programming

Time: 02:30 PM TO 05:00 PM **Total Marks: 70**

Instructions:

1. **Attempt all questions.**
2. **Make Suitable assumptions wherever necessary.**
3. **Figures to the right indicate full marks.**
4. **Use of programmable & Communication aids are strictly prohibited.**
5. **Use of non-programmable scientific calculator is permitted.**
6. **English version is authentic.**

			Marks
Q.1	**(a)**	Explain Garbage collection in java.	**03**
પ્રશ્ન.1	**(**અ)	જાવામાં Garbage collection સમજાવો.	૦૩
	(b)	Explain JVM in detail.	**04**
	(બ)	JVM ને વિગતવાર સમજાવો.	૦૪
	(c)	Write a program in java to print Fibonacci series for N terms.	**07**
	(ક)	Fibonacci series પ્રિન્ટ કરવા માટે નો જાવા પ્રોગ્રામ લખો.	૦૭
		OR	
	(c)	Write a program in java to find out minimum from any ten numbers using command line argument.	**07**
	(ક)	કમાન્ડ લાઇન arguments નો ઉપયોગ કરીને કોઈપણ દસ સુંખ્યાઓ માંથી ન્યૂનતમ શોધવા માટે જાવા પ્રોગ્રામ લખો.	૦૭
Q.2	**(a)**	List out basic concepts of Jana OPP. Explain any one in details	**03**
પ્રશ્ન.2			
	(b)	Explain final keyword with example.	**04**
	(બ)	final કી-વર્ડ ઉદાહરણ સાથે સમજાવો.	૦૪
	(c)	What is constructor? Explain parameterized constructor with example.	**07**
	(ક)	કન્સ્ટ્રકટર શું છે? Parameterized કન્સ્ટ્રકટર ને ઉદાહરણ સાથે સમજાવો	૦૭
		OR	
Q.2	**(a)**	Explain the Java Program Structure with example.	**03**
પ્રશ્ન.**2**	**(**અ)	ઉદાહરણ સાથે જાવા પ્રોગ્રામ સ્ટ્રક્ચર સમજાવો.	૦૩
	(b)	Explain static keyword with suitable example.	**04**
	(બ)	યોગ્ય ઉદાહરણ સાથે static કી-વર્ડ સમજાવો.	૦૪
	(c)	Define Inheritance. List out types of it. Explain multilevel and hierarchical inheritance with suitable example.	**07**
	(ક)	ઇનહેરીટન્સ વ્યાખ્યાયિત કરો. તેના પ્રકારોની યાદી બનાવો. Multilevel અને Hierarchical ઇનહેરીટન્સ ને યોગ્ય ઉદાહરણ સાથે જાવો	૦૭
Q. 3	**(a)**	Explain this keyword with suitable example.	**03**

પ્રશ્ન.3	(અ)	this કી-વર્ડને યોગ્ય ઉદાહરણ સાથે જાવો.	૦૩
	(b)	Explain different access controls in Java.	**04**
	(બ)	જાવામાં વિવિધ એક્સેસ કંટ્રોલ સમજાવો.	૦૪
	(c)	What is interface? Explain multiple inheritance using interface with example.	**07**
	(ક)	ઈન્ટરફેસ શું છે? ઈન્ટરફેસ દ્વારા ઉદાહરણ સાથે multiple inheritance સમજાવો.	૦૭
		OR	
Q. 3	**(a)**	Explain super keyword with example.	**03**
પ્રશ્ન.3	(અ)	super કી-વર્ડ ઉદાહરણ સાથે સમજાવો.	૦૩
	(b)	What is package? Write steps to create a package and give example of it.	**04**
	(બ)	પેકેજ શું છે? પેકેજ બનાવવાના પગલાં લખો અને તેનું ઉદાહરણ આપો.	૦૪
	(c)	Define: Method Overriding. List out Rules for method overriding. Write a java program that implements method overriding.	**07**
	(ક)	વ્યાખ્યાયિત કરો: થ્રેડ ઓવરરાઇડિંગ. થ્રેડ ઓવરરાઇડિંગ માટેના નિયમોની યાદી બનાવો તથા એક જાવા પ્રોગ્રામ લખો જે થ્રેડ ઓવરરાઇડિંગને ઇમ્પલેમેન્ટ કરે.	૦૭
Q. 4	**(a)**	Explain abstract class with suitable example.	**03**
પ્રશ્ન.4	(અ)	યોગ્ય ઉદાહરણ સાથે abstract class સમજાવો	૦૩
	(b)	What is Thread? Explain Thread life cycle.	**04**
	(બ)	થ્રેડ શું છે? થ્રેડ જીવનચક્ર સમજાવો.	૦૪
	(c)	Write a program in java that creates the multiple threads by implementing the Thread class.	**07**
	(ક)	જાવામાં એક પ્રોગ્રામ લખો જે Thread Class નો અમલ કરીને બહુવિધ થ્રેડો બનાવે છે.	૦૭
		OR	
Q. 4	**(a)**	Explain final class with suitable example.	**03**
પ્રશ્ન.4	(અ)	યોગ્ય ઉદાહરણ સાથે final class સમજાવો.	૦૩
	(b)	Explain thread priorities with suitable example.	**04**
	(બ)	યોગ્ય ઉદાહરણ સાથે thread ની પ્રાથમિકતાઓ સમજાવો.	૦૪
	(c)	What is Exception? Write a program that shows the use of Arithmetic Exception.	**07**
	(ક)	Exception શું છે? Arithmetic Exception નો ઉપયોગ દર્શાવતો પ્રોગ્રામ લખો.	૦૭
Q.5	**(a)**	Write a Java Program to find sum and average of 10 numbers of an array.	**03**
પ્રશ્ન.5	(અ)	એરેની 10 સંખ્યાઓનો સરવાળો અને સરેરાશ શોધવા માટેનો જાવા પ્રોગ્રામ લખો.	૦૩
	(b)	Write a Java program to handle user defined exception for 'Divide by Zero' error.	**04**
	(બ)	'DivideByZero' એરર માટે યુઝર ડિફાઈન્ડ Exception હેન્ડલ કરવા માટે જાવા પ્રોગ્રામ લખો.	૦૪
	(c)	Write a java program to create a text file and perform read operation on the text file	**07**
	(ક)	ટેક્સ્ટ ફાઇલ બનાવવા માટે જાવા પ્રોગ્રામ લખો અને ટેક્સ્ટ ફાઇલ પર રીડ ઑપરેશન કરો.	૦૭
		OR	
Q.5	**(a)**	Explain java I/O process.	**03**

પ્રશ્ન.5	(અ)	Java I/O પ્રક્રિયા સમજાવો.	૦૩
	(b)	Explain throw and finally in Exception Handling with example.	**04**
	(બ)	Exception હેન્ડલિંગમાં Throw અને finally કી-વર્ડ ઉદાહરણ સાથે જાવો.	૦૪
	(c)	Write a java program to display the content of a text file and perform append operation on the text file.	**07**
	(ક)	ટેક્સ્ટ ફાઇલ ના કન્ટેન્ટ ડિસ્પ્લે કરવા અને ટેક્સ્ટ ફાઇલ પર એપેન્ડ ઓપરેશન કરવા માટે જાવા પ્રોગ્રામ લખો કરો.	૦૭

****

Seat No. / Enrolment No.:

GUJARAT TECHNOLOGICAL UNIVERSITY

Diploma Engineering – SEMESTER – 4 (NEW) – EXAMINATION – Winter-2024

Subject Code: 4343203 **Date: 22-11-2024**

Subject Name: Java Programming

Time: 02:30 PM TO 05:00 PM **Total Marks: 70**

Instructions:

1. **Attempt all questions.**
2. **Make Suitable assumptions wherever necessary.**
3. **Figures to the right indicate full marks.**
4. **Use of simple calculators and non-programmable scientific calculators are permitted**
5. **English version is authentic.**

			Marks
Q.1	**(a)**	List out various Primitive data types in Java.	**03**
પ્રશ્ન.1	**(**અ)	Java ના વિવિધ પ્રકારના Primitive data typeની યાદી આપો.	૦૩
	(b)	Explain Structure of Java Program with suitable example.	**04**
	(બ)	યોગ્ય ઉદાહરણ સાથે Java Programનું સ્ટ્રક્ચર સમજાવો.	૦૪
	(c)	List arithmetic operators in Java. Develop a Java program using any three arithmetic operators and show the output of program.	**07**
	(ક)	Java ના arithmetic operatorsની યાદી આપો. કોઈ પણ ત્રણ arithmetic operatorsનો ઉપયોગ કરીને Java Program વિકસાવો અને તેનું output બતાવો.	૦૭
		OR	
	(c)	Write syntax of Java for loop statement. Develop a Java program to find out prime number between 1 to 10.	**07**
	(ક)	Javaમાં for લુપ માટેની સિન્ટેક્ષ લખો. ૧ થી ૧૦ વચ્ચે આવતા પ્રાઈમ નંબર શોધવા માટેનો java કોડ વિકસાવો.	૦૭
Q.2	**(a)**	List the differences between Procedure-Oriented Programming (POP) and Object-Oriented Programming (OOP).	**03**
પ્રશ્ન.2	**(અ)**	Procedure-Oriented Programming (POP) અને Object-Oriented Programming (OOP) ના તફાવતોની યાદી આપો.	**૦૩**
	(b)	Explain ***static*** keyword with example.	**04**
	(બ)	યોગ્ય ઉદાહરણ સાથે ***static*** કીવર્ડ સમજાવો.	૦૪
	(c)	Define Constructor. List types of Constructors. Develop a java code to explain Parameterized constructor.	**07**
	(ક)	Constructorની વ્યાખ્યા આપો. Constructorના વિવિધ પ્રકારોની યાદી આપો. Parameterized constructor સમજાવવા માટેનો java code વિકસાવો.	૦૭

		OR	
Q.2	**(a)**	List the basic OOP concepts in Java and explain any one.	**03**
પ્રશ્ન.2	**(**અ)	java મા મૂળભૂત OOP conceptsની યાદી આપો અને કોઈ પણ એક સમજાવો.	૦૩
	(b)	Explain ***final*** keyword with example.	**04**
	(બ)	યોગ્ય ઉદાહરણ સાથે ***final*** કીવર્ડ સમજાવો.	૦૪
	(c)	Write scope of java access modifier. Develop a java code to explain **public** modifier	**07**
	(ક)	java access modifierમાટેનો scope લખો. **public** modifier સમજાવવા માટેનો java code વિકસાવો.	૦૭
Q. 3	**(a)**	List out different types of inheritance and explain any one with example.	**03**
પ્રશ્ન.3	**(**અ)	વિવિધ પ્રકારના inheritance ની યાદી આપો અને કોઈ પણ એક ઉદાહરણ સાથે સમજાવો.	૦૩
	(b)	Explain any two String buffer class methods with suitable example.	**04**
	(બ)	કોઈ પણ બે String buffer class methods યોગ્ય ઉદાહરણ સાથે સમજાવો.	૦૪
	(c)	Define Interface. Write a java program to demonstrate multiple inheritance using interface.	**07**
	(ક)	Interfaceની વ્યાખ્યા આપો. Interfaceની મદદ થી multiple inheritance નો java program લખો.	૦૭
		OR	
Q. 3	**(a)**	Give differences between Abstract class and Interface.	**03**
પ્રશ્ન.3	**(**અ)	Abstract class અને Interface નો તફાવત આપો.	૦૩
	(b)	Explain any two String class methods with suitable example.	**04**
	(બ)	કોઈ પણ બે String class methods યોગ્ય ઉદાહરણ સાથે સમજાવો.	૦૪
	(c)	Explain package and list out steps to create package with suitable example.	**07**
	(ક)	Package સમજાવો અને package create કરવા માટેના સ્ટેપ્સની યાદી બનાવો.	૦૭
Q. 4	**(a)**	List types of errors in Java.	**03**
પ્રશ્ન.4	(અ)	java માં errorના પ્રકારોની યાદી આપો.	૦૩
	(b)	Explain try catch block with example.	**04**
	(બ)	try catch block યોગ્ય ઉદાહરણ સાથે સમજાવો.	૦૪
	(c)	List out any four differences between method overloading and overriding. Write a java code to explain method overriding.	**07**
	(ક)	method overloading અને overriding વચ્ચેના ચાર તફાવત આપો. method overriding સમજાવવા માટેનો java program લખો	૦૭
		OR	
Q. 4	**(a)**	List any four inbuilt exceptions.	**03**
પ્રશ્ન.4	(અ)	કોઈ પણ ચાર inbuilt exceptions ની યાદી આપો.	૦૩
	(b)	Explain "**throw**" keyword with suitable example.	**04**
	(બ)	યોગ્ય ઉદાહરણ સાથે "**throw**" કીવર્ડ સમજાવો.	૦૪
	(c)	Compare **'this'** keyword Vs **'Super'** keyword. Explain **super** keyword with suitable Example.	**07**
	(ક)	**'this'** કીવર્ડ **'Super'** કીવર્ડ સાથે સરખાવો. યોગ્ય ઉદાહરણ સાથે **super** કીવર્ડ સમજાવો.	૦૭
Q.5	**(a)**	List Different Stream Classes.	**03**

પ્રશ્ન.5	(અ)	વિવિધ Stream Classes ની યાદી આપો.	૦૩
	(b)	Write a java program to develop user defined exception for "Divide by zero" error.	**04**
	(બ)	'Divide by Zero' એરર માટે યુઝર ડીફાઇન એક્સેપ્સન હેન્ડલ કરવા માટે જાવા પ્રોગ્રામ લખો.	૦૪
	(c)	Write a program in Java that reads the content of a file byte by byte and copy it into another file.	**07**
	(ક)	જાવામાં એક પ્રોગ્રામ લખો જે બાઈટ બાય બાઈટ ફાઈલના કન્ટેન્ટ વાંચે અને તેને બીજી ફાઇલ માં કોપી કરે.	૦૭
		OR	
Q.5	**(a)**	List different file operations in Java.	**03**
પ્રશ્ન.5	(અ)	javaના વિવિધ file operationsની યાદી આપો.	૦૩
	(b)	Write a java program to explain finally block in exception handling.	**04**
	(બ)	એક્સેપ્સન હેન્ડલિંગ માં finally block સમજાવતો જાવા પ્રોગ્રામ લખો.	૦૪
	(c)	Write a java program to create a file and perform write operation on this file.	**07**
	(ક)	ફાઇલ ક્રિએટ કરવા અને તેમાં લખવા માટેનો જાવા પ્રોગ્રામ લખો	૦૭